**Final Year Design Project**

# Project Way




**By**

| | |
|---|---|
| **Muskan** | **076940** |
| **Nisha** | **076942** |
| **Aseema** | **077003** |
| **Project ID:** | **BSIT22-G22** |

***Under the supervision of***

**Prof. Usman Ahmad**

***Bachelor of Science in Information Technology (2022-2026)***

**FACULTY OF COMPUTING &**

**INFORMATION TECNOLOGY (FCIT)**

**GOVT.ISLAMIA GRADUATE COLLEGE, GUJRANWALA**

# Project Way

**A project presented to**

**University of the Punjab, Lahore**

**In Partial fulfilment**
**Of the requirement for the degree of**

***Bachelor of science in Information Technology (2022-2026)***

**By**


| | |
|---|---|
| **Muskan** | **076940** |
| **Nisha** | **076942** |
| **Aseema** | **077003** |

**FACULTY OF COMPUTING &**
**INFORMATION TECHNOLOGY (FCIT)**
**GOVT. ISLAMIA GRADUATE COLLEGE, GUJRANWALA**

# DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Signature: ----------

Muskan [076940]

Signature: ----------

Nisha [076942]

Signature: ----------

Aseema [077003]

# CERTIFICATE OF APPROVAL

It is certify that that final year design project (FYDP) of BSIT "Project Way" was developed by **Muskan (076940), Nisha (076942), and Aseema (077003)** under the supervision of **"Prof. Usman Ahmad"** in my opinion; it is fully adequate, in scope and quality for the degree of Bachelor in Science in Information Technology.

Signature: ----------

**FYDP Supervisor:**

**Signatures (Faculty Advisory Committee (FAC)**

| | FAC1 | FAC2 | FAC3 |
|---|---|---|---|
| Name | | | |
| Signature | | | |

Signature: ----------

**Head of FYDP Coordinator Office:**

Signature: ---------- Dated: ----------

**Chairperson, Department of Information Technology**

# Executive Summary

Many Teachers and Students face challenges in managing Final Year Project activities like selecting project ideas, Students groups, tracking tasks. Face-to-face interaction between teacher and student about project idea selection or rejection take a lot of time. So, it is a continuous circle until FYP idea selected. Students don't know how to track their progress, manage their tasks and draw diagrams.

Project Way is a web application that is built to help Admin, Teachers and Students to manage FYP activities easily. Teachers can upload project ideas, review ideas, manage Students in groups automatically, give feedback and view the progress of Students project. Students can view, select, submit project ideas to Teachers. Team members can easily schedule their tasks in real time. Students can interact with Teachers in case of any task issues through Help box and draw diagrams using Diagram Editor in single platform.

The system is built using MERN stack (MongoDB, Express.js, React and Node.js). MongoDB is used to save all the data such as Admin, Teachers and Students information, project ideas, project progress and feedbacks. Express.js is used to handle backend logic like login, registration, upload ideas, submit ideas, give feedback and will make sure that each user (Admin, Teachers and Students) can use their own features. React is a JavaScript library that creates frontend screens and users (Admin, Teachers and Students) will see and interact with it. Node.js is used to run server and help frontend and database to work together easily.

Our goal is to provide all users (Admin, Teachers and Students) a simple, clean and easy interface for managing FYP activities.

# Acknowledgement

All praise to the Almighty Allah the Most Merciful who blessed us with the skills, knowledge and patience to complete this project successfully. Untold blessings and peace be upon the greatest of mankind, The Holy Prophet Hazrat Muhammad (Sallallahu Alaihi Wasallam), who's timeless teaching's, lead us in the path of knowledge and wisdom. We always present our deepest gratitude to our parents who put every effort and pray for us and always presented us with their love, prayers and sacrifices all our lives. Their incredible drive to succeed has been our driving force during our four years of studying the BSIT.

We want to give our heartfelt thanks to Prof. Usman Ahmad, our Head of the Department of Information Technology & Computer Science and also our project supervisor, for his support, tolerance and high level of expertise which really helped us in making this project "Project Way" from the conception through its completion. His Leadership was essential both to the completion of the project, as well as to the fulfilment of the daring academic achievement that sparked off our journey towards a prestigious professional future.

We also give to our sincere thanks to Prof. M. Asif Afzal, for his continued guidance and for passing critical software engineering knowledge on to us that contributed greatly to writing the standard documentation and to apply the quality software development strategies in this project.

Finally, we also convey our gratitude to our institution namely the Govt. Islamia Graduate College, Gujranwala and the University of the Punjab in Lahore providing us with their valuable resources, plans and facilities on our way to our academic dreams.

Thank you.

Signature: ----------

Muskan [076940]

Signature: ----------

Nisha [076942]

Signature: ----------

Aseema [077003]

# Abbreviations

Table of all the abbreviations and acronyms used in FYP, along with their respective brief description. The abbreviation list in alphabetical order is given below:

| | |
|---|---|
| API | Application Programming Interface |
| BR | Business Requirement |
| CORS | Cross-Origin Resource Sharing |
| ERD | Entity Relationship Diagram |
| FYP | Final Year Project |
| FR | Functional Requirement |
| FT | Functional Testing |
| HTTP | Hyper Text Transfer Protocol |
| IT | Integration Testing |
| JSON | JavaScript Object Notation |
| JWT | Java Web Token |
| MERN | MongoDB, Express.js, React.js, Node.js |
| MVC | Model View Controller |
| NPM | Node Package Manager |
| OOP | Object Oriented Programming |
| PT | Performance Testing |
| RQ | Requirement |
| SDK | Software Development Kit |
| SMTP | Simple Mail Transfer Protocol |
| UC | Use Case |
| UT | Unit Testing |

# Table of Contents


1. Introduction ..... 2
   1.1 Problem Statement ..... 2
   1.2 Problem Solution ..... 2
   1.3 Objectives of the Proposed System ..... 2
   1.4 Scope ..... 2
   1.5 System Components ..... 2
      1.5.1 Module 1: Login/Registration ..... 3
      1.5.2 Module 2: Admin Dashboard ..... 3
      1.5.3 Module 3: Teacher Dashboard ..... 3
      1.5.4 Module 4: Student Dashboard ..... 3
      1.5.5 Module 5: Progress Tracking ..... 3
      1.5.6 Module 6: Diagram Editor ..... 4
      1.5.7 Module 7: Help Box ..... 4
      1.5.8 Module 8: Notification ..... 4
   1.6 Related System Analysis/Literature Review ..... 4
   1.7 Vision Statement ..... 4
   1.8 System Limitations and Constraints ..... 5
   1.9 Tools and Technologies ..... 5
   1.10 Project Deliverables ..... 6
   1.11 Project Planning ..... 6
   1.12 Summary ..... 7
2. Analysis ..... 9
   2.1 User Classes and Characteristics ..... 9
   2.2 Requirement Identifying Technique ..... 9
   2.3 Functional Requirements ..... 9
      2.3.1 Functional Requirement 1 ..... 10
      2.3.2 Functional Requirement 2 ..... 10
      2.3.3 Functional Requirement 3 ..... 11
      2.3.4 Functional Requirement 4 ..... 11
      2.3.5 Functional Requirement 5 ..... 12
      2.3.6 Functional Requirement 6 ..... 12

2.3.7 Functional Requirement 7 ..... 13
2.3.8 Functional Requirement 8 ..... 13
2.3.9 Functional Requirement 9 ..... 14
2.4 Non-Functional Requirements ..... 14
2.4.1 Reliability ..... 14
2.4.2 Usability ..... 14
2.4.3 Performance ..... 15
2.5 External Interface Requirements ..... 15
2.5.1 User Interface Requirements ..... 15
2.5.2 Software Requirements ..... 15
2.5.3 Hardware Requirements ..... 15
2.5.4 Communication Interfaces ..... 15
2.6 Use Case Analysis ..... 15
2.6.1 Use Case 1 ..... 16
2.6.2 Use Case 2 ..... 17
2.6.3 Use Case 3 ..... 17
2.6.4 Use Case 4 ..... 18
2.6.5 Use Case 5 ..... 19
2.6.6 Use Case 6 ..... 20
2.6.7 Use Case 7 ..... 21
2.6.8 Use Case 8 ..... 21
2.6.9 Use Case 9 ..... 22
2.7 Use Case Diagram ..... 23
2.7.1 Use Case Diagram 1 ..... 23
2.7.2 Use Case Diagram 2 ..... 24
2.7.3 Use Case Diagram 3 ..... 24
2.7.4 Use Case Diagram 4 ..... 25
2.8 Summary ..... 25
3. System Design ..... 27
3.1 Design Considerations ..... 27
3.2 Design Models ..... 28
3.2.1 Domain Model ..... 28

3.2.2 Class Diagram ..... 29
3.2.3 Sequence Diagram ..... 29
3.2.4 State Transition Diagram ..... 34
3.2.5 Entity Relationship (ER) Diagram ..... 35
3.3 Architectural Design ..... 36
3.3.1 UML Class Relationship Diagram ..... 36
3.3.2 UML Component Diagram ..... 36
3.3.3 Model View Controller (MVC) Diagram ..... 37
3.4 Data Design ..... 38
3.4.1 Data Dictionary ..... 38
3.5 User Interface Design ..... 41
3.5.1 Screen Images ..... 41
3.5.2 Screen Objects and Actions ..... 46
3.6 Behavioral Model ..... 49
3.7 Design Decisions ..... 54
3.8 Summary ..... 54
4. Implementation ..... 56
4.1 Algorithms ..... 56
4.2 External APIs/SDKs ..... 59
4.3 Code Repository ..... 60
4.4 Summary ..... 61
5. Testing and Evaluation ..... 63
5.1 Unit Testing ..... 63
5.2 Functional Testing. ..... 65
5.3 Integration Testing ..... 73
5.4 Performance Testing ..... 75
5.5 Summary ..... 76
6. System Conversion ..... 78
6.1 Conversion Method ..... 78
6.2 Deployment ..... 78
6.2.1 Deployment Steps ..... 78
6.2.2 Data Conversion ..... 79

6.2.3 Training ..... 79
6.3 Post Deployment Testing ..... 80
6.3.1 Post Deployment Testing Steps ..... 80
6.4 Challenges and Solutions ..... 81
6.5 Summary ..... 81
7. Introduction ..... 83
7.1 Evaluation ..... 83
7.2 Requirement Traceability Matrix ..... 83
7.3 Conclusion ..... 85
7.4 Future Work ..... 85
References ..... 86
A. Appendix-A: Use Case Description(Fully Dressed Format) ..... 88
B. Appendix-B: General Coding Standards & Guidelines ..... 97
C. Appendix-C: Application Prototype ..... 102

# List of Figures

Figure 1: Gantt Chart ..... 6
Figure 2: Use Case 1 ..... 23
Figure 3: Use Case 2 ..... 24
Figure 4: Use Case 3 ..... 24
Figure 5: Use Case 4 ..... 25
Figure 6: Domain Model Diagram ..... 28
Figure 7: Class Diagram ..... 29
Figure 8: Module 1 Sequence Diagram ..... 30
Figure 9: Module 2 Sequence Diagram ..... 30
Figure 10: Module 3 Sequence Diagram ..... 31
Figure 11: Module 4 Sequence Diagram ..... 31
Figure 12: Module 5 Sequence Diagram ..... 32
Figure 13: Module 6 Sequence Diagram ..... 32
Figure 14: Module 7 Sequence Diagram ..... 33
Figure 15: Module 8 Sequence Diagram ..... 33
Figure 16: State Transition Diagram ..... 34
Figure 17: Entity Relationship Diagram ..... 35
Figure 18: UML Class Relationship Diagram ..... 36
Figure 19: UML Component Diagram ..... 36
Figure 20: MVC Diagram ..... 37
Figure 21: Screen Image 1 ..... 41
Figure 22: Screen Image 2 ..... 42
Figure 23: Screen Image 3 ..... 42
Figure 24: Screen Image 4 ..... 43
Figure 25: Screen Image 5 ..... 43
Figure 26: Screen Image 6 ..... 44
Figure 27: Screen Image 7 ..... 44
Figure 28: Screen Image 8 ..... 45
Figure 29: Screen Image 9 ..... 45
Figure 30: Screen Image 10 ..... 46
Figure 31: Module 1 Sequence Diagram ..... 49
Figure 32: Module 2 Sequence Diagram ..... 49
Figure 33: Module 3 Sequence Diagram ..... 50
Figure 34: Module 4 Sequence Diagram ..... 50
Figure 35: Module 5 Sequence Diagram ..... 51
Figure 36: Module 6 Sequence Diagram ..... 51
Figure 37: Module 7 Sequence Diagram ..... 52
Figure 38: Module 8 Sequence Diagram ..... 52
Figure 39: State Transition Diagram ..... 53
Figure 40: Screen Image 1 ..... 102
Figure 41: Screen Image 2 ..... 102

Figure 42: Screen Image 3 ..... 103
Figure 43: Screen Image 4 ..... 103
Figure 44: Screen Image 5 ..... 104
Figure 45: Screen Image 6 ..... 104
Figure 46: Screen Image 7 ..... 105
Figure 47: Screen Image 8 ..... 105
Figure 48: Screen Image 9 ..... 106
Figure 49: Screen Image 10 ..... 106

# List of Tables

Table 1- 1: Related System Analysis with Proposed Solution ..... 4
Table 1- 2: Tools and Technologies ..... 5
Table 2- 1: User Classes and Characteristics of Proposed System ..... 9
Table 2- 2: Functional Requirement 1 ..... 10
Table 2- 3: Functional Requirement 2 ..... 10
Table 2- 4: Functional Requirement 3 ..... 11
Table 2- 5: Functional Requirement 4 ..... 11
Table 2- 6: Functional Requirement 5 ..... 12
Table 2- 7: Functional Requirement 6 ..... 12
Table 2- 8: Functional Requirement 7 ..... 13
Table 2- 9: Functional Requirement 8 ..... 13
Table 2- 10: Functional Requirement 9 ..... 14
Table 2- 11: Use Case 1 ..... 16
Table 2- 12: Use Case 2 ..... 17
Table 2- 13: Use Case 3 ..... 17
Table 2- 14: Use Case 4 ..... 18
Table 2- 15: Use Case 5 ..... 19
Table 2- 16: Use Case 6 ..... 20
Table 2- 17: Use Case 7 ..... 21
Table 2- 18: Use Case 8 ..... 21
Table 2- 19: Use Case 9 ..... 22
Table 3- 1: Alphabetical List of System Entities with Description ..... 38
Table 4- 1: Algorithm 1 ..... 56
Table 4- 2: Algorithm 2 ..... 56
Table 4- 3: Algorithm 3 ..... 57
Table 4- 4: Algorithm 4 ..... 57
Table 4- 5: Algorithm 5 ..... 58
Table 4- 6: Algorithm 6 ..... 59
Table 4- 7: Details of APIs used in this Project ..... 59
Table 5- 1: Unit Test Case 1 ..... 63
Table 5- 2: Unit Test Case 2 ..... 64
Table 5- 3: Unit Test Case 3: ..... 64
Table 5- 4: Functional Test Case 1 ..... 65
Table 5- 5: Functional Test Case 2 ..... 66
Table 5- 6: Functional Test Case 3 ..... 67
Table 5- 7: Functional Test Case 4 ..... 68
Table 5- 8: Functional Test Case 5 ..... 69
Table 5- 9: Functional Test Case 6 ..... 69
Table 5- 10: Functional Testing 7 ..... 70
Table 5- 11: Functional Test Case 8 ..... 71
Table 5- 12: Functional Test Case 9 ..... 72

Table 5- 13: Integration Test Case 1 ..... 73
Table 5- 14: Integration Test Case 2 ..... 74
Table 5- 15: Performance Test Case 1 ..... 75
Table 6- 1: Deployment Steps ..... 78
Table 6- 2: Data Conversion ..... 79
Table 6- 3: Training Instruction 1 ..... 79
Table 6- 4: Training Instruction 2 ..... 79
Table 6- 5: Training Instruction 3 ..... 80
Table 6- 6: Post Deployment Testing Steps ..... 80
Table 6- 7: Challenges and Solution ..... 81
Table 7- 1: Progress Table ..... 83
Table 7- 2: Requirement Traceability Matrix ..... 83
Table A- 1: Use Case 1 ..... 88
Table A- 2: Use Case 2 ..... 89
Table A- 3: Use Case 3 ..... 89
Table A- 4: Use Case 4 ..... 90
Table A- 5: Use Case 5 ..... 91
Table A- 6: Use Case 6 ..... 92
Table A- 7: Use Case 7 ..... 93
Table A- 8: Use Case 8 ..... 93
Table A- 9: Use Case 9 ..... 94




# Chapter 1: Introduction

# 1. Introduction

Many Teachers and Students face challenges in managing Final Year Project activities like selecting project ideas, Students groups, tracking tasks. Face-to-face communication between teacher and student about project idea selection or rejection take a lot of time. So, it is a continuous circle until FYP idea selected. Students don't know how to track their progress, manage their tasks and draw diagrams. This system will solve the problems like idea selection, task management, track progress and drawing diagrams.

## 1.1 Problem Statement

Teachers face challenges in managing project ideas and Students groups after project approval. Students lack a single platform that combine idea submission, task scheduling, tracking progress, communication and diagram creation for FYP.

## 1.2 Problem Solution

Project Way is a web application that is built to help Admin, Teachers and Students to manage FYP activities easily. Teachers can upload project ideas, review ideas, manage Students in groups automatically, give feedback and view the progress of Students project. Students can view, select, submit project ideas to Teachers. Team members can easily schedule their tasks in real time. Students can interact with Teachers in case of any task issues through Help box and draw diagrams using Diagram Editor in single platform. Our goal is to provide Teachers and Students simple and easy interface for managing FYP.

## 1.3 Objectives of the Proposed System

Our project has the following objectives:

1. Admin can manage Teachers and Students data securely.
2. Teachers upload/review ideas and track Students progress.
3. Automatically sort Students in groups after approval and display group members for easy representation.
4. Implement task scheduler for Students where all group members see same tasks assigned by the leader and track project progress.
5. Student communicate with teachers in case of any task issue through Help box.
6. Building a Diagram Editor for creating project diagrams.
7. Teachers and Students receive notifications.

## 1.4 Scope

Project Way is a web application where:

1. Teacher upload/review idea, give feedback online and track Students progress.
2. Students can submit project idea online.
3. Students can manage tasks, see their progress and communicate with Teachers.
4. Students use built-in Diagram Editor for drawing.

## 1.5 System Components

We are building a web application named as **"Project Way",** we have:

1. Client Web App (using React.js)
2. Admin Web App (using React.js with Admin Features)
3. Backend API Server (Node.js+Express.js)
4. Database (MongoDB)

**1. Client Web App (for Teacher and Students)**

- Teacher upload project ideas and templates.
- Teacher review, manage student groups and track progress.
- Student view and submit ideas.
- Student track progress, use Diagram Editor and Help Box.

**2. Admin Web App**

- Admin manage Teachers and Students.
- Admin assign roles and permissions.

**3. Backend API Server (Node.js+Express.js)**

- Restful APIs for all client interactions.
- User authentication and authorization.
- Secure role-based access control.

**4. Database (MongoDB)**

- Store user information (Admin, Teachers and Students).
- Stores all data such as project ideas, tasks, groups, diagrams and issues.

### 1.5.1 Module 1: Login/Registration

- Admin register Teachers and Students with their name and email.
- Teachers and Students log in with name, email and password.
- System assign each user to the correct dashboard according to their role.

### 1.5.2 Module 2: Admin Dashboard

- Admin register, view, update and delete Teachers and Students data.
- Admin assign roles and permission to each user.

### 1.5.3 Module 3: Teacher Dashboard

- Teachers upload project ideas and templates.
- Teachers review project ideas and manage student groups.
- Teachers track each Student group progress.

### 1.5.4 Module 4: Student Dashboard

- Students see project ideas and templates shared by Teachers.
- Students submit ideas and get feedback from Teachers.

### 1.5.5 Module 5: Progress Tracking

- Leader divides tasks among their team members with title and deadline.
- Each member tracks their progress individually and also whole group progress.
- Teachers see each group progress.

### 1.5.6 Module 6: Diagram Editor

- Built-in Diagram Editor for drawing diagrams (like flow charts, use case etc.)
- Students can save and download diagrams.

### 1.5.7 Module 7: Help Box

- Students and Teachers communicate with each other in case of any issue.

### 1.5.8 Module 8: Notification

- Teachers will receive notification when Students submit project ideas and issues.
- Students will receive notification when Teachers give feedbacks.

## 1.6 Related System Analysis/Literature Review

We visit some applications that provides some similar functionality that our project provides. But where these applications provide benefits it also has some weaknesses. Also see how our project fill the gap. See below table:

*Table 1- 1: Related System Analysis with Proposed Solution*

| Application Name | Weakness | Proposed Project Solution |
|---|---|---|
| **ClickUp** | It has very complex interface for beginners and too many features make it confusing and hard to understand. | Project Way provides a simple and easy-to-use interface for beginners to understand and use it easily. |
| **Microsoft Teams** | It focuses on communication such as chats and meetings but not provide project tracking and feedback. | Project Way provides project tracking, feedback system and Teachers can monitor group progress easily. |
| **Trello** | It is useful for task management but it does not provide proper communication between Teachers and Students in an academic environment. | Project Way provides special environment for FYP Students where Teachers and Students communicate with each other about FYP activities. |

## 1.7 Vision Statement

**For [**Students and Teachers involved in Final Year Projects**]**

**Who [**need an easy and organized way to manage project ideas, tasks, diagrams and feedback**]**

**The [**Project Way**]**

**Is [**a Web based FYP management system**]**

**That [**provides role-based dashboards for Admin, Teachers and Students with idea submission, progress tracking, diagram editor, help box in one place**]**

**Unlike [**manual paper-based processes, scattered tools**]**

**Our product [**Offers an integrated user-friendly platform that automates workflows, improves communication and helps teachers to monitor FYP students**]**

## 1.8 System Limitations and Constraints

### i. Limitations:

1. Offline access is not possible.
2. It only supports small files for Student project ideas/template upload by Teachers.
3. Not provides Students instant response from Teachers.

### ii. Constraints:

1. The system must be developed using the MERN stack.
2. Access is restricted to three roles: Admin, Teachers and Students.
3. User Registration requires mandatory email verification.
4. This application is a web-based platform and requires an active internet connection.

## 1.9 Tools and Technologies

*Table 1- 2: Tools and Technologies*

| | **Tools** | **Versions** | **Rationale** |
|---|---|---|---|
| **Tools, Libraries and Technologies** | Visual Studio Code | 1.104.3 | Easy Coding & Editing |
| | Git & GitHub | Latest | Code Sharing |
| | **Libraries** | **Version** | **Rationale** |
| | Mongoose | 8.19.3 | Help Node.js & MongoDB Easily |
| | Axios | 1.13.2 | React with Backend API |
| | **Technologies** | **Version** | **Rationale** |
| | MongoDB | 1.46.11 | Store User & Project Data |
| | Node.js | 22.14.6 | Run Backend JS Code |

| | | | |
|---|---|---|---|
| | Express.js | 5.1.0 | Handle API & Server Request |
| | React | 19.2.0 | Build Interactive Fronted |

## 1.10 Project Deliverables

Our project will have following deliverables by the end of 7th semester:

1. Project Proposal
2. Software Requirement Specifications
3. Software Design Specifications
4. Prototype
5. Comprehensive Evaluation Document (Proposal, SRS, SDS)

Our project will have following deliverables by the end of 8th semester:

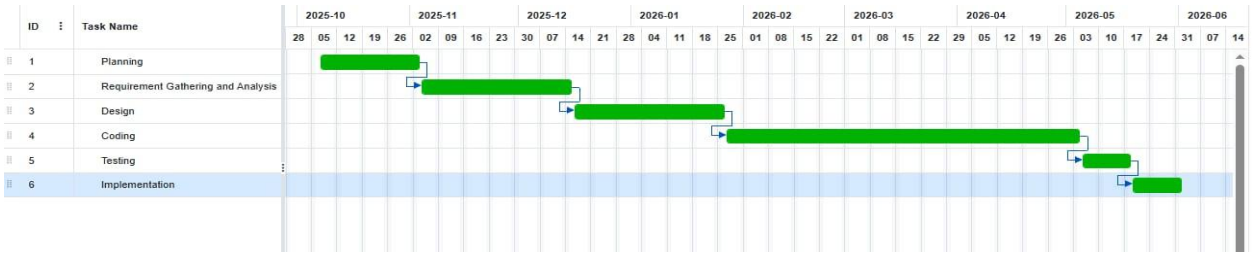
1. Implementation
2. Testing
3. Deployment
4. Final Report
5. Presentation & Final Evaluation

## 1.11 Project Planning

Gantt chart representing the plans of carrying out the project activities, representing the milestones as well as resources required for each task:

*Figure 1: Gantt Chart*

| ID | Task Name | Start | End | Duration | Progress % | Dependency |
|---|---|---|---|---|---|---|
| 1 | Planning | 2025-10-07 | 2025-11-03 | 20 days | 100 | |
| 2 | Requirement Gathering and Analysis | 2025-11-04 | 2025-12-15 | 30 days | 100 | 1FS |
| 3 | Design | 2025-12-16 | 2026-01-26 | 30 days | 100 | 2FS |
| 4 | Coding | 2026-01-27 | 2026-05-04 | 70 days | 100 | 3FS |
| 5 | Testing | 2026-05-05 | 2026-05-18 | 10 days | 100 | 4FS |
| 6 | Implementation | 2026-05-19 | 2026-06-01 | 10 days | 100 | 5FS |



## 1.12 Summary

This chapter introduces the Project Way system that helps Teachers and Students to manage FYP activities easily. It explains the problem, the web-based solution, project goals, scope and system modules. It compares existing platforms, outlines tools and technologies and defines deliverables. Overall, it sets the foundation for building a Project Way for FYP Students.




# Chapter 2: Requirement Analysis

# 2. Analysis

This Chapter focuses on requirement identification, gathering and analysis. The users categorized into three classes. For requirement identification and elicitation, open interviews, observation and use case analysis are used. Also, identified requirements are categorized into functional and non-functional requirements, along with their use case description and diagram.

## 2.1 User Classes and Characteristics

The Project Way will have three user classes (Admin, Teacher and Student):

*Table 2- 1: User Classes and Characteristics of Proposed System*

| User Class | Characteristics |
|---|---|
| **Admin** | The Admin manages all users and access. It creates Teacher and Student accounts, assign roles and permissions. |
| **Teacher** | The Teacher uploads project ideas/templates, review ideas, manage Students groups, track progress and send feedback on Students issues. |
| **Student** | The Student view project ideas/templates, submit project ideas, create groups by adding members, assign tasks, track progress, write issues, get feedback and use Diagram Editor. |

## 2.2 Requirement Identifying Technique

In this project, we have used the **Use Case Model** as our primary technique for identifying functional requirements. This approach is effective for interactive end-user applications ensuring a clear understanding of system functionalities and interactions. In this, we can explain the roles and their interactions with the system.

The Use Case Model includes:

- **Use Case Diagram**: A graphical representation of system functionalities and interactions between users and the system.
- **Use Case Analysis**: A detailed description of each use case in term of its stakeholders, preconditions, post conditions and steps performed in it.

## 2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This action is organized by features as a system feature name and specific functional requirements associated with this feature. So, for each module functional requirements given below:

### 2.3.1 Functional Requirement 1

*Table 2- 2: Functional Requirement 1*

| | |
|---|---|
| **Identifier** | FR-1 |
| **Title** | Email-Password Authentication |
| **Requirement** | Admin, Teacher and Student shall be able to log in by entering email and password and then clicking Log In. |
| **Source** | Business Requirement BR010 |
| **Rationale** | Provide secure login and protected access to Admin, Teacher and Student panels. |
| **Business Rule** | The system shall not allow access to any protected page without a valid login session. |
| **Dependencies** | None (base requirement) |
| **Priority** | High |

### 2.3.2 Functional Requirement 2

*Table 2- 3: Functional Requirement 2*

| | |
|---|---|
| **Identifier** | FR-2 |
| **Title** | Manage Users and Roles (Admin) |
| **Requirement** | The Admin shall be able to create Teacher and Student accounts, assign roles and set permissions for different modules. |
| **Source** | Business Requirement BR001 |
| **Rationale** | Control who can use which features and keep the system secure and organized. |
| **Business Rule** | Only Admin users shall be allowed to create or change roles and permissions. |
| **Dependencies** | FR-1 |
| **Priority** | High |

### 2.3.3 Functional Requirement 3

*Table 2- 4: Functional Requirement 3*

| | |
|---|---|
| **Identifier** | FR-3 |
| **Title** | Upload Project Ideas and Templates (Teacher) |
| **Requirement** | 1. The Teacher shall be able to upload project ideas and templates file by clicking Upload.<br>2. Students shall be able to view these ideas and templates later. |
| **Source** | Business Requirements BR002, BR005 |
| **Rationale** | Give students clear project options and standard documents to follow. |
| **Business Rule** | Only logged-in Teachers shall be able to upload, edit or delete project ideas and templates. |
| **Dependencies** | FR-1, FR-2 |
| **Priority** | High |

### 2.3.4 Functional Requirement 4

*Table 2- 5: Functional Requirement 4*

| | |
|---|---|
| **Identifier** | FR-4 |
| **Title** | Submit Project Idea (Student) |
| **Requirement** | 1. The Student shall be able to fill a form with project title, description, team details and submit it to the Teacher.<br>2. The Student shall be able to see the status of the idea (Accepted or Rejected). |
| **Source** | Business Requirements BR006 |
| **Rationale** | Allow students to propose projects and get approval from teachers. |
| **Business Rule** | A student group shall have only one active main project idea at a time. |
| **Dependencies** | FR-1, FR-3 |
| **Priority** | High |

### 2.3.5 Functional Requirement 5

*Table 2- 6: Functional Requirement 5*

| | |
|---|---|
| **Identifier** | FR-5 |
| **Title** | Group and Task Management (Student Leader) |
| **Requirement** | 1. The Team Leader shall be able to create a group by adding members, create tasks with title, description and due date and assign tasks to group members.<br>2. All group members shall be able to view their tasks and see a progress bar for group completion. |
| **Source** | Business Requirement BR007 |
| **Rationale** | Help student groups plan work and track progress in one place. |
| **Business Rule** | Only the group leader shall be allowed to create, assign or delete tasks for that group. |
| **Dependencies** | FR-1, FR-4, FR-2 |
| **Priority** | Medium to High |

### 2.3.6 Functional Requirement 6

*Table 2- 7: Functional Requirement 6*

| | |
|---|---|
| **Identifier** | FR-6 |
| **Title** | Project Progress Tracking (Teacher and Student) |
| **Requirement** | 1. The teacher shall be able to open a progress view for each student group and see each group whole progress.<br>2. The students shall also be able to view their progress. |
| **Source** | Business Requirement BR004, BR0007 |
| **Rationale** | Allow teachers and students to monitor performance. |
| **Business Rule** | Teacher shall only see the progress of groups that they are accepted. |
| **Dependencies** | FR-5, FR-2 |
| **Priority** | Medium |

### 2.3.7 Functional Requirement 7

*Table 2- 8: Functional Requirement 7*

| | |
|---|---|
| **Identifier** | FR-7 |
| **Title** | Help Box (Issue submission and feedback) |
| **Requirement** | 1. The student shall be able to write issues about task in help box and submit to teacher.<br>2. The Teacher shall be able to read the issue and send feedback. |
| **Source** | Business Requirement BR008 |
| **Rationale** | Provide a simple communication channel between students and teachers inside Project Way. |
| **Business Rule** | Issues and replies shall be stored and viewable later by both student and teacher. |
| **Dependencies** | FR-1, FR-2, FR-4, FR-5 |
| **Priority** | Medium |

### 2.3.8 Functional Requirement 8

*Table 2- 9: Functional Requirement 8*

| | |
|---|---|
| **Identifier** | FR-8 |
| **Title** | Diagram Editor Tool (Student) |
| **Requirement** | The Student shall be able to open the Diagram Editor, create diagrams using shapes and connectors, save diagrams in the system and download them as an image. |
| **Source** | Business Requirement BR009 |
| **Rationale** | Let students design and export diagrams (UML, ERD, etc.) directly inside the project system. |
| **Business Rule** | Diagrams shall be linked to the Students project and shall be editable later by the Student. |
| **Dependencies** | FR-1 |

| | |
|---|---|
| **Priority** | Medium |

### 2.3.9 Functional Requirement 9

*Table 2- 10: Functional Requirement 9*

| | |
|---|---|
| **Identifier** | FR-9 |
| **Title** | Notification System |
| **Requirement** | 1. The system shall send notifications to Teachers and Students new idea submission, idea approval/rejection or issue reply.<br>2. Users shall be able to view notifications in their dashboard. |
| **Source** | Business Requirement BR011 (linked to BR002, BR006, BR007, BR008) |
| **Rationale** | Keep all users updated about important changes without manually checking each page. |
| **Business Rule** | Notification delivery should occur within a few seconds after the event is stored and the system shall mark notifications as read/unread. |
| **Dependencies** | FR-1, FR-2, FR-3, FR-4, FR-5, FR-7 |
| **Priority** | High |

## 2.4 Non-Functional Requirements

The Non-Functional Requirements outlines the quality attributes of the system such as performance, security, scalability and usability, which support the functional requirements specified above. These requirements are non-technical but essential for ensuring the system overall effectiveness. They should be clearly defined, measurable and verifiable to ensure the system meets the expectations for reliability, efficiency and user experience.

### 2.4.1 Reliability

Project Way should run without frequent crashes or data loss during normal use. Data such as project ideas, tasks, groups, issues and diagrams should stay safe even if the server restarts. If an error happens (for example: database not reachable), the system should show a clear message and not save data.

### 2.4.2 Usability

The system should be easy to learn for Admin, Teacher and Student with simple menus and clear button names. Common actions like login, submit idea, create task, open diagram editor

should be possible in few clicks and without complex steps. Error messages should be simple and tell the user what went wrong and how to fix it (e.g., “Title is required”).

### 2.4.3 Performance

The login page and main dashboards should load in 3-5 seconds on a normal internet connection. The Diagram Editor should react quickly when the user drags, drops or moves shapes. The system should still work properly when up to 30 users are using it at the same time.

## 2.5 External Interface Requirements

External Interface Requirements specify all inputs and outputs from the software system that involve entities external to the system. They define boundary between system and everything else it must interact with.

### 2.5.1 User Interface Requirements

Project Way is a web app with separate dashboards for Admin, Teacher and Student. The UI uses a clean layout: top navigation bar and side menu with clear buttons. The system uses a single font style and consistent colors for primary buttons and headings. Common elements like navigation menu, logout button and notifications icon appear on all dashboards. Buttons and labels use simple English such as “Submit Idea”, “Add Task”, “View Progress”, “Help Box”. Pages are designed for normal laptop/desktop resolution. Error and success messages are shown near the related form fields or as a small alert on the page.

### 2.5.2 Software Requirements

Project Way connects to a database (MongoDB) for storing users, project ideas, tasks, groups, issues and diagrams. The backend uses a REST API (JSON format) between the React frontend and the server. The system may use an email service for sending notifications like idea submit or issue replies. The app runs on top of a compatible web server/Node.js environment.

### 2.5.3 Hardware Requirements

Project Way does not require special hardware. It runs on student and teacher laptops with internet access and a modern browser. Mouse and keyboard are used for interactions like click, drag and drop in Diagram Editor and typing in forms.

### 2.5.4 Communication Interfaces

The system uses HTTP/HTTPS to communicate between browser and server. Users access the app through a web browser over the internet. If email notifications are enabled, the system uses standard email protocols (e.g., SMTP) to send messages to user email addresses.

## 2.6 Use Case Analysis

Use Case Analysis is a requirement engineering technique used to identify, clarify and organize the functional requirements of the system by describing the interactions between external actors and the system to achieve specific goals. It focuses on user goals and the step-by-step interaction between a user and the system.

### 2.6.1 Use Case 1

*Table 2- 11: Use Case 1*

| | |
|---|---|
| **Use Case ID** | UC-1 |
| **Use Case Name** | Login with Email and Password |
| **Actors** | Admin, Teacher, Student |
| **Description** | This use case allows an existing user to log in to Project Way using email and password. |
| **Trigger** | User opens the web app and clicks the “Login” button. |
| **Preconditions** | 1. User account already exists in the system.<br>2. Login page is loaded. |
| **Postconditions** | User is authenticated.<br>User is redirected to their dashboard (Admin, Teacher or Student). |
| **Normal Flow** | 1. User opens the web app.<br>2. Login page appears.<br>3. User enters email and password.<br>4. User clicks “Login”.<br>5. System checks email and password in the database.<br>6. If valid, system creates a session.<br>7. System redirects user to the correct dashboard based on role. |
| **Alternative Flows** | 1. System does not find a matching account.<br>2. System shows an error message like "Invalid email or password".<br>3. User can try again. |
| **Business Rules** | 1. Only valid registered users can log in.<br>2. Role (Admin, Teacher, Student) is checked after login. |
| **Assumptions** | 1. Authentication backend is working.<br>2. User has internet access. |

### 2.6.2 Use Case 2

*Table 2- 12: Use Case 2*

| | |
|---|---|
| **Use Case ID** | UC-2 |
| **Use Case Name** | Upload Project Idea / Template |
| **Actors** | Teacher |
| **Description** | Teacher uploads project ideas and templates for students to see. |
| **Trigger** | Teacher clicks “Upload” in Teacher dashboard. |
| **Preconditions** | 1. Teacher is logged in.<br>2. Teacher dashboard is open. |
| **Postconditions** | New idea/template is saved and visible to students. |
| **Normal Flow** | 1. Teacher opens Teacher dashboard.<br>2. Teacher clicks on “Upload”.<br>3. Teacher uploads a ideas/template file and write message.<br>4. Teacher clicks “Upload”.<br>5. System saves the idea/template in the database.<br>6. System shows a success message. |
| **Alternative Flows** | 1. Teacher leaves input field empty.<br>2. System shows a validation message.<br>3. Teacher fills the missing fields and submit again. |
| **Business Rules** | Only Teachers can upload ideas/templates. |
| **Assumptions** | File upload is working. |

### 2.6.3 Use Case 3

*Table 2- 13: Use Case 3*

| | |
|---|---|
| **Use Case ID** | UC-3 |
| **Use Case Name** | Submit Project Idea |

| | |
|---|---|
| **Actors** | Student |
| **Description** | Student submits project idea to the teacher for approval. |
| **Trigger** | Student clicks “Submit” in Student dashboard. |
| **Preconditions** | 1. Student is logged in.<br>2. Student dashboard is open. |
| **Postconditions** | Student idea is saved with status “Pending”. |
| **Normal Flow** | 1. Student opens Student dashboard.<br>2. Student clicks “Submit Idea”.<br>3. Student fills form (title, description, group information).<br>4. Student clicks “Submit”.<br>5. System saves the idea with status “Pending”.<br>6. System shows a confirmation message. |
| **Alternative Flows** | 1. Some required fields are empty.<br>2. System shows an error message.<br>3. Student fills them and submit again. |
| **Business Rules** | One group can have only one active idea request at a time. |
| **Assumptions** | Teacher will later review the idea. |

### 2.6.4 Use Case 4

*Table 2- 14: Use Case 4*

| | |
|---|---|
| **Use Case ID** | UC-4 |
| **Use Case Name** | Review Ideas and Manage Groups |
| **Actors** | Teacher |
| **Description** | Teacher reviews student ideas and places students lists (accepted, rejected). |
| **Trigger** | Teacher clicks “All ideas” in Teacher dashboard. |
| **Preconditions** | 1. Teacher is logged in. |

| | |
|---|---|
| | 2. There are submitted student ideas. |
| **Postconditions** | 1. Each idea gets a status (Accepted/Rejected).<br>2. Students are assigned to lists. |
| **Normal Flow** | 1. Teacher opens Teacher dashboard.<br>2. Teacher see ideas request.<br>4. Teacher opens one idea and reads details.<br>5. Teacher sets status (Accepted or Rejected).<br>7. System saves status and group information. |
| **Alternative Flows** | System shows message “No idea requests”. |
| **Business Rules** | Only the Teacher can accept or reject ideas of Students. |
| **Assumptions** | Groups and sections (morning/evening) are already defined in the system. |

### 2.6.5 Use Case 5

*Table 2- 15: Use Case 5*

| | |
|---|---|
| **Use Case ID** | UC-5 |
| **Use Case Name** | Create Tasks and Track Progress |
| **Actors** | Student (Leader and members) |
| **Description** | Team leader creates tasks and group members track progress. |
| **Trigger** | Leader clicks “Tasks Management” in Student dashboard. |
| **Preconditions** | 1. Student is logged in and type leader name and system assign them as group leader.<br>2. Group exists. |
| **Postconditions** | Tasks are saved and group progress is updated. |
| **Normal Flow** | 1. Leader opens Student dashboard.<br>2. Leader goes to “Tasks Management” page.<br>3. Leader clicks “Add Task”. |

| | |
|---|---|
| | 4. Leader enters task title, description, due date and assigns member.<br>5. Leader clicks “Save”.<br>6. System saves task and updates progress bar.<br>7. Group members can view their tasks and mark them as done. |
| **Alternative Flows** | If the student is not in a group then system shows message “No group assigned”. |
| **Business Rules** | Only group leader can create or delete tasks. |
| **Assumptions** | Progress bar is calculated from completed vs total tasks. |

### 2.6.6 Use Case 6

*Table 2- 16: Use Case 6*

| | |
|---|---|
| **Use Case ID** | UC-6 |
| **Use Case Name** | View Group Progress |
| **Actors** | Teacher |
| **Description** | Teacher views project progress of each student group. |
| **Trigger** | Teacher clicks “Progress Tracking” in Teacher dashboard. |
| **Preconditions** | 1. Teacher is logged in.<br>2. Groups and tasks exist. |
| **Postconditions** | Teacher sees whole student groups progress. |
| **Normal Flow** | 1. Teacher opens Teacher dashboard.<br>2. Teacher clicks “Progress Tracking”.<br>3. System shows list of groups.<br>4. Teacher selects list.<br>5. System shows progress bar/percentage. |
| **Alternative Flows** | System shows message “No tasks created for this group”. |
| **Business Rules** | Teacher view whole Student groups progress. |
| **Assumptions** | Task status is up to date from student side. |

### 2.6.7 Use Case 7

*Table 2- 17: Use Case 7*

| | |
|---|---|
| **Use Case ID** | UC-7 |
| **Use Case Name** | Create, Save and Download Diagram |
| **Actors** | Student |
| **Description** | Student creates diagrams using the Diagram Editor and can save or download them. |
| **Trigger** | Student clicks “Diagram Editor” in Student dashboard. |
| **Preconditions** | Student is logged in. |
| **Postconditions** | Diagram is saved in the system or downloaded as an image. |
| **Normal Flow** | 1. Student opens Student dashboard.<br>2. Student clicks “Diagram Editor”.<br>3. Diagram Editor page opens.<br>4. Student drags shapes and connectors to build a diagram.<br>5. Student clicks “Save” to store it.<br>6. Student can click “Download” to download as an image. |
| **Alternative Flows** | 1. Student tries to leave page.<br>2. System can warn about unsaved changes. |
| **Business Rules** | Diagrams are linked to the project. |
| **Assumptions** | Browser supports canvas/SVG and download. |

### 2.6.8 Use Case 8

*Table 2- 18: Use Case 8*

| | |
|---|---|
| **Use Case ID** | UC-8 |
| **Use Case Name** | Submit Issue and View Teacher Feedback |
| **Actors** | Student, Teacher |

| | |
|---|---|
| **Description** | Student sends an issue to Teacher and Teacher give feedback. |
| **Trigger** | Student clicks “Help Box” in Student dashboard. |
| **Preconditions** | Student is logged in. |
| **Postconditions** | Issue and teacher reply are saved and viewable. |
| **Normal Flow** | 1. Student opens Student dashboard.<br>2. Student clicks “Help Box”.<br>3. Student writes issue title and message.<br>4. Student clicks “Send”.<br>5. System saves issue and notifies the Teacher.<br>6. Teacher opens Teacher dashboard and goes to “Help Box”.<br>7. Teacher reads the issue and types a reply.<br>8. System saves the reply and notifies the Student. |
| **Alternative Flows** | 1. Student sends empty issue.<br>2. System shows error and asks to write a message. |
| **Business Rules** | Issues and replies are kept for record. |
| **Assumptions** | Notification system is working for teacher and student alerts. |

### 2.6.9 Use Case 9

*Table 2- 19: Use Case 9*

| | |
|---|---|
| **Use Case ID** | UC-9 |
| **Use Case Name** | Notifications |
| **Actors** | Teacher, Student |
| **Description** | User views system notifications about important events. |
| **Trigger** | User clicks on “Notifications” icon. |
| **Preconditions** | 1. User is logged in.<br>2. Some events already happened like idea submit, issue submit, feedback etc. |

| | |
|---|---|
| **Postconditions** | Notifications are shown and can be marked as read. |
| **Normal Flow** | 1. User logs in.<br>2. User clicks on “Notifications”.<br>3. System shows a list of notifications.<br>4. User clicks on notification to open related page (idea, issue). |
| **Alternative Flows** | System shows message “No notifications”. |
| **Business Rules** | Notifications should show who did what and when. |
| **Assumptions** | Notification data is stored in database. |

## 2.7 Use Case Diagram

### 2.7.1 Use Case Diagram 1

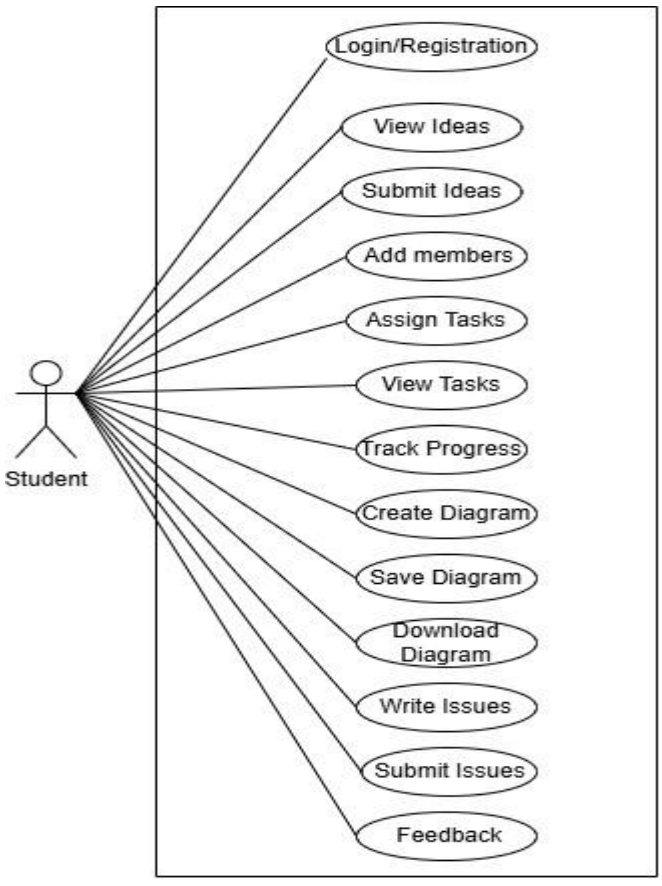



*Figure 2: Use Case 1*

### 2.7.2 Use Case Diagram 2

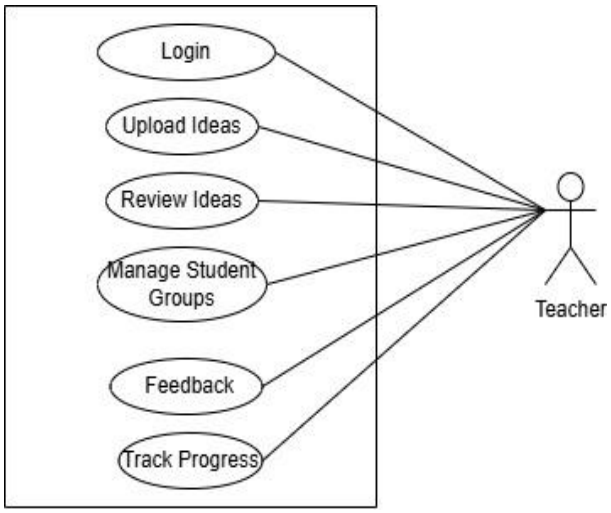



*Figure 3: Use Case 2*

### 2.7.3 Use Case Diagram 3

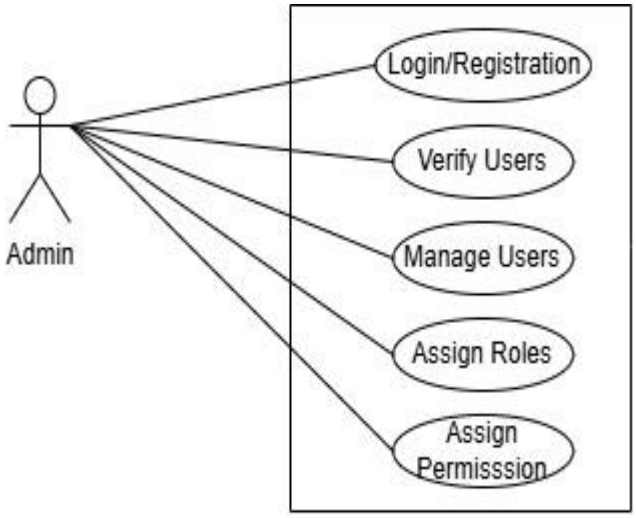



*Figure 4: Use Case 3*

### 2.7.4 Use Case Diagram 4




*Figure 5: Use Case 4*

## 2.8 Summary

This chapter explains how the requirements for Project Way were found and documented. Users are divided into three main types: Admin, Teacher and Student. Requirements were collected using method like use case analysis for each panel and feature. The gathered requirements are organized into functional requirements (what the system should do) and non-functional requirements (performance, security, usability) and are linked with clear use case descriptions and diagrams for the Admin, Teacher and Student panels including the Task Management and Diagram Editor tools.




# Chapter 3: Design and Architecture

# 3. System Design

The Project Way connects Teachers and Students to work together in easy manner. It operates as a web application developed by using MERN stack. MongoDB stores all user data, projects, groups, tasks and diagrams securely. React.js is used for frontend development to provide easy to use interface. Node.js with Express.js handles backend logic APIs and data processing.

### Dependencies and Interactions

- Frontend (React.js): It handles the user interface for Admin, Teachers and Students dashboards. It communicates with backend APIs to send and receive data such as user information, project ideas, task assignments, progress updates and notifications.
- Backend (Node.js with Express): It is responsible for business logic, processing requests from the frontend, managing authentication and authorization, handling data validation and controlling data flow from the database.
- Database (MongoDB): It stores all data including Teachers and Students records, project ideas, group information, task schedules, project progress and diagrams data.
- Email Notifications (NodeMailer): Integrated within the backend to send emails to Teachers and Students about task assignments, approvals, feedbacks and project updates in real time.
- Communication (WebSocket/Socket.io): It enables instant messaging or live updates between Students and Teachers about task issues and project progress.
- Diagram Editor (Frontend library or custom React component): It allow users to create and edit diagrams with data saved.

### Design Constraints

- Performance: The web app should load quickly and work smoothly even when many users use it at the same time.
- Security: All user data must be kept safe and private with strong rules to stop unauthorized access.
- Cross-platform: The web app should work well on computers and mobile devices like phones and tablets.

## 3.1 Design Considerations

### Assumptions and Dependencies

- All Students and Teachers have internet access.
- Every user has email for login.
- Students knows basic computer web skills to use Editor.
- Upload files for project ideas/template size will be small.

### Limitations

- Offline access is not possible.
- It only supports small files for Student project ideas/template upload by Teachers.

- Not provides Students instant response from Teachers.

### Risks

- Technical issues such as system bugs and failure.
- Technical issues with file uploads or downloads.
- Data breaches and unauthorized access to Teachers and Students data.

## 3.2 Design Models

These models represent the conceptual design of the system that help us to visualize and analyze the structure and behavior of the system.

### 3.2.1 Domain Model

This model represents the static structure of the system including classes, attributes and relationship.

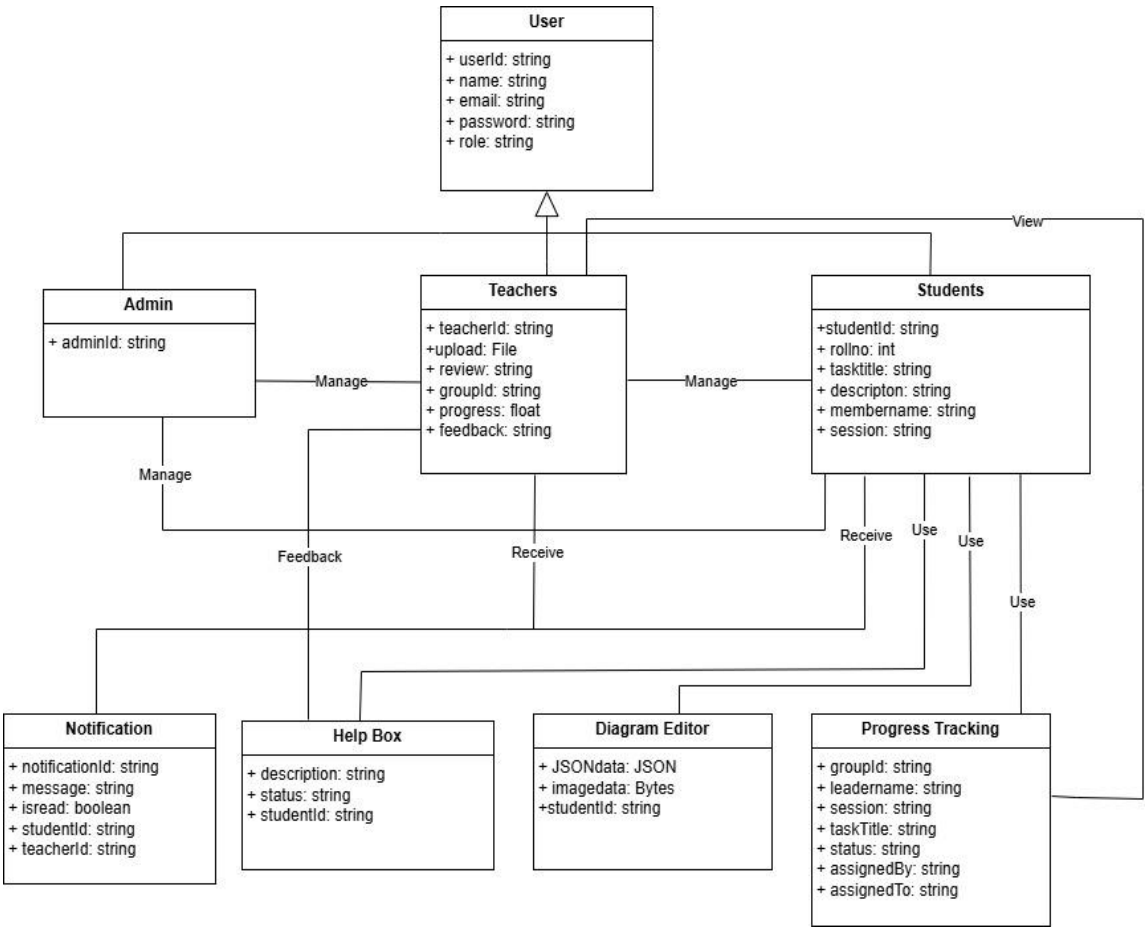



*Figure 6: Domain Model Diagram*

### 3.2.2 Class Diagram

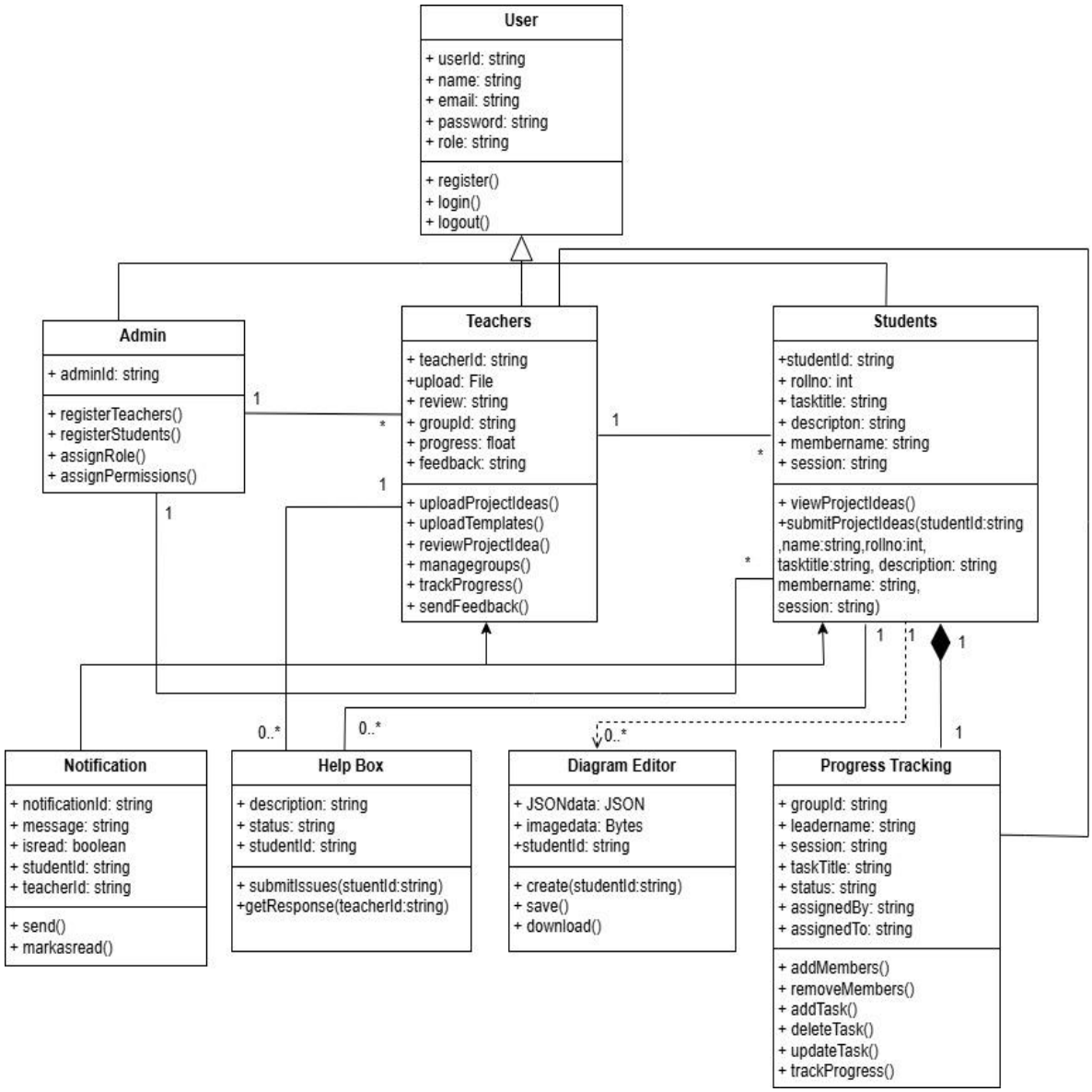



*Figure 7: Class Diagram*

### 3.2.3 Sequence Diagram

A Sequence Diagram is a UML behavioral diagram that model the interaction between objects in a system and arranged in time sequence. It shows how and in what order objects exchange message to carry out a specific use case. For Project Way, all modules sequence diagram given below:

## Module 1: Registration

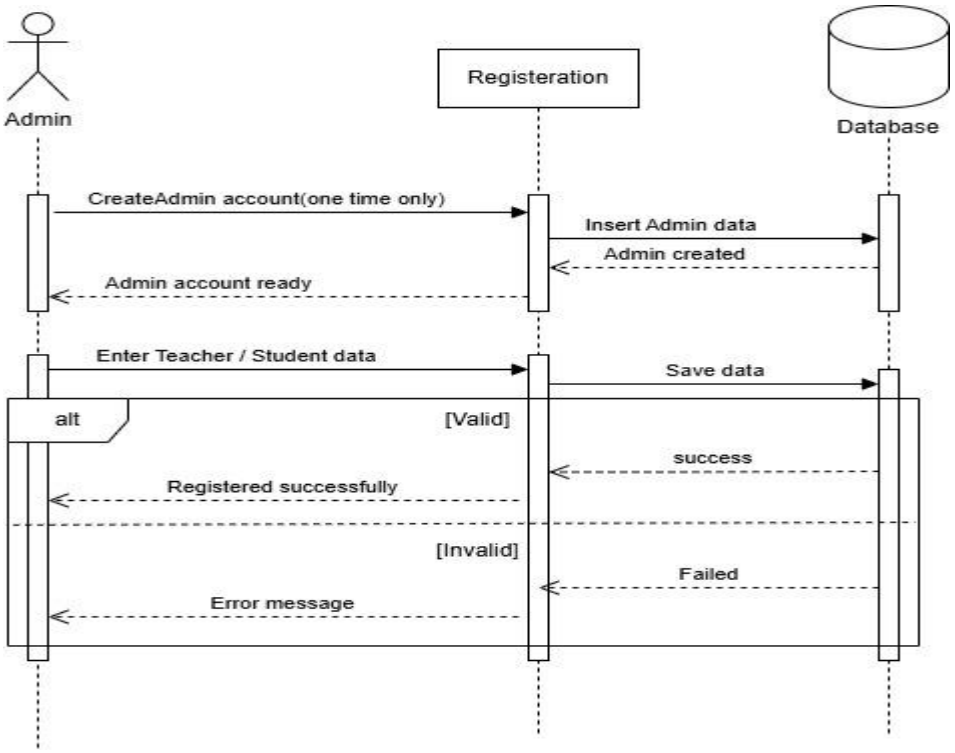



*Figure 8: Module 1 Sequence Diagram*

## Module 2: Login

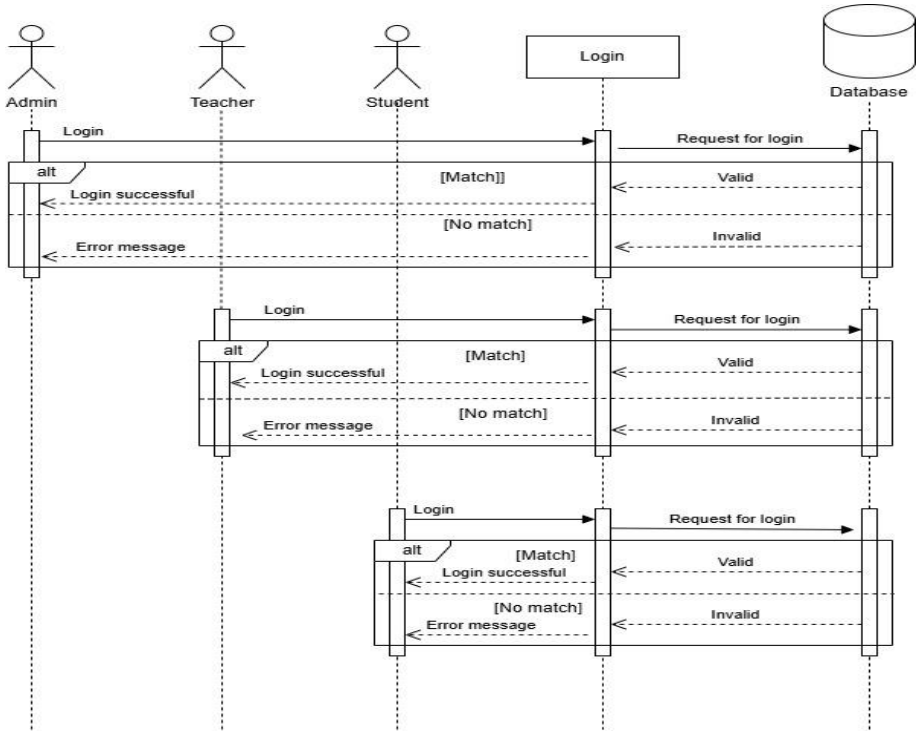



*Figure 9: Module 2 Sequence Diagram*

### Module 3: Admin Dashboard

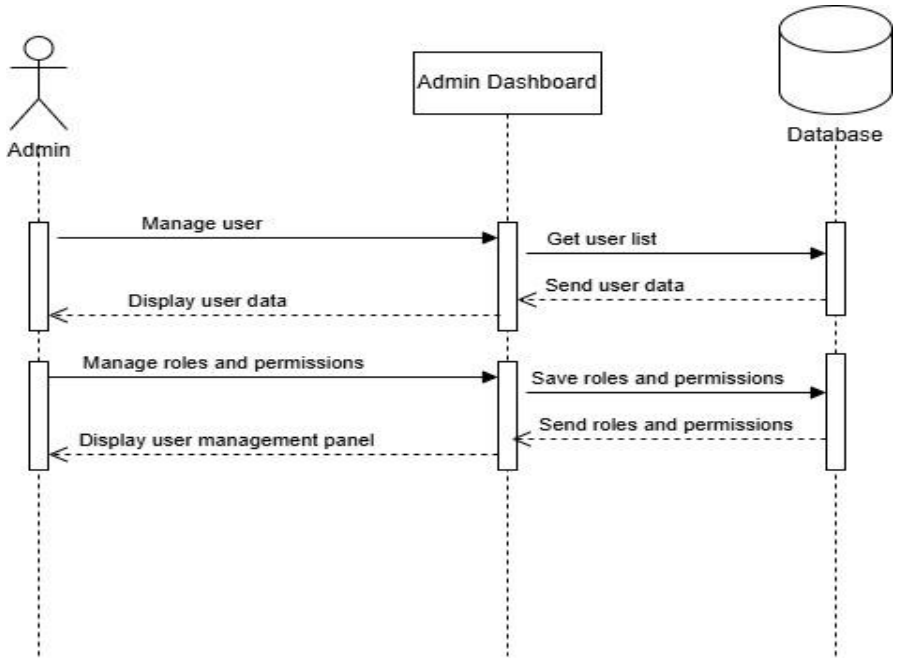



*Figure 10: Module 3 Sequence Diagram*

### Module 4: Teacher Dashboard

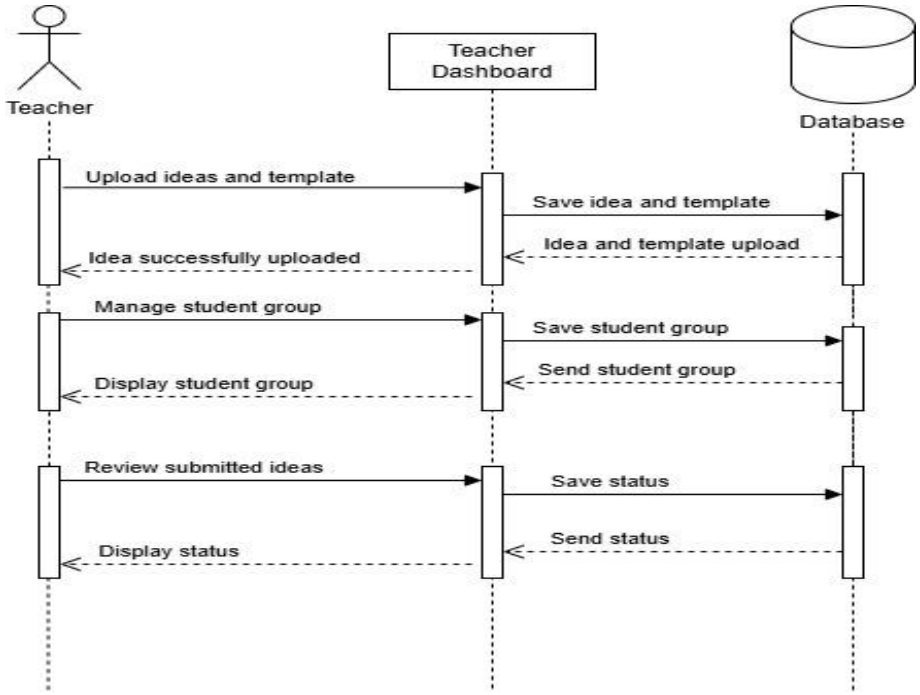



*Figure 11: Module 4 Sequence Diagram*

## Module 5: Student Dashboard

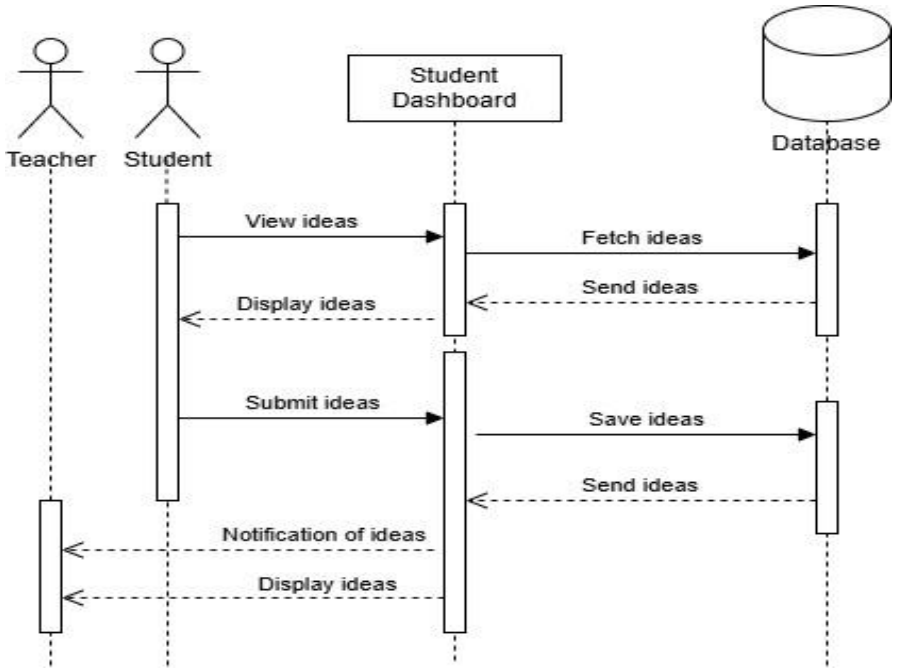



*Figure 12: Module 5 Sequence Diagram*

## Module 6: Progress Tracking

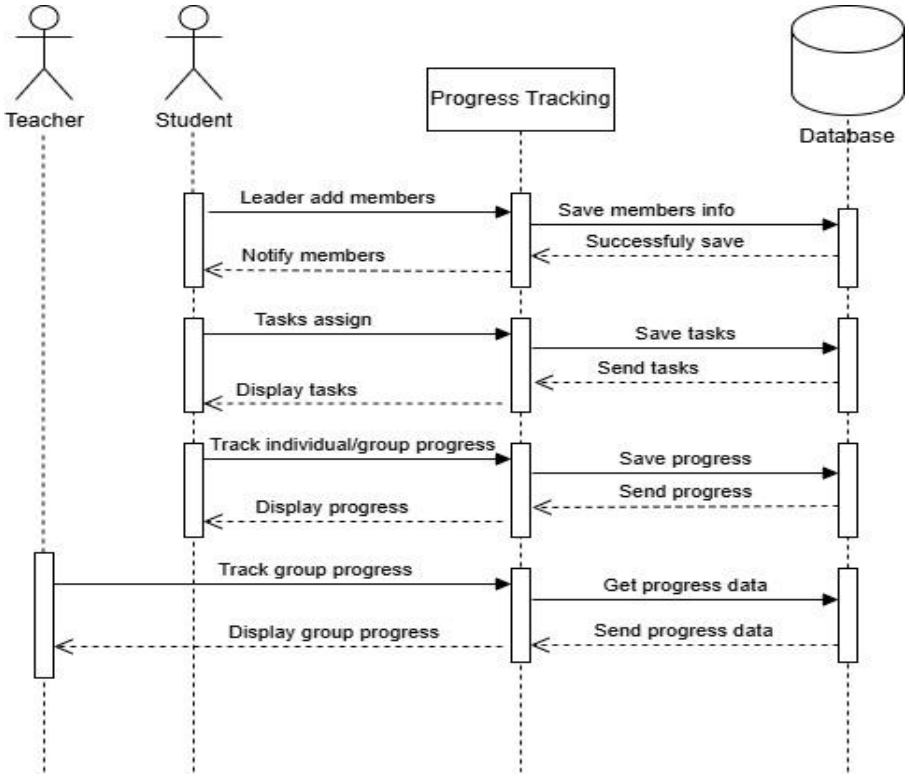



*Figure 13: Module 6 Sequence Diagram*

## Module 7: Diagram Editor

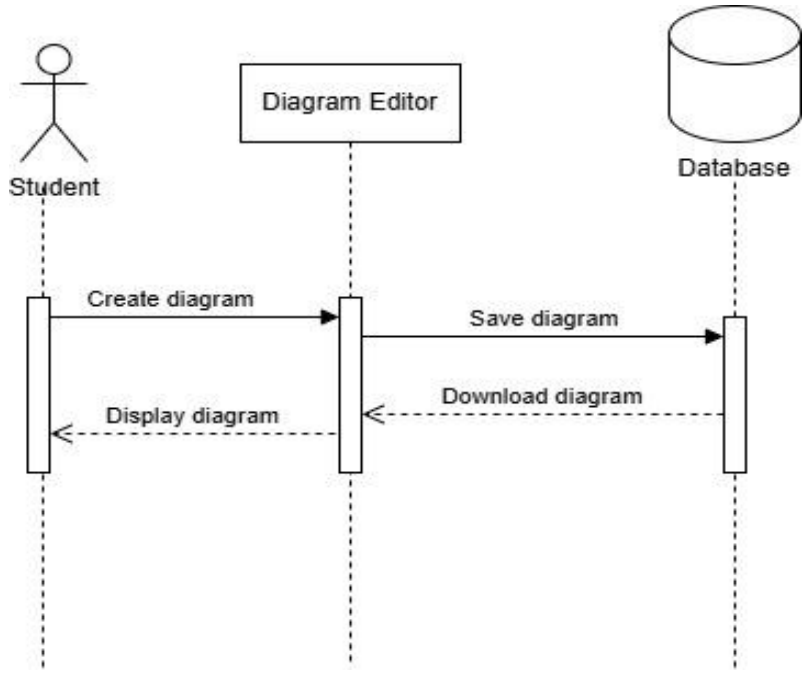



*Figure 14: Module 7 Sequence Diagram*

## Module 8: Help Box

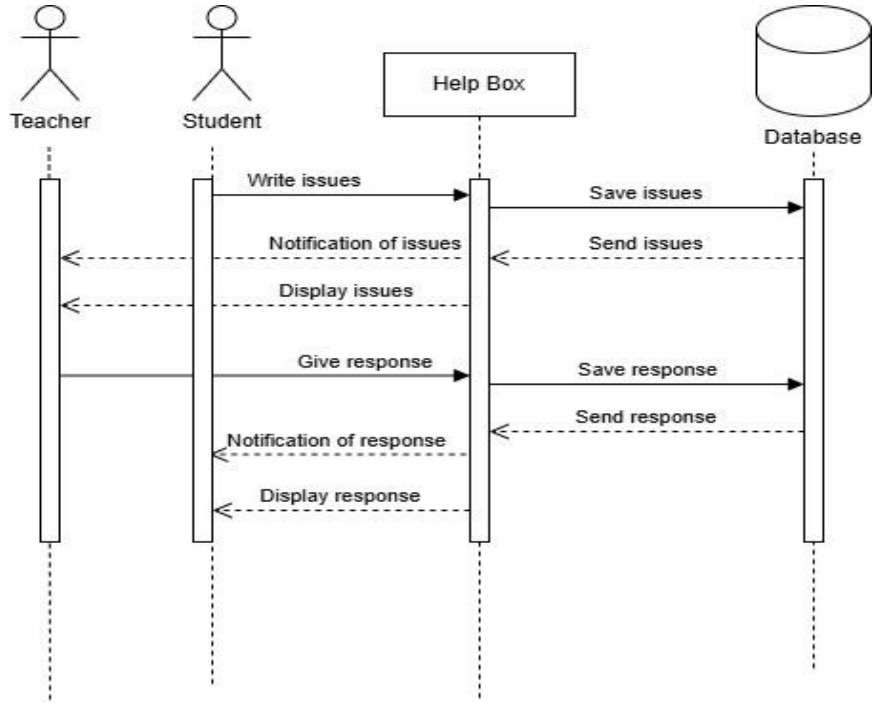



*Figure 15: Module 8 Sequence Diagram*

### 3.2.4 State Transition Diagram

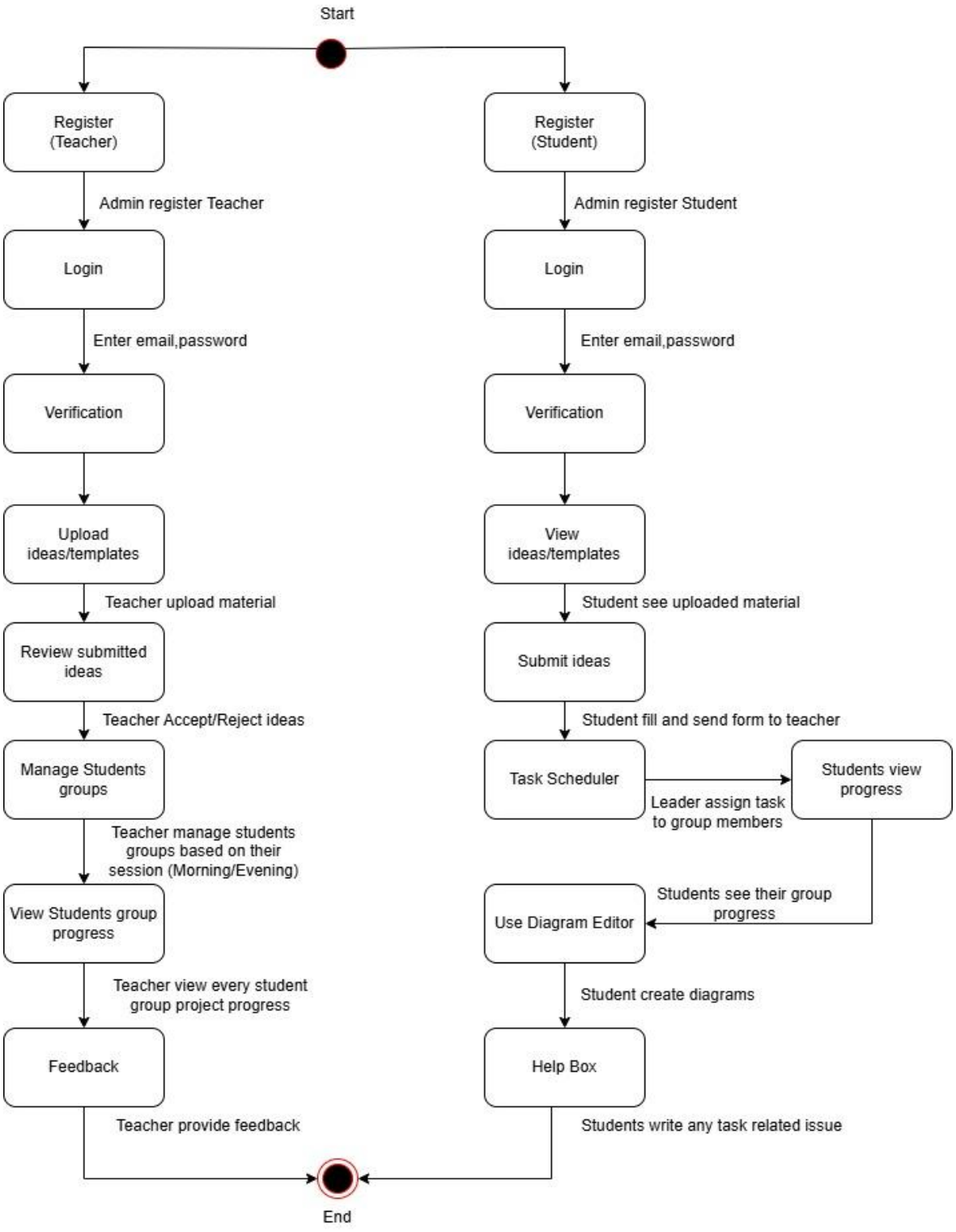



*Figure 16: State Transition Diagram*

### 3.2.5 Entity Relationship (ER) Diagram

Entity Relationship is a conceptual model that defines entities and their relationships in database. It helps us to visualize and understand how different entities in real world are connected to one another.

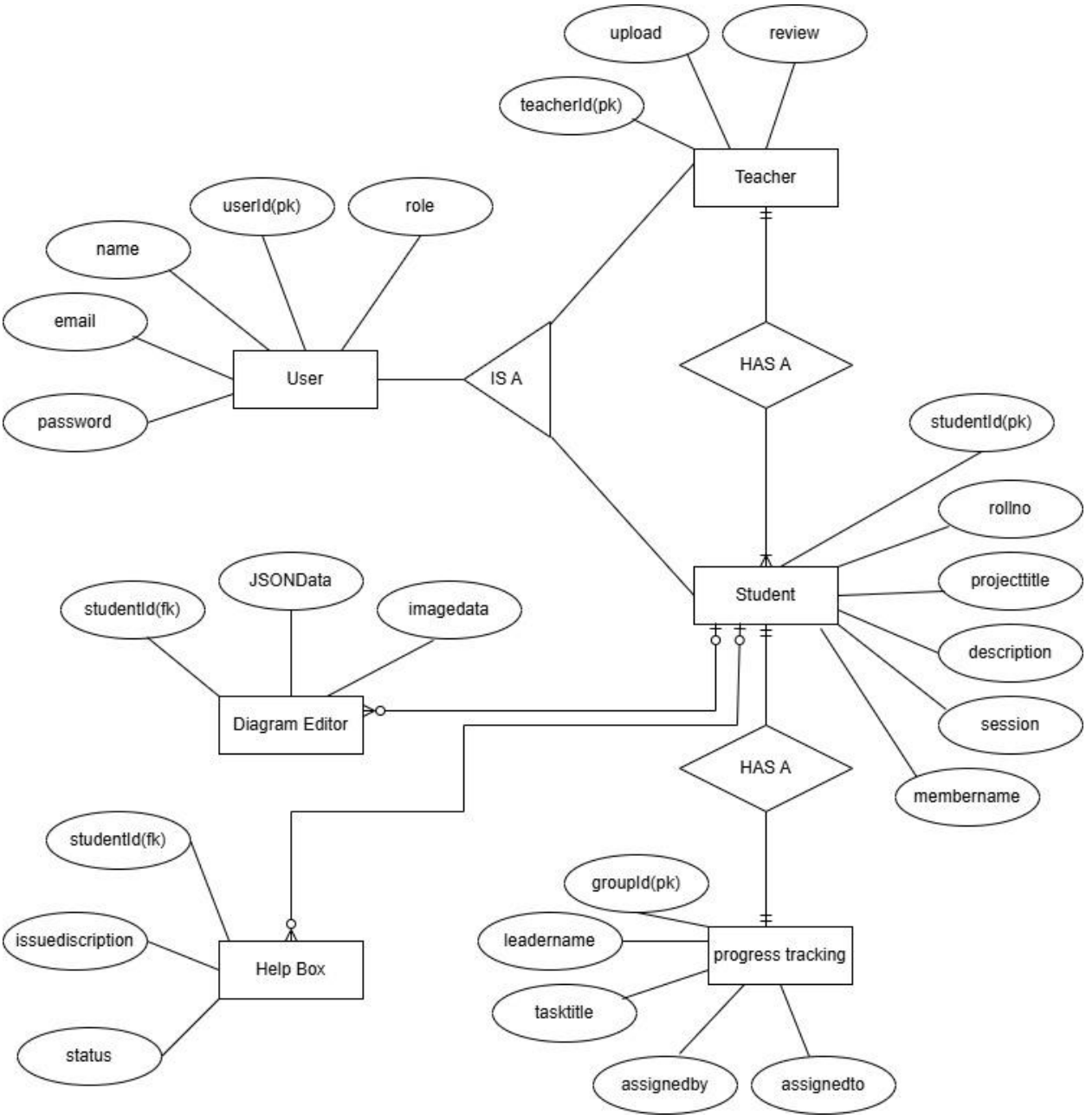



*Figure 17: Entity Relationship Diagram*

## 3.3 Architectural Design

### 3.3.1 UML Class Relationship Diagram

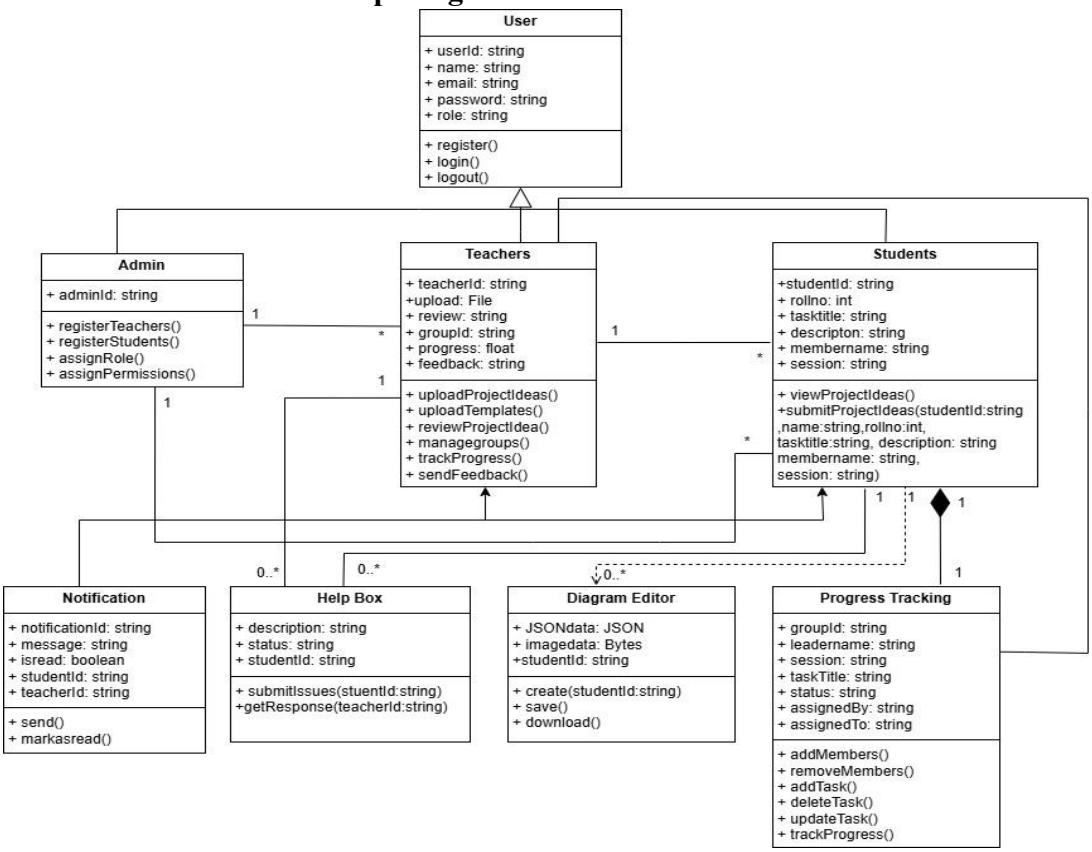



*Figure 18: UML Class Relationship Diagram*

### 3.3.2 UML Component Diagram

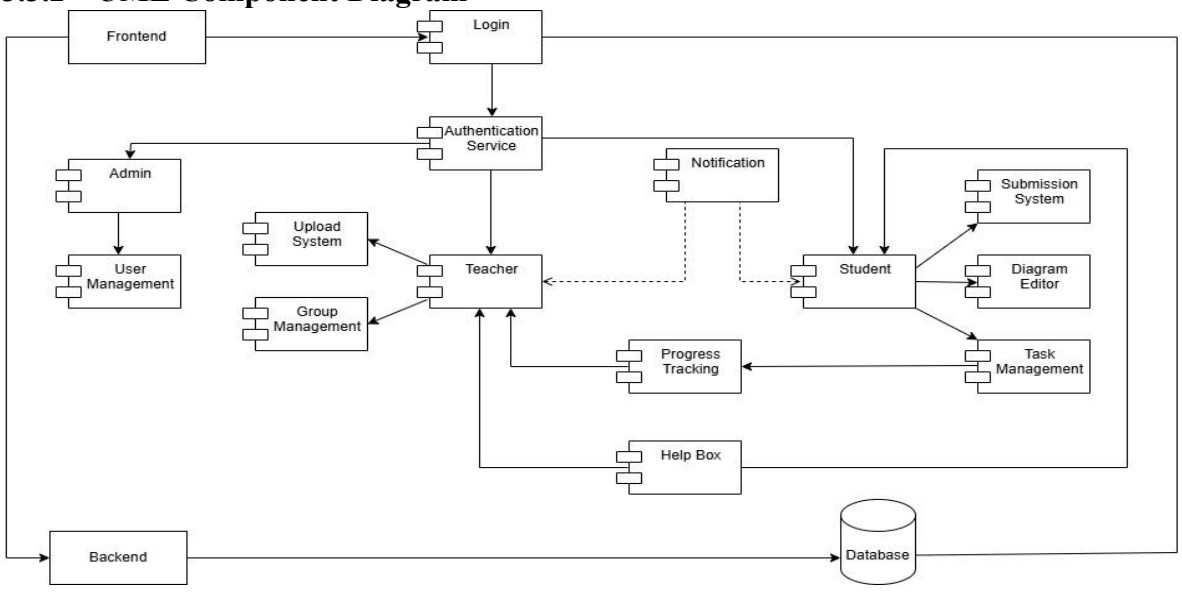



*Figure 19: UML Component Diagram*

### 3.3.3 Model View Controller (MVC) Diagram

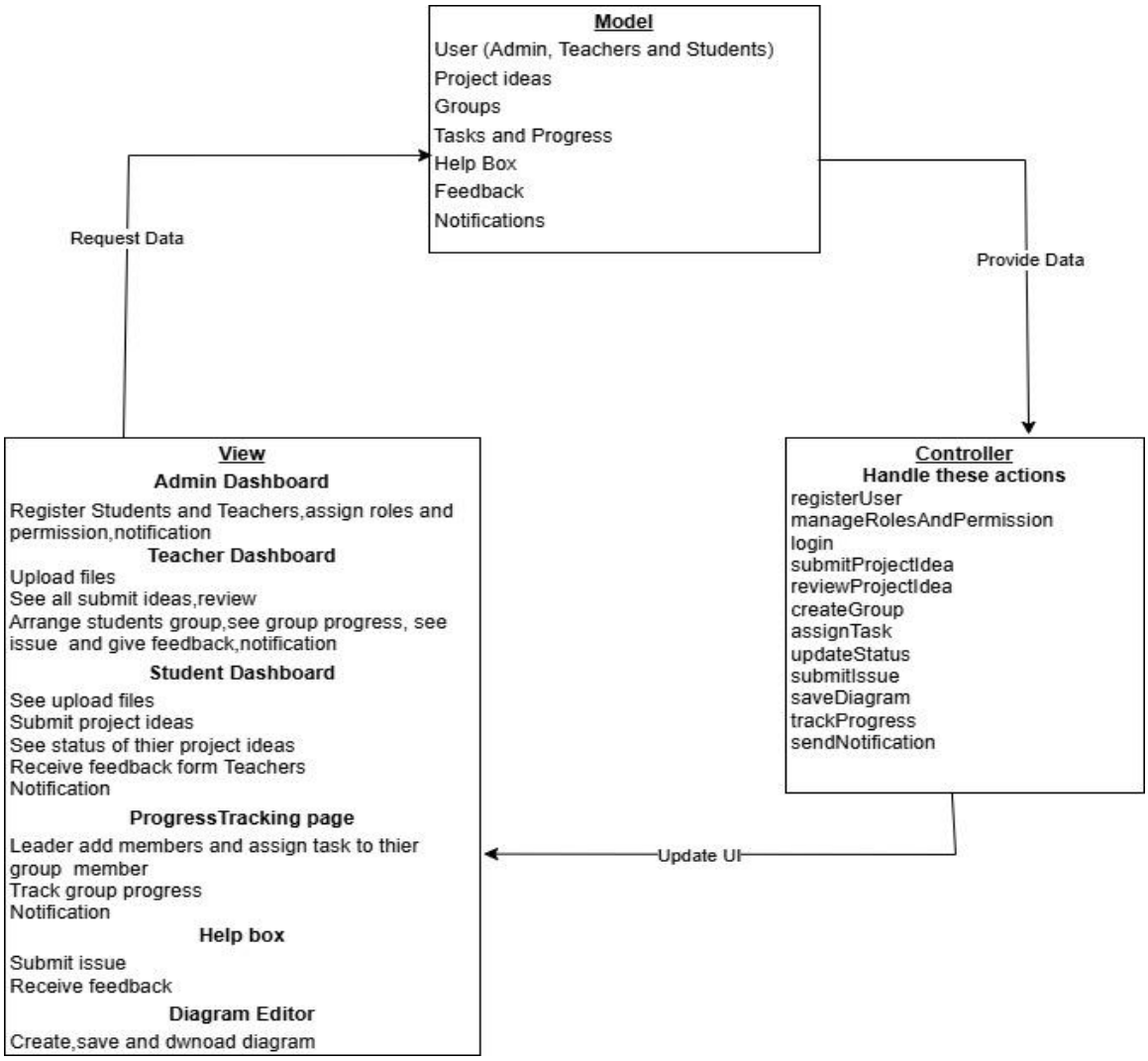



*Figure 20: MVC Diagram*

### Model View Controller (MVC) Pattern

We used the MVC pattern to separate the design into Parts:

i. Model for data (like users, tasks, groups)
ii. View for what the user sees (React)
iii. Controller for system logic (Express.js)

## 3.4 Data Design

Project Way uses MongoDB (NoSQL Database) to store data. The database is split into collections (users, tasks etc.) and connected using Ids. This makes updates easy and keeps data correct.

### 3.4.1 Data Dictionary

- **Alphabetical List of System Entities with Description**

*Table 3- 1: Alphabetical List of System Entities with Description*

| Entities | Description |
|---|---|
| Admin | Admin registers Teachers/Students, manage roles and permissions. |
| Diagram Editor | Students create, save and download diagrams using built-in Diagram Editor. |
| Help Box | Students submit any task related issue to Teachers and receive feedback from Teachers. |
| Notification | Notify Teachers when Students submit project idea and send any task related issue. Students get notifications when Teacher accept or reject an idea. |
| Progress Tracking | Represent tasks assigned to Students within groups. It contains task title, deadline, status, assigned Students. |
| Students | Students see uploaded ideas, submit project idea and see the status of project idea. |
| Teachers | Teachers upload materials (project ideas/templates), review project ideas and manage Students in groups. |

- **Object Oriented (OO) Approach:** Objects, Attributes & Methods

1. **Object:** User
   **Attributes**

- userId: Unique Id for each user.
- name: Users full name.
- email: Login email.
- password: Encrypted password.
- role: Admin, Teacher and Student.
  **Methods**
- register(): New user register.
- login(): User login.
- logout(): Logout session.

2. **Object:** Admin
   **Attributes**
- adminId: Unique Id for Admin.
  **Methods**
- registerTeachers(): Add new teachers.
- registerStudents(): Add new students.
- assignRole(): Define user as teacher/student.
- assignPermissions(): Give access rights.

3. **Object:** Teachers
   **Attributes**
- teacherId: Unique Id for Teacher.
- upload: Upload data.
- review: Review data.
- groupId: Unique students group Id.
- progress: Progress data.
- feedback: Feedback data.
  **Methods**
- uploadProjectIdeas(): Share project ideas.
- uploadTemplates(): Upload material.
- reviewProjectIdea(): Accept/reject Student ideas.
- manageGroups(): Manage Student groups.
- trackProgress(): Monitor group progress.
- sendFeedback(): Give feedback to Students.

4. **Object:** Students
   **Attributes**
- studentId: Unique Id for student.
- rollno: Students roll number.
- projecttitle: Students describe their title.
- description: Students describe their project.
- Groupmember: Member names.
- session: Morning/Evening batch.

   **Methods**
- viewProjectIdeas(): View Teacher uploaded ideas.
- submitProjectIdeas(studentId: string, name: string, rollno: int, projecttitle: string, description: string, membername: string, session: string): Send project idea with id, name, roll no, projectitle, description and session.

5. **Object:** Progress Tracking
   **Attributes**
- groupId: Unique Id for each group.
- leaderName: Group leader.
- session: Morning/Evening.
- taskTitle: Task name.
- status: Pending, In Progress, Done.
- assignedBy: Who assigned task.
- assignedTo: Task receiver.

   **Methods**
- addMembers(): Add group members.
- addTask(): Add new task.
- updateTask(): Modify task.
- trackProgress(): View progress.

6. **Object:** Diagram Editor
   **Attributes**
- JSONdata: Diagram data.
- imageData: Saved image.
- studentId: Who created diagram.

   **Methods**
- create(studentId: string): Create a new diagram.
- save(): Save diagram.
- download(): Export diagram image.

7. **Object:** Help Box
   **Attributes**
- description: Issue detail.
- status: Pending/resolved.
- studentId: Who submitted.
  **Methods**
- submitIssue(studentId: string): Submit a help request.
- sendResponse(teacherId: string): Reply to the issue.

8. **Object:** Notification
   **Attributes**
- notificationId: Unique notification ID.
- message: Notification text.
- isRead: See or not.
- studentId and teacherId: Linked users.
  **Methods**
- send(): Send notification.
- markAsRead(): Mark as seen.

## 3.5 User Interface Design

### 3.5.1 Screen Images

**Admin Dashboard**

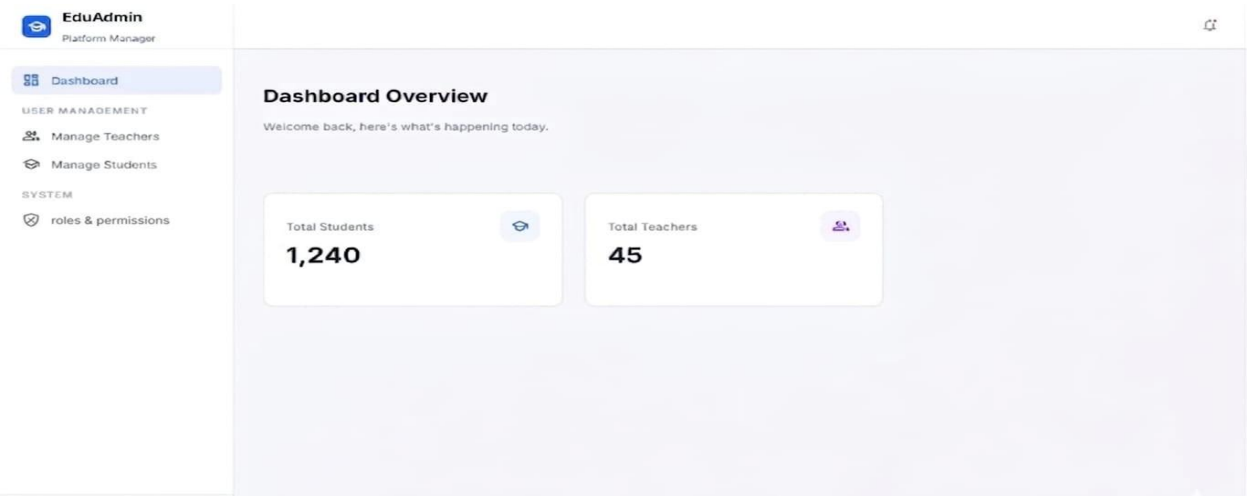



*Figure 21: Screen Image 1*

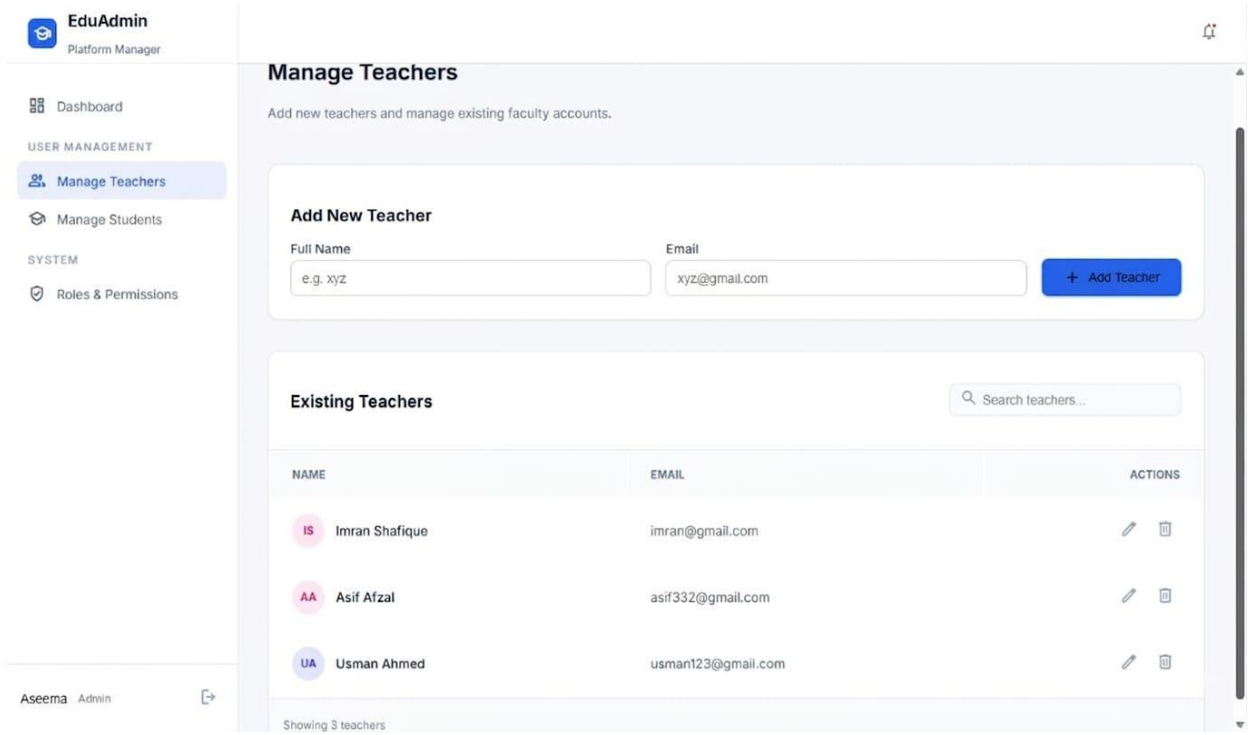



*Figure 22: Screen Image 2*

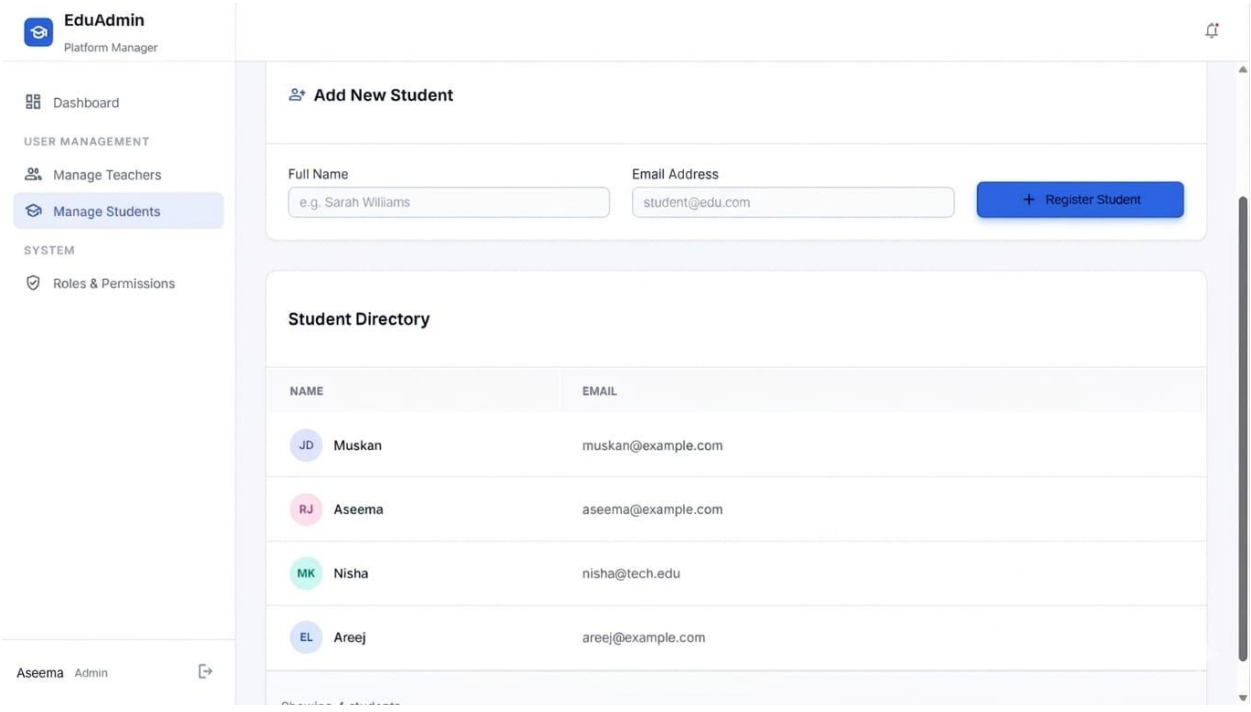



*Figure 23: Screen Image 3*

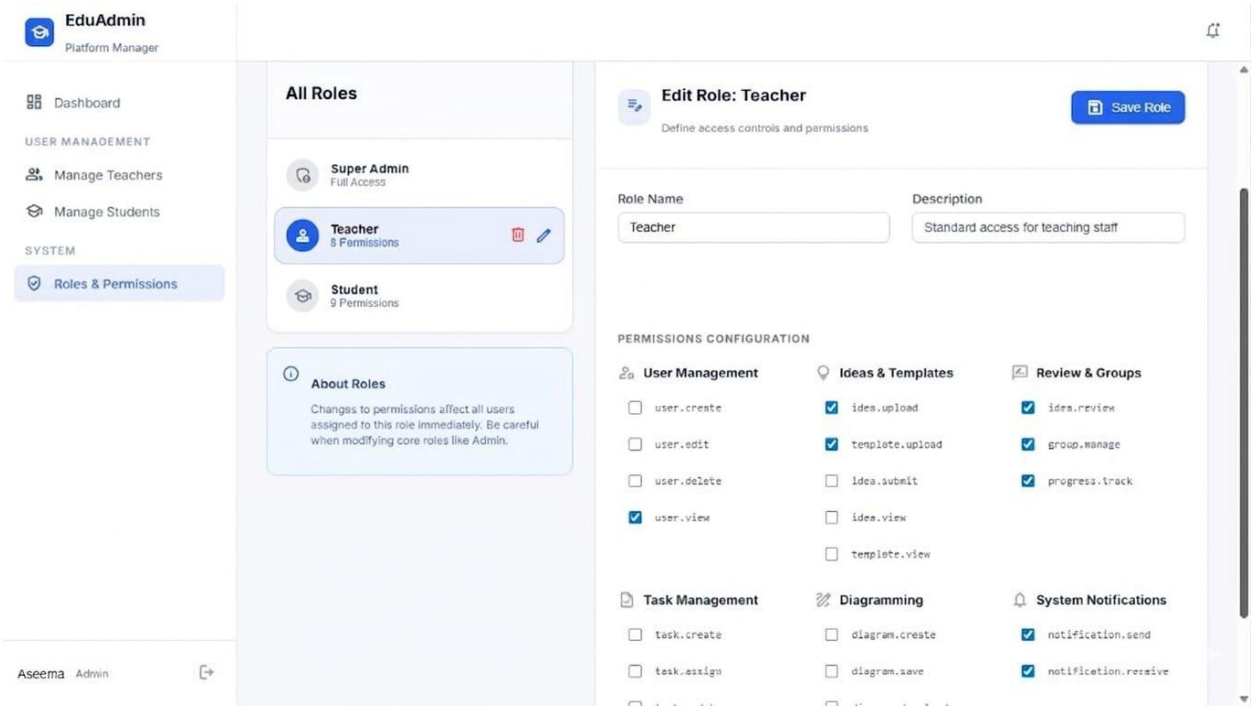



*Figure 24: Screen Image 4*

## Teacher Dashboard

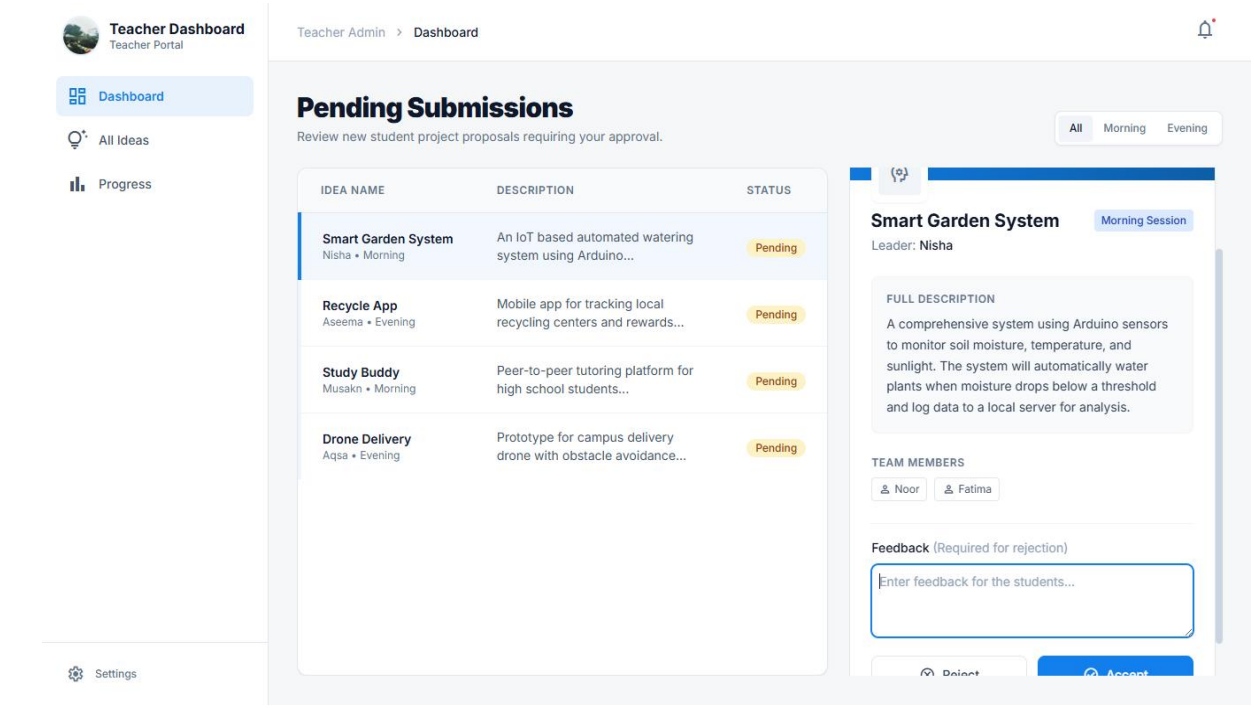



*Figure 25: Screen Image 5*

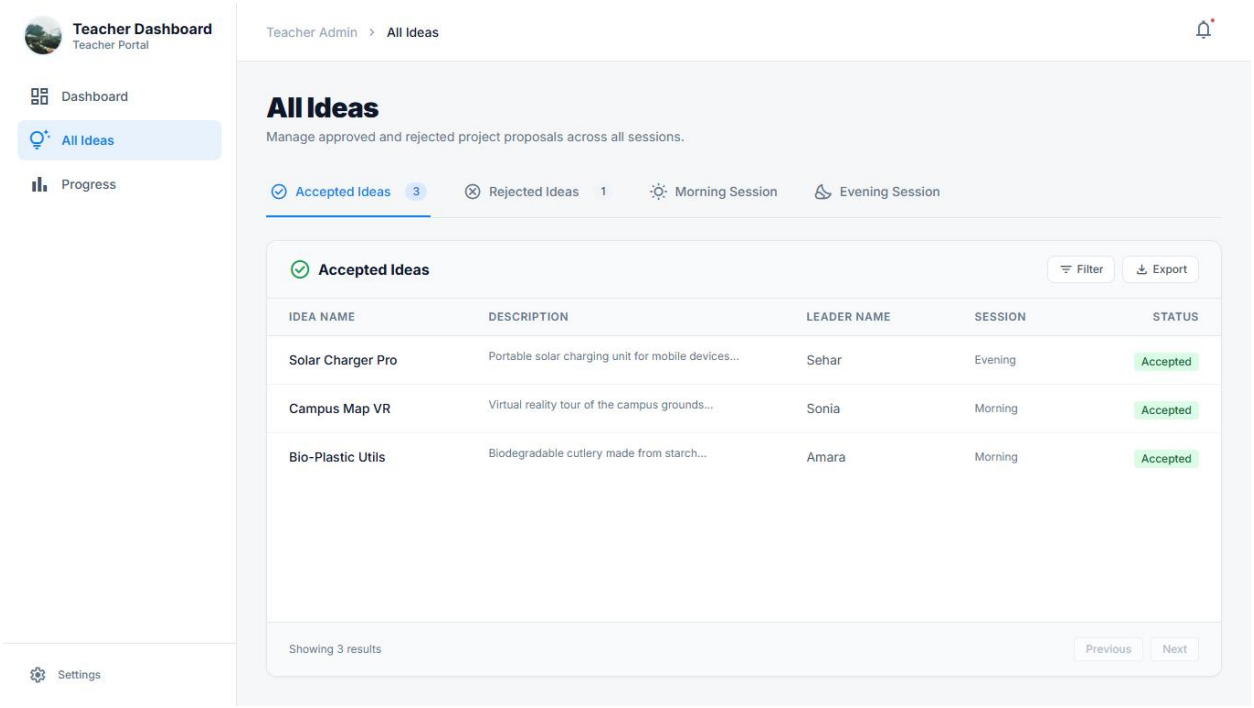



*Figure 26: Screen Image 6*

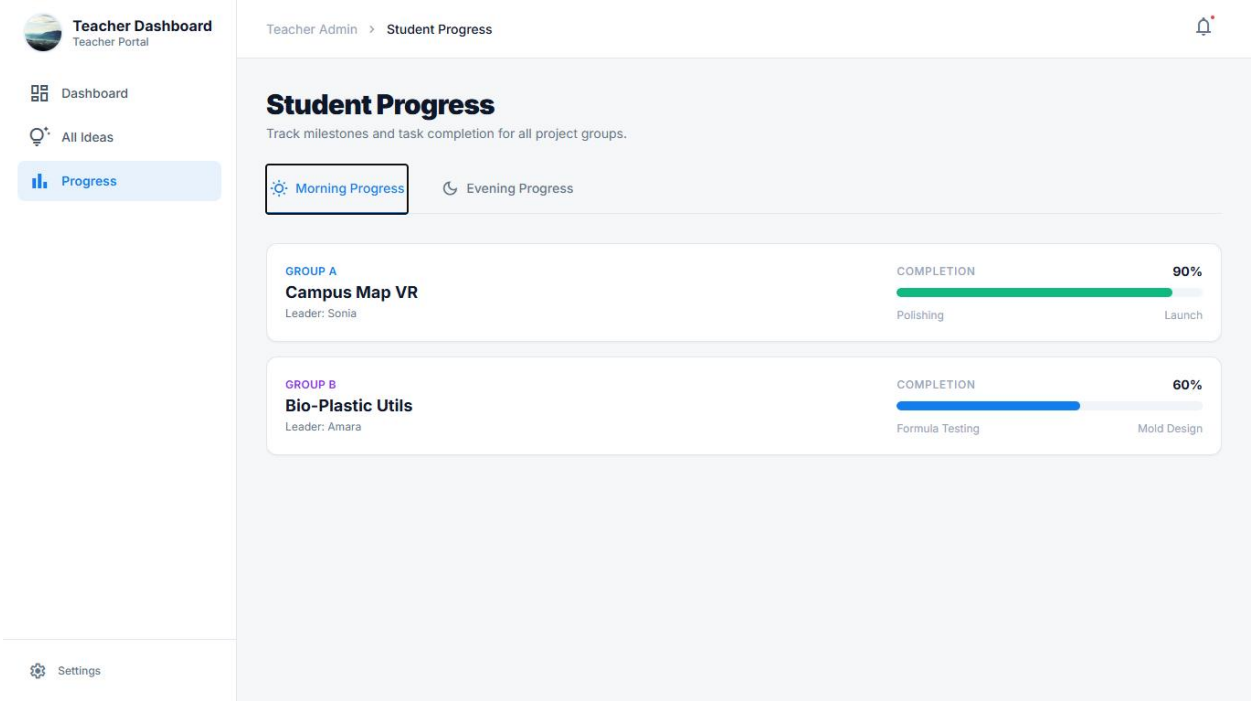



*Figure 27: Screen Image 7*

## Student Dashboard

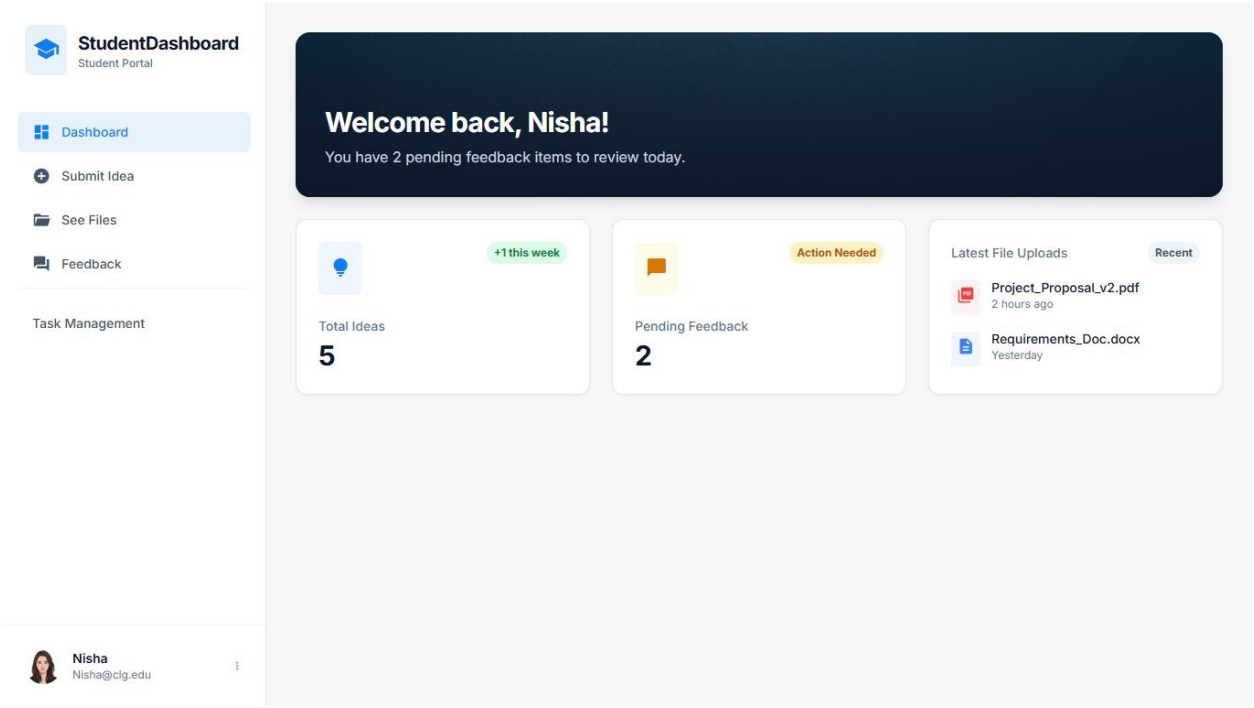



*Figure 28: Screen Image 8*

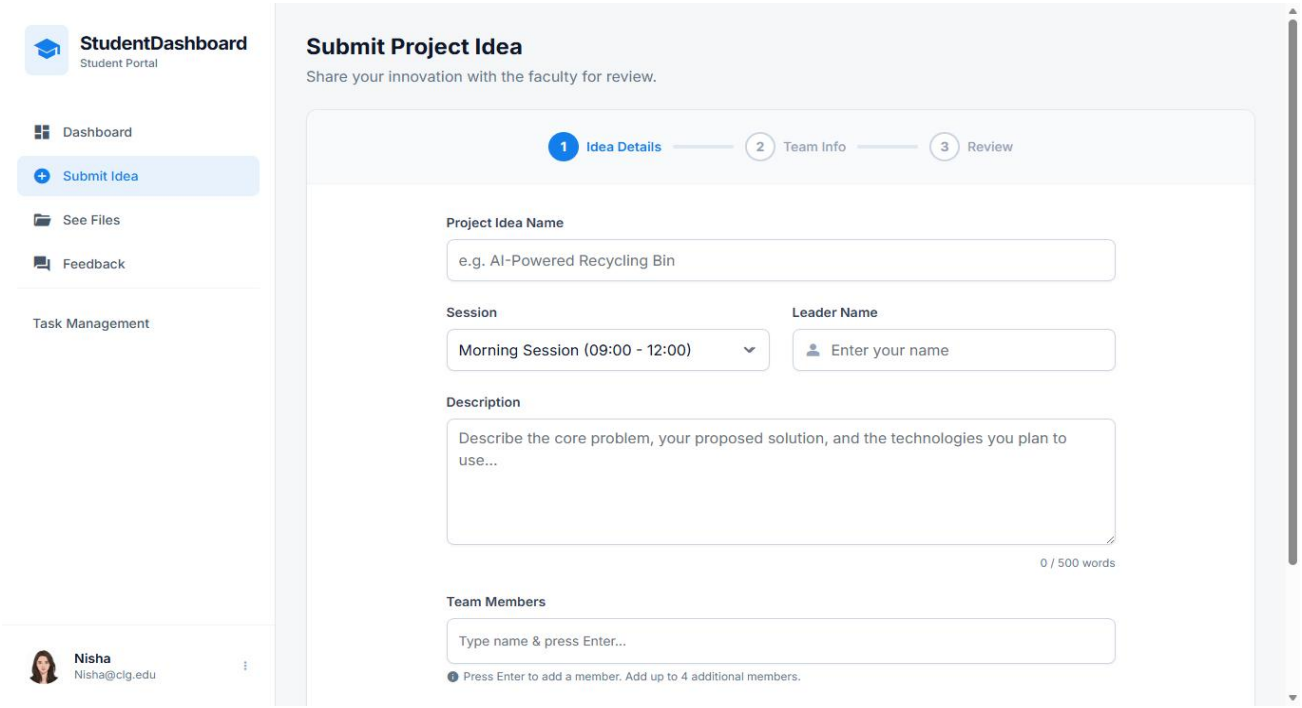



*Figure 29: Screen Image 9*

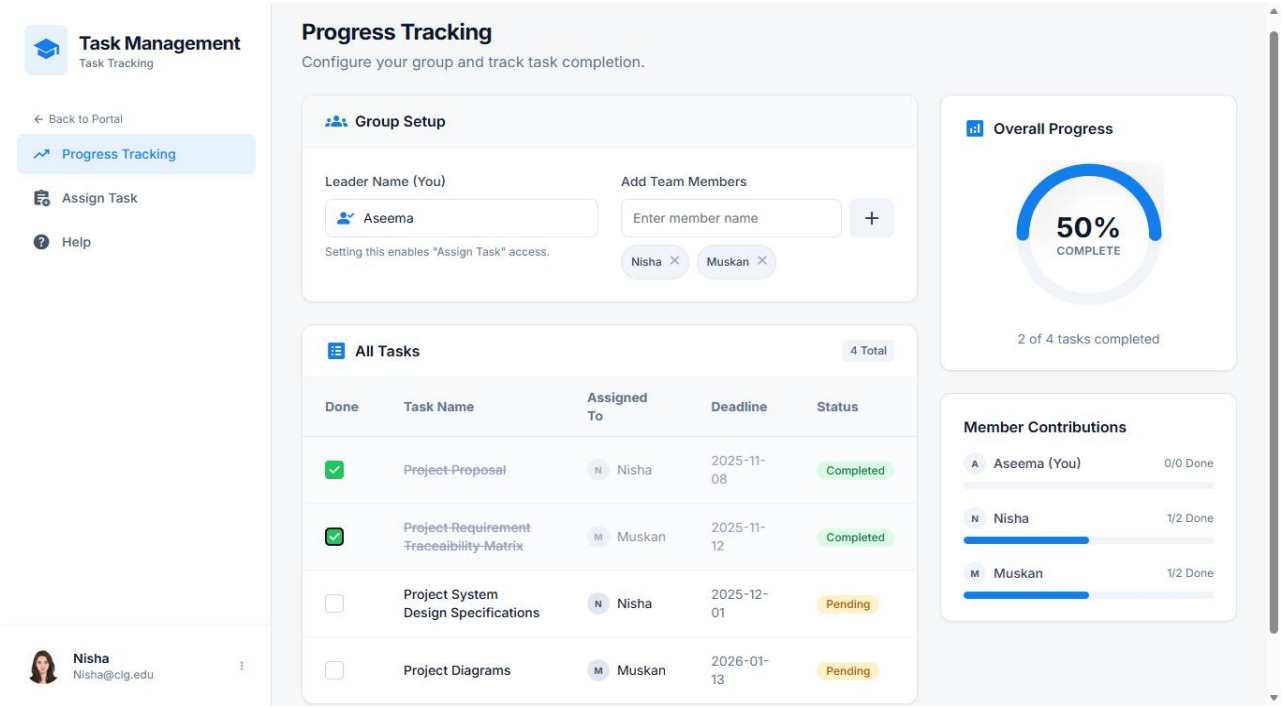



*Figure 30: Screen Image 10*

### 3.5.2 Screen Objects and Actions

**Admin Dashboard**

**Use Case 1: View Admin Dashboard**

**Screen Objects**

- Top bar: System name and Logo.
- Sidebar: Menu items “Dashboard”, “Manage Teachers”, “Manage Students”, “Roles & Permissions”.
- Dashboard cards: Small boxes with totals (teachers, students, etc.).
- Recent activity list: Simple list or table of latest actions.

**Actions**

- Click “Dashboard” in sidebar to open the dashboard.
- Read cards to see main counts (teachers, students, etc.).
- Click a card to go to the related screen (for example teachers list).
- Click profile icon and choose “Logout” to exit.

### Use Case 2: Manage Teachers

#### Screen Objects

- Sidebar: “Manage Teachers” highlighted.
- Page title: “Manage Teachers”.
- Search box and optional filter dropdowns.
- Teacher table: Columns like Name, Email, Department, Status Actions.
- Buttons: “Add Teacher” (top), “Edit” and “Delete” in each row.
- Add/Edit form: Fields for teacher details and buttons “Save”, “Cancel”.

#### Actions

- Click “Manage Teachers” in sidebar to open the screen.
- Search or filter to find a specific teacher.
- Click “Add Teacher”, fill the form and press “Save” to create a teacher.
- Click “Edit” in a row, change data, press “Save” to update.
- Click “Delete” in a row and confirm to remove the teacher.

### Teacher Dashboard

### Use Case 1: View Ideas on Teacher Dashboard

#### Screen Objects

- Top bar: System name and Logo
- Sidebar: Menu items “All Ideas”, (“Morning”, “Evening”, “Accepted”, “Rejected”) and “Progress”.
- Ideas table/list: Column Idea Title, Short Description, Group/Student, Session (Morning/Evening), Status, Actions.
- Status badge: Shows “Pending” for each idea.

#### Actions

- Click on an idea row or “View” button to open its full details.
- Click “Morning” or “Evening” to see ideas only from that session.
- Look at the Status badge to check which ideas are Accepted or Rejected.

### Use Case 2: Teacher Accepts or Rejects an Idea

#### Screen objects

- Idea detail page: Shows idea title, full description, group members, session and current status (usually Pending).
- Decision buttons: “Accept Idea” and “Reject Idea”.
- Comment box: Optional text area for feedback or reason.
- Confirmation message: Small message after accept/reject.

**Actions**

- Open a Pending idea from the list by clicking its row or “View”.
- Read the idea description and group details.
- (Optional) Write a short comment in the comment box.
- Click “Accept Idea” to approve and change status to Accepted.
- Click “Reject Idea” to decline and change status to Rejected.
- Go back to the list and see the idea now in the “Accepted” or “Rejected” list.

**Student Dashboard**

**Use Case 1: View Student Dashboard**

**Screen Objects**

- Top bar: System name and Logo.
- Sidebar: Menu items “Dashboard”, “Submit Ideas”, “Feedback”, “Task Management”.
- Total Ideas (number of ideas submitted by the student/group).
- Approved Ideas (number of approved ideas by the teacher).
- Files section: Simple list or table showing uploaded files.

**Actions**

- Click “Dashboard” in the sidebar to open the student dashboard home screen.
- View the Total Ideas card to see how many ideas the student/group has submitted.
- View the approved ideas card to check how many ideas are approved.
- Scroll down to see the list of uploaded files and check which files are already uploaded.

**Use Case 2: Submit a New Idea**

**Screen Objects**

- Sidebar: “Submit Ideas” menu item.
- Submit Idea form: Fields for Idea Title, Idea Description, Session (Morning/Evening).
- Buttons: “Submit Idea”, “Cancel”.
- Status info: After submit, idea status is set to “Pending”.

**Actions**

- Click “Submit Ideas” in the sidebar to open the idea submission page.
- Enter the idea title and description in the form fields.
- Select the correct session (Morning or Evening).
- Click “Submit Idea” to send the idea to the teacher, the system sets the status to Pending.
- Go back to the Dashboard to see the Total Ideas and Approved Ideas numbers updated.

## 3.6 Behavioral Model

### Interaction Diagram (Either Sequence and Collaboration Diagram)

### Module 1: Registration

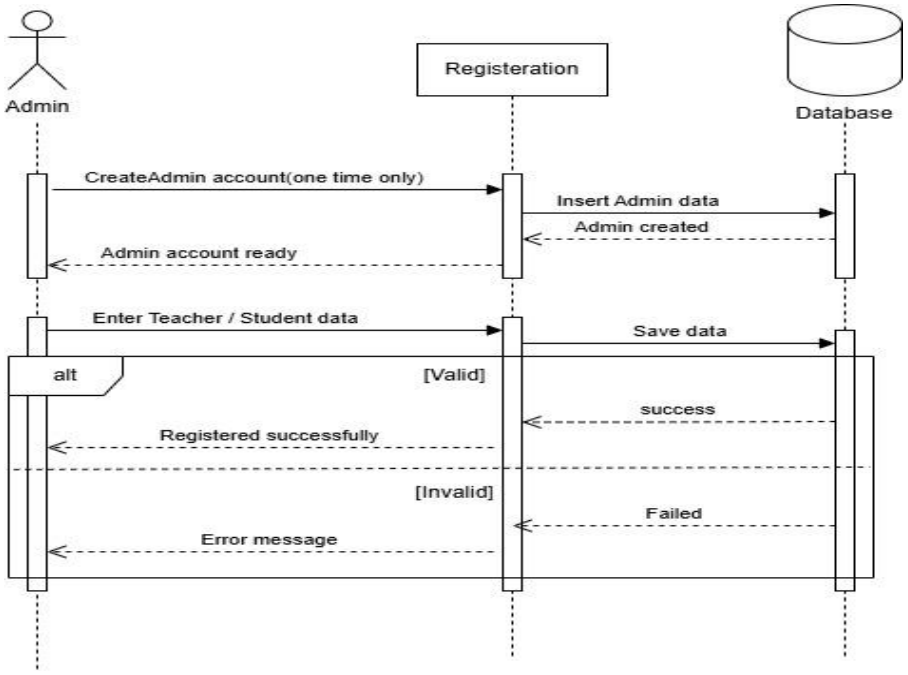



*Figure 31: Module 1 Sequence Diagram*

### Module 2: Login

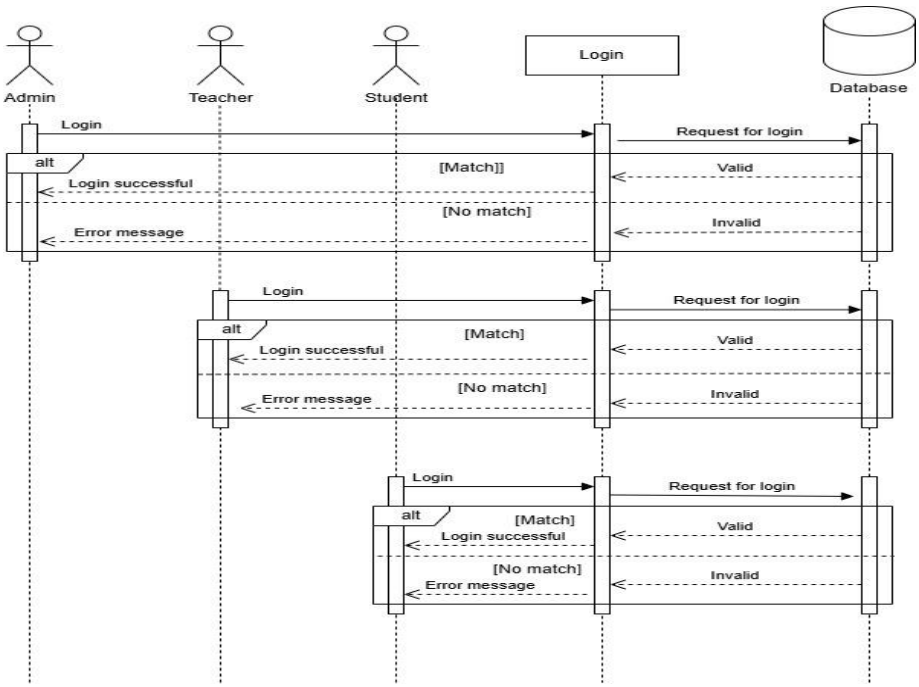



*Figure 32: Module 2 Sequence Diagram*

## Module 3: Admin Dashboard

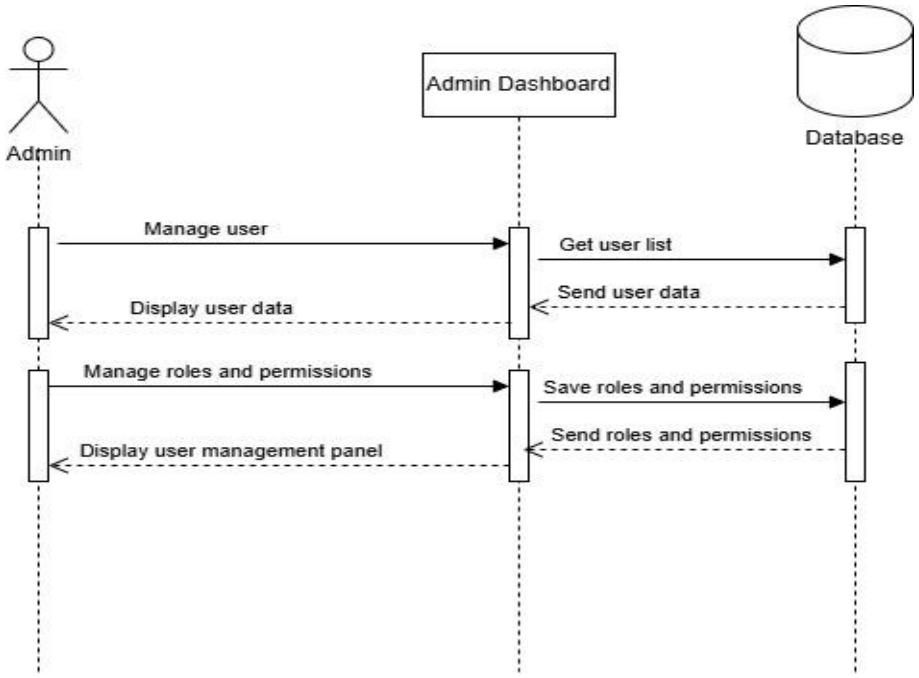



*Figure 33: Module 3 Sequence Diagram*

## Module 4: Teacher Dashboard

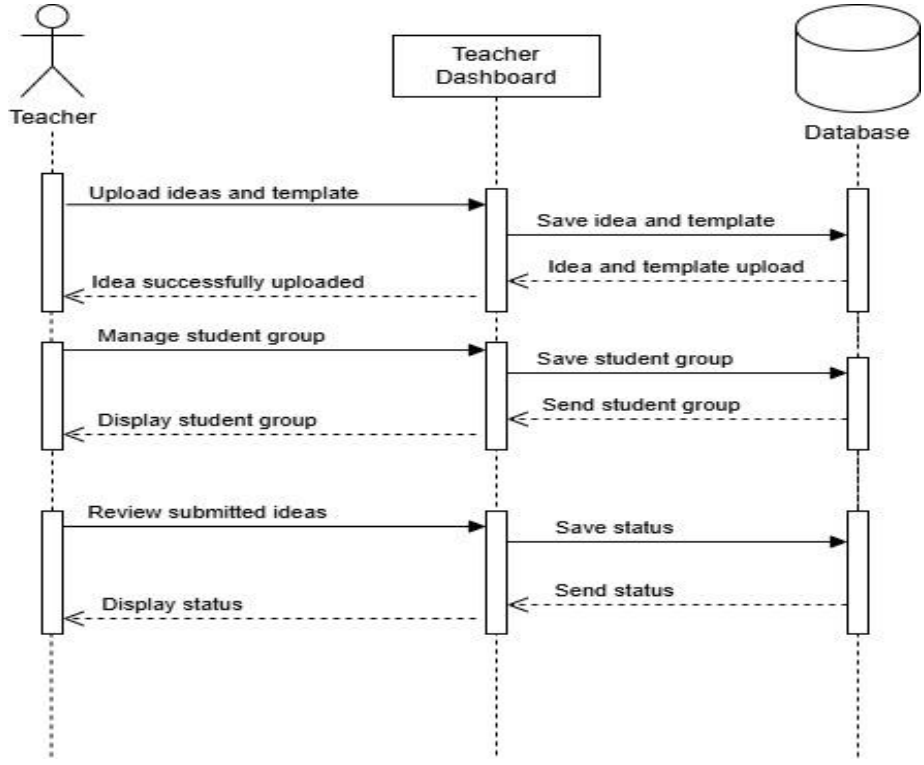



*Figure 34: Module 4 Sequence Diagram*

## Module 5: Student Dashboard

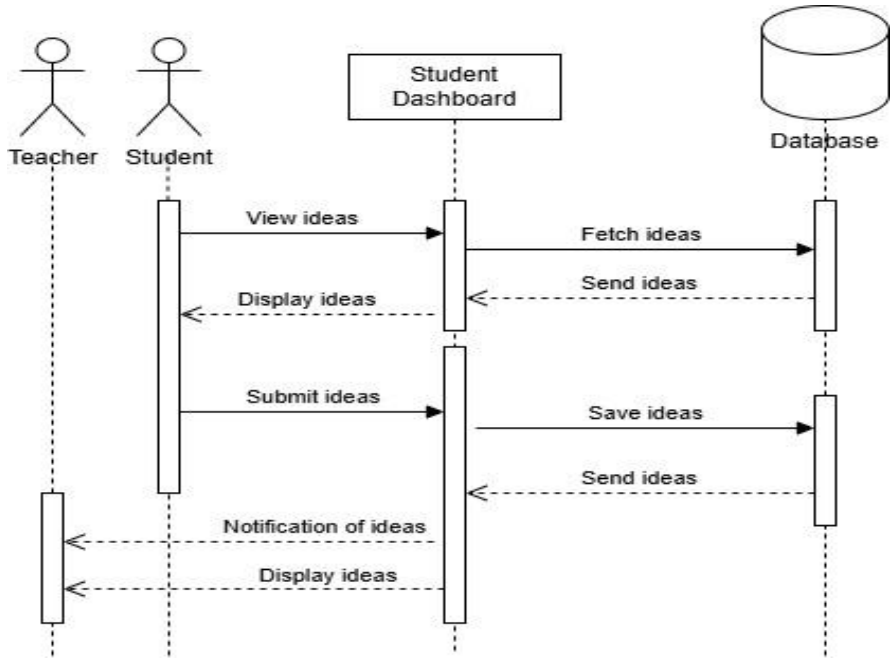



*Figure 35: Module 5 Sequence Diagram*

## Module 6: Progress Tracking

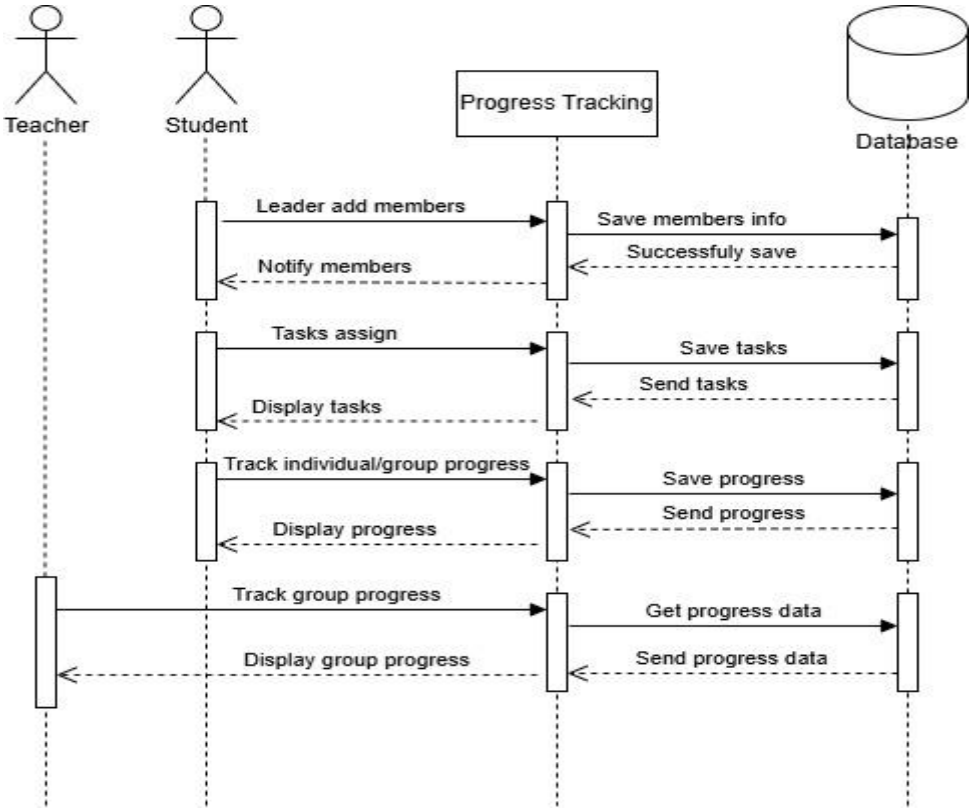



*Figure 36: Module 6 Sequence Diagram*

### Module 7: Diagram Editor

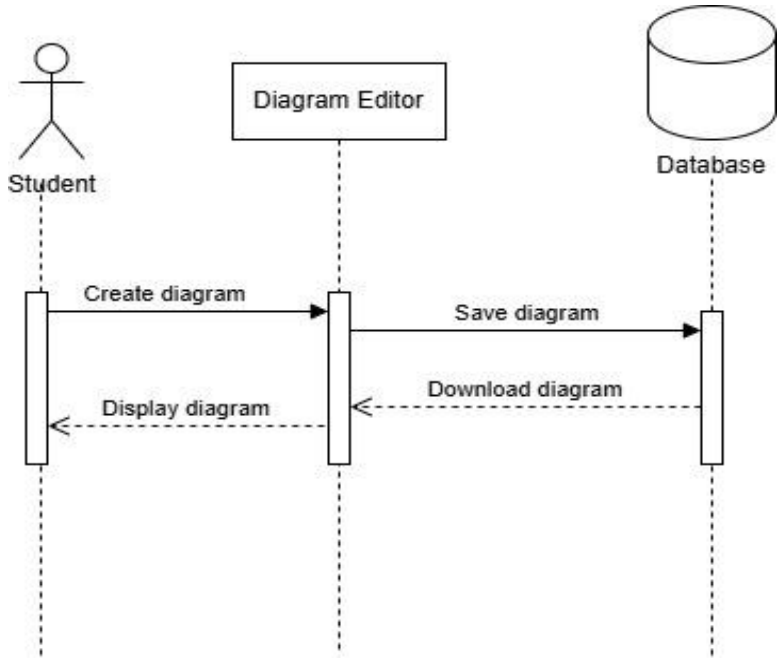



*Figure 37: Module 7 Sequence Diagram*

### Module 8: Help Box

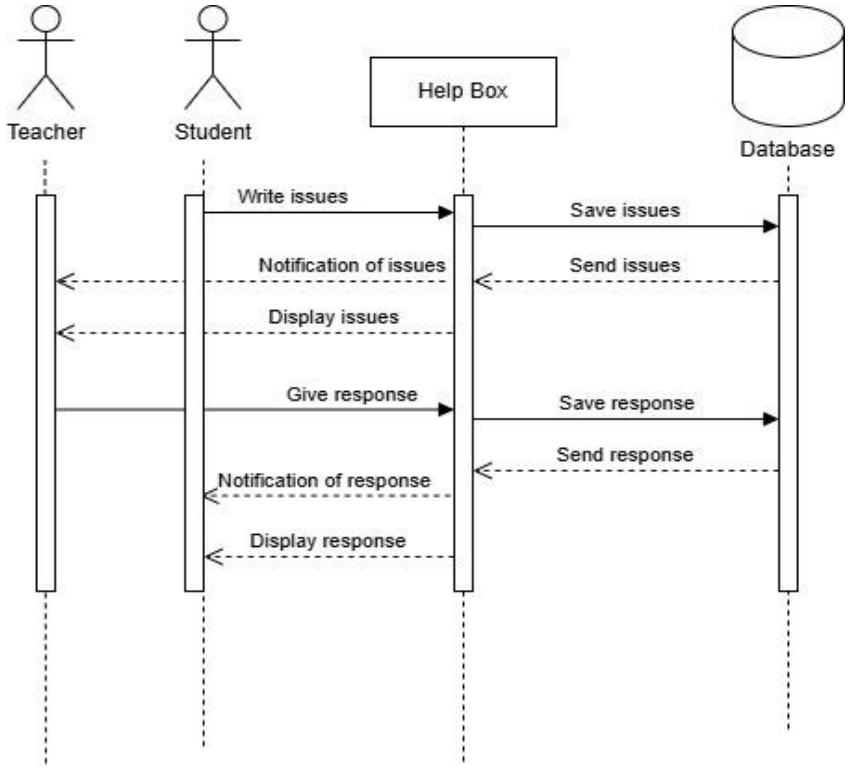



*Figure 38: Module 8 Sequence Diagram*

## State Transition Diagram

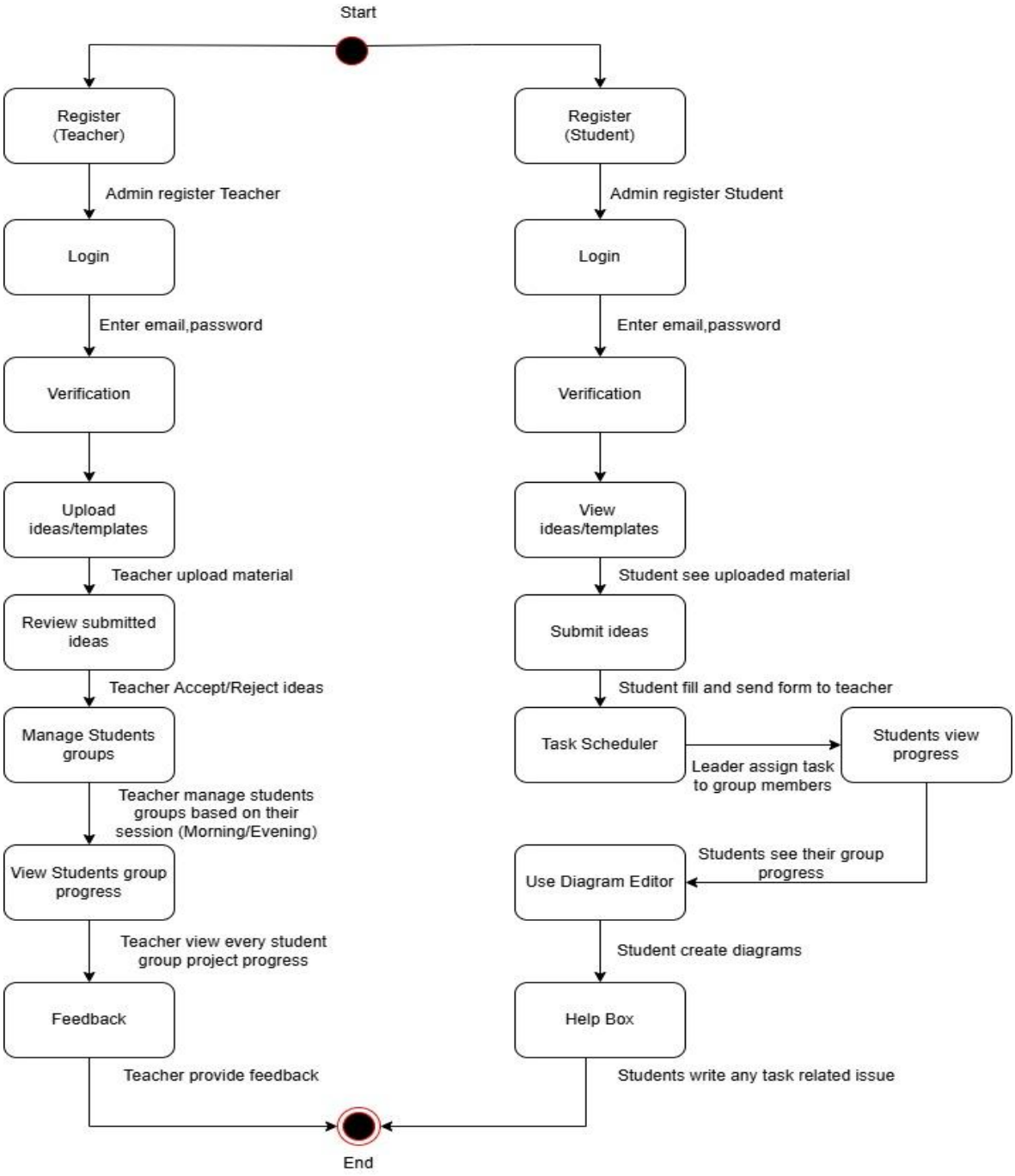



*Figure 39: State Transition Diagram*

## 3.7 Design Decisions

### 1. Object Oriented Programming (OOP)

We used Object-Oriented Design with objects like Users, Projects, Tasks and Groups to keep everything organized and easy to manage.

### 2. MERN Stack

We chose the MERN Stack because MongoDB stores data easily, React.js builds the user screen, Node.js/Express for managing backend work.

### 3. Database Design

We choose MongoDB (No SQL Database) to store data. The Database is split into collections (users, tasks etc.) and connected using Ids. This makes updates easy and keeps data correct.

### 4. MVC

We used the MVC pattern to separate the design into Parts:

- Model for data (like users, tasks, groups)
- View for what the user sees (React.js)
- Controller for system logic (Express.js)

## 3.8 Summary

We designed the system using Object Oriented Design which helps us easy to understand and simple to manage. We used the MERN Stack to build a fast and easily adjustable web app. The database is divided into small parts linked by Ids, so it is easy to update and manage. These steps make sure the system runs well and handle the project needs.




# Chapter 4: Implementation

# 4. Implementation

This chapter explains how the Project Way system is built using MERN stack. Instead of full code, it uses pseudo code to show the main logic and flow of the important features. The focus is on core modules like user registration and login, idea submission, idea approval, task management and role-based access for Admin, Teacher and Student.

## 4.1 Algorithms

**Register User**

*Table 4- 1: Algorithm 1*

| **Algorithm 1:** Register User (Admin) |
|---|
| **Input:** name, email, role |
| **Output:** Success or error message |
| **Pseudo Code:**<br>function RegisterUser(name, email, role)<br>1. existingUser ← DB.findUserByEmail(email)<br>2. if existingUser != NULL then return “Email already exists”<br>3. newUser ← {name, email, role, password: NULL, isFirstLogin: true}<br>4. DB.save (newUser)<br>5. return "User registered successfully"<br>6. end |

**Login User**

*Table 4- 2: Algorithm 2*

| **Algorithm 2:** Login User |
|---|
| **Input:** email, password |
| **Output:** Login success with token or error |
| **Pseudo Code:**<br>function LoginUser(email, password)<br>1. user ← DB.findUserByEmail(email)<br>2. if user == NULL OR user.password == NULL then return “Login error” |

| |
|---|
| 3. if email != user.email OR password !=user.password then return “Login error” |
| 4. token ← JWT.sign({userId: user.id, role: user.role}, jwtSecret) |
| 5. redirectToDashboard(user.role, token) |
| 6. return “Login success” |
| 7. end |

**Submit Project idea**

*Table 4- 3: Algorithm 3*

| **Algorithm 3:** Submit Idea (Student) |
|---|
| **Input:** studentId, title, description, session, leadername and membername |
| **Output:** Idea saved with status “Pending” or error |
| **Pseudo Code:**<br>function SubmitIdea(studentId, title, description, session, leaderName, memberName)<br>1. if title = “” OR description = “” OR session = “” OR leaderName = “” OR memberName = “” then return “Validation error”<br>2. newIdea ← {studentId, title, description, session, leaderName, memberName, status: “Pending”}<br>3. DB.save(newIdea)<br>4. return “Idea submitted successfully”<br>5. end |

**Review Idea**

*Table 4- 4: Algorithm 4*

| **Algorithm 4:** Review Idea (Teacher Accept / Reject) |
|---|
| **Input:** teacherId, ideaId, decision (Accept/Reject), comment |
| **Output:** Idea status updated or error |
| **Pseudo Code:**<br>function ReviewIdea(teacherId, ideaId, decision, comment) |

1. idea ← DB.findIdeaById(ideaId)

2. if idea == NULL OR idea.status != “Pending” then return “Idea not available for review”

3. if decision = “Accept” then idea.status ← “Accepted”

else if decision = “Reject” then idea.status ← “Rejected”

else return “Invalid decision”

4. idea.comment ← comment

5. DB.save(idea)

6. return “Idea review saved successfully”

7. end

**Assign Task**

*Table 4- 5: Algorithm 5*

| **Algorithm 5:** Assign Task (Leader) |
|---|
| **Input:** leaderId, memberId, projectId, title, description, dueDate |
| **Output:** Task created or error |
| **Pseudo Code:**<br>function AssignTask(leaderId, memberId, projectId, title, description, dueDate)<br>1. leader ← DB.findUserById(leaderId), member ← DB.findUserById(memberId)<br>2. if leader == NULL OR member == NULL OR leader.isLeader = false then return “Invalid leader or member”<br>3. newTask ← {projectId, title, description, assignedBy: leaderId, assignedTo: memberId, status: “Pending”, dueDate}<br>4. DB.save(newTask)<br>5. return “Task assigned successfully”<br>6. end |

**Diagram Editor**

*Table 4- 6: Algorithm 6*

| **Algorithm 6:** Save and Download Diagram (Diagram Editor) |
|---|
| **Input:** studentId, diagramType, diagramData |
| **Output:** Diagram saved or error |
| **Pseudo Code:**<br>function SaveDiagram(studentId, diagramType, diagramData)<br>1 if diagramData = “” then return “Validation error: diagram is empty”<br>2 diagram ← {studentId, diagramType, data: diagramData}<br>3 DB.save(diagram)<br>4 return “Diagram saved successfully”<br>5. end |

## 4.2 External APIs/SDKs

*Table 4- 7: Details of APIs used in this Project*

| Name of API | Description of API | Purpose of usage | List down the API endpoints/function class in which it is used |
|---|---|---|---|
| Nodemailer (Node.js module) | A Node.js library used to send emails from the server. | Used to send system generated emails such as notifications. | /api/notify, sendEmail() |
| Bcrypt | A library for hashing passwords securely. | Protects user passwords during account creation and password updates. | FirstTimeSetPassword API,hashPassword(), UserModel.pre(‘save’) |
| Mangoose (MongoDB ODM) | An Object Data Modeling library for MongoDB and Node.js. | Define schemas and handles database operations for users, ideas and tasks. | UserModel,IdeaModel, TaskModel |

| | | | |
|---|---|---|---|
| Express.js | A minimal and Flexible Node.js web application framework. | Manages backend routing, middleware and API handling between frontend and backend. | authRoutes.js,idea Routes.js, taskRoutes.js |
| CORS (Express middleware) | Middleware to enable Cross-Origin Resource Sharing. | Allow frontend to communicate with backend APIs running on different ports. | App.use(cors()) in server.js |
| Dotenv | A module that loads environment variables from a .env file. | Secures sensitive data such as database URI, JWT secrets keys and email credentials. | Config.js,server.js, database connection file |
| JWT (jsonwebtoken) | A library for creating and verifying JSON Web Tokens. | Handles authentication and authorization using secure token-based login sessions. | /api/login, generateToken(), verifyToken() middleware |
| React Router | A routing library for React applications. | Controls navigation between pages like Admin, Teacher and Student dashboards. | Routes defined in App.js or routes.jsx |
| Axios/Fetch API | HTTP client (Axios) or built-in browser API (Fetch) for making requests. | Sends requests from frontend to backend APIs (e.g., login, idea submit, task management) | authService.js, ideasService.js, component-level API calls |

## 4.3 Code Repository

We used Git and GitHub to manage version control and collaboration for the Project Way system. We installed Git, created GitHub accounts and set up a remote repository for the project. Each group member worked on her own local copy and branch then pushed changes to the main branch after completing features and resolving conflicts.

**Git Repository Link**

The repository for our project can be accessed using the following link:

https://github.com/buttgbuttb4-bit/Project-Way

## 4.4 Summary

This chapter explained the core implementation of the Project Way system using the MERN stack. Pseudocode was used to show the logic of the main features including user registration by the admin, first-time password setup, user login, idea submission by students and idea review by teachers. External tools and libraries such as Mongoose, JWT, Bcrypt and Express were also integrated at key points for data handling, authentication, password security and API routing. Development was managed via Github with active branch management and pull request to ensure the collaboration programming.




# Chapter 5: Testing and Evaluation

# 5. Testing and Evaluation

This chapter focuses on testing and evaluation of our project. The validation and verification of different components will be considered. For now, as our project is under development, mostly testing is done manually and no automated scripts have been followed yet.

## 5.1 Unit Testing

### Unit Test Case 1

*Table 5- 1: Unit Test Case 1*

| | |
|---|---|
| **Testcase ID** | UT1 |
| **Requirement ID** | RQ-002 |
| **Title** | Validate login with correct and empty credentials |
| **Description** | Test the login validation function that checks username and password before sending a request to the server. |
| **Objective** | Ensure that the login logic allows valid input and blocks empty fields with an error message. |
| **Driver/Precondition** | 1. Login validation function is available in the Login module.<br>2. Function accepts email and password as input and returns a validation result or error message. |
| **Test Steps** | Call the validation function with a valid email and non-empty password. |
| **Input** | Case 1: email = “student@uni.edu”, password = “Secret123”.<br>Case 2: email = “”, password = “Secret123”. |
| **Expected Result** | Case 1: Validation passes, function returns “OK” or true.<br>Case 2: Validation fails with message “Email is required”. |
| **Actual Result** | All three cases behave as expected when unit test is executed. |
| **Remarks** | Pass |

**Unit Test Case 2**

*Table 5- 2: Unit Test Case 2*

| | |
|---|---|
| **Testcase ID** | UT2 |
| **Requirement ID** | RQ-008 |
| **Title** | Calculate overall progress percentage |
| **Description** | Test the Progress Tracking module correctly and calculates the overall completion percentage based on the list of tasks. |
| **Objective** | Ensure that the progress percentage formula (completedTasks / totalTasks) * 100 is implemented correctly in code for different task lists. |
| **Driver/Precondition** | 1. ProgressTracking component function for calculating completedTasks, totalTasks and progressPercentage is available.<br>2. Task list is passed as an array of task objects with a status field ("Completed" or "Pending"). |
| **Test Steps** | 1. Create a task list with 4 tasks, where 2 have status "Completed" and 2 have status "Pending".<br>2. Call the progress calculation logic with this task list.<br>3. Read the returned progressPercentage value. |
| **Input** | Tasks array: 4 tasks, 2 completed, 2 pending |
| **Expected Result** | totalTasks = 4, completedTasks = 2, progressPercentage = 50. |
| **Actual Result** | Values returned as expected by the unit under test. |
| **Remarks** | Pass |

**Unit Test Case 3**

*Table 5- 3: Unit Test Case 3:*

| | |
|---|---|
| **Testcase ID** | UT3 |
| **Requirement ID** | RQ-010 |
| **Title** | Save diagram data to storage |
| **Description** | Test the internal save function of the Diagram Editor that stores the current diagram into local storage. |

| | |
|---|---|
| **Objective** | Ensure that when the save function is called with a diagram object, the data is correctly stored. |
| **Driver/Precondition** | 1. Diagram Editor module exposes a save function that accepts a diagramData object.<br>2. Local storage mock is available to verify saved data. |
| **Test Steps** | 1. Create a sample diagramData object with 2 nodes and 1 connector.<br>2. Call the diagram save function with this object.<br>3. Read the stored diagram data from mock storage. |
| **Input** | Diagram data:<br>Nodes: 2 (“Student”, “System”)<br>Connectors: 1 (from “Student” to “System”). |
| **Expected Result** | Stored diagram data is equal to the input diagramData. |
| **Actual Result** | Data read from storage matches the original diagram object. |
| **Remarks** | Remarks |

## 5.2 Functional Testing.

### Functional Test Case 1

*Table 5- 4: Functional Test Case 1*

| | |
|---|---|
| **Testcase ID** | FT1 |
| **Requirement ID** | RQ-001 |
| **Title** | Admin registers Teachers and Students and assigns roles |
| **Description** | Verify that Admin can create new user accounts (Teacher/Student) from the Admin Dashboard and assign them the correct role. |
| **Objective** | Ensure that the user registration form correctly saves user details and role information for Teachers and Students. |
| **Driver/Precondition** | 1. Admin account exists and is logged in.<br>2. Admin Dashboard is accessible with “User Registration/Manage Users” page. |

| | |
|---|---|
| **Test Steps** | 1. Navigate to the Admin Dashboard and open the "User Registration/Add User" page.<br>2. Enter name and email for a new teacher, select role = "Teacher" and click "Register".<br>3. Repeat the process for a new student with role = "Student".<br>4. Open the "Manage Users" page and check both new users. |
| **Input** | User 1: name = Ali, email = ali@example.com, role = Teacher<br>User 2: name = Sara, email = sara@example.com, role = Student |
| **Expected Result** | 1. System creates two new user records with the given names, emails and roles.<br>2. Both users appear in the user list with correct roles (Teacher, Student). |
| **Actual Result** | New teacher and student accounts created and listed with correct roles. |
| **Remarks** | Pass |

**Functional Test Case 2**

*Table 5- 5: Functional Test Case 2*

| | |
|---|---|
| **Testcase ID** | FT2 |
| **Requirement ID** | RQ-002 |
| **Title** | User Login Authentication |
| **Description** | Verify that user (Admin, Teacher, Student) can log in with valid credentials and redirected to their respective dashboards. |
| **Objective** | Ensure the security and role-based access control (RBAC) of the system. |
| **Driver/Precondition** | 1. User is already registered in the database.<br>2. Internet Connection is active.<br>3. "Login" page is accessible. |
| **Test Steps** | 1. Navigate to the "Login" page.<br>2. Enter a valid Email and Password. |

| | |
|---|---|
| | 3. Click on the “Login” button. |
| **Input** | Email: ali@example.com<br>Password: 2345 |
| **Expected Result** | System generates a JWT token and redirects the user to the correct Dashboard based on their role (Admin, Teacher, Student) |
| **Actual Result** | User is successfully authenticated and redirected to the correct panel. |
| **Remarks** | pass |

**Functional Test Case 3**

*Table 5- 6: Functional Test Case 3*

| | |
|---|---|
| **Testcase ID** | FT3 |
| **Requirement ID** | RQ-003 |
| **Title** | Teacher uploads project ideas and templates |
| **Description** | Verify that a teacher can upload a project idea file or template from the Teacher Dashboard. |
| **Objective** | Ensure that the upload system for ideas and templates works correctly and stores the uploaded file with correct details. |
| **Driver/Precondition** | 1. Teacher is registered and logged in.<br>2. Teacher Dashboard is accessible.<br>3. “Upload Ideas/Templates” page is available on the Teacher Dashboard. |
| **Test Steps** | 1. Navigate to the Teacher Dashboard and open the “Upload Ideas/Templates” page.<br>2. Click on the “Upload File” button.<br>3. Select a file and enter a description if required.<br>4. Click on the “Upload” button. |
| **Input** | File: Project Ideas/Template.docx<br>Title: Final Year Project Ideas |
| **Expected Result** | 1. File is successfully uploaded and saved in the system. |

| | |
|---|---|
| | 2. A success message is shown “File uploaded successfully”.<br>3.The uploaded idea/template appears in the teacher ideas/templates list with correct name and details. |
| **Actual Result** | File uploaded; success message shown; file visible in the list with correct title and name. |
| **Remarks** | Pass |

**Functional Test Case 4**

*Table 5- 7: Functional Test Case 4*

| | |
|---|---|
| **Testcase ID** | FT4 |
| **Requirement ID** | RQ-004 |
| **Title** | Teacher reviews project ideas and manage Student groups |
| **Description** | Verify that teacher can review students project ideas and correctly manage Student groups from the Teacher Dashboard. |
| **Objective** | Ensure that idea review and student group management work together correctly for the teacher. |
| **Driver/Precondition** | 1. Teacher is registered and logged in.<br>4. Some students have already submitted ideas.<br>3. Teacher Dashboard with “All Ideas” page is accessible. |
| **Test Steps** | 1. Navigate to Teacher Dashboard and open the “All Ideas” list.<br>2. Select a pending idea and change its status to “Accepted” and “rejected”. |
| **Input** | Idea status: “pending”<br>Group: Accepted/Rejected and Morning/Evening |
| **Expected Result** | 1. Idea status changes from “Pending” to “Accepted/Rejected”.<br>2. Selected students are added to the group and the group appears in the list. |
| **Actual Result** | Idea marked as Accepted/Rejected; students assigned to group successfully; group shown in list. |
| **Remarks** | Pass |

### Functional Test Case 5

*Table 5- 8: Functional Test Case 5*

| | |
|---|---|
| **Testcase ID** | FT5 |
| **Requirement ID** | RQ-005 |
| **Title** | Teacher tracks Students project progress |
| **Description** | Verify that teacher can view group-wise project progress correctly on the Teacher Dashboard. |
| **Objective** | Ensure that the Progress Tracking System shows overall progress for each group. |
| **Driver/Precondition** | 1. Teacher is registered and logged in.<br>2. Groups and tasks already exist for student group.<br>3. Teacher Dashboard and "Progress Tracking" page is accessible. |
| **Test Steps** | 1. Navigate to the Teacher Dashboard and open the "Progress Tracking" page.<br>2. Select a group from the group list.<br>3. Check the overall progress percentage shown for the group. |
| **Input** | Group: Morning/Evening |
| **Expected Result** | Overall group progress is calculated and displayed correctly based on the tasks. |
| **Actual Result** | Teacher can see overall progress value for the group. |
| **Remarks** | Pass |

### Functional Test Case 6

*Table 5- 9: Functional Test Case 6*

| | |
|---|---|
| **Testcase ID** | FT6 |
| **Requirement ID** | RQ-006 |
| **Title** | Students views shared project ideas and template files |
| **Description** | Verify that a student can successfully view the list of project ideas and templates uploaded by teachers. |

| | |
|---|---|
| **Objective** | Ensures that the system correctly fetches data from the database and it displays on the student dashboard. |
| **Driver/Precondition** | 1. Student is registered and logged in.<br>2. Student Dashboard is accessible.<br>3. Teachers have already uploaded project ideas/templates. |
| **Test Steps** | 1. Navigate to the Student Dashboard.<br>2. Locate and click on "View project ideas/templates" section.<br>3. Observe the list of available ideas/templates displayed on the screen. |
| **Input** | Click event on the "View" button. |
| **Expected Result** | The list of project ideas and templates is displayed correctly. |
| **Actual Result** | List of shared ideas/templates is visible on the dashboard with correct detail. |
| **Remarks** | Pass |

**Functional Test Case 7**

*Table 5- 10: Functional Testing 7*

| | |
|---|---|
| **Testcase ID** | FT7 |
| **Requirement ID** | RQ-007 |
| **Title** | Submit Project Idea |
| **Description** | Verify that a student can submit their idea by filling out all required fields. |
| **Objective** | Ensure that the submit idea is correctly validated and stored in the database with a "pending" status. |
| **Driver/Precondition** | 1. Student is registered and logged in.<br>2. "Submit idea" page is accessible. |
| **Test Steps** | 1. Navigate to the "Submit idea" page.<br>2. Enter Project Idea Name, Description, Session, Leader Name and Team Members. |
| **Input** | Project Idea Name: "Project Way" |

| | |
|---|---|
| | Description: “FYP Management Project”<br>Session: “Morning”<br>Leader Name: “Aseema”<br>Team Members: “Nisha”, “Muskan” |
| **Expected Result** | Record is created in the database with status “Pending” and success message is displayed. |
| **Actual Result** | Idea is submitted successfully. |
| **Remarks** | Pass |

**Functional Test Case 8**

*Table 5- 11: Functional Test Case 8*

| | |
|---|---|
| **Testcase ID** | FT8 |
| **Requirement ID** | RQ-008 |
| **Title** | Students manage tasks and track progress |
| **Description** | Verify that student can see assigned tasks, update their status and view progress on the Task Management page. |
| **Objective** | Ensure that the task management feature allows students to update their tasks correctly and that progress is shown properly. |
| **Driver/Precondition** | 1. Student is registered and logged in.<br>2. Group leader has assigned at least one task to this student.<br>3. “Task Management” page is accessible from the student side. |
| **Test Steps** | 1. Navigate to the “Task Management” page from the student dashboard.<br>2. View the list of assigned tasks.<br>3. Select one task and change its status from “Pending” to “In Progress” or “Completed”.<br>4. Save or update the task. |
| **Input** | Task: “Project Proposal”<br>New Status: “Completed” |

| | |
|---|---|
| **Expected Result** | 1. Selected task status is updated in the database to the new value.<br>2. The updated status and overall progress for the group is refreshed. |
| **Actual Result** | Task status updated and progress view. |
| **Remarks** | Pass |

**Functional Test Case 9**

*Table 5- 12: Functional Test Case 9*

| | |
|---|---|
| **Testcase ID** | FT9 |
| **Requirement ID** | RQ-010 |
| **Title** | Students create, save and download diagrams |
| **Description** | Verify that student can create a diagram on the Diagram Editor page, save it and download it as an image. |
| **Objective** | Ensure that the diagram editor tool correctly saves diagram data and allows the student to download the diagram. |
| **Driver/Precondition** | 1. Student is registered and logged in.<br>2. Diagram Editor page is accessible from the student dashboard. |
| **Test Steps** | 1. Navigate to the “Diagram Editor” page.<br>2. Create a simple diagram by adding a few shapes and connectors.<br>3. Click on the “Save” button to save the diagram.<br>4. After save, click on the “Download” button and choose a file format (e.g., PNG ). |
| **Input** | Diagram type: “Use Case Diagram”<br>File format for download: “PNG” |
| **Expected Result** | 1. Diagram is saved in the system<br>2. The diagram file is generated and downloaded to the user device in the selected format. |
| **Actual Result** | Diagram saved and Download. |
| **Remarks** | Pass |

## 5.3 Integration Testing

### Integration Test Case 1

*Table 5- 13: Integration Test Case 1*

| | |
|---|---|
| **Testcase ID** | IT1 |
| **Requirement ID** | RQ-009 |
| **Title** | Students submit issues and receive feedback |
| **Description** | Verify that when a student submits an issue through the Help Box the request is stored, a teacher can reply and the student can see the feedback. |
| **Objective** | Ensure the integration between the Help Box, teacher reply and the student view the responses. |
| **Driver/Precondition** | 1. Student is registered and logged in.<br>2. Help Box page is accessible.<br>3. Teacher account exists and can open the feedback page. |
| **Test Steps** | 1. Student opens the “Help Box” page.<br>2. Student enters a subject and message describing the issue and clicks “Submit”.<br>3. Teacher give feedback to Students. |
| **Input** | Subject: “Issue”<br>Message: “Description” |
| **Expected Result** | 1. Help request is saved in the system.<br>2. Teacher give feedback.<br>3. Student can see the reply message |
| **Actual Result** | Request created; teacher reply saved; student can see feedback. |
| **Remark** | Pass |

**Integration Test Case 2**

*Table 5- 14: Integration Test Case 2*

| | |
|---|---|
| **Testcase ID** | IT2 |
| **Requirement ID** | RQ-011 |
| **Title** | Email alert are sent to users when they are registers.<br>Alerts are sent to dashboards when users perform specific action. |
| **Description** | Verify that when a Admin Register Users the notification system triggers an email alert to the Users. |
| **Objective** | Ensure the integration between notification system and email service (e.g., Nodemailer). |
| **Driver/Precondition** | 1. Student and Teacher users are registered with valid email addresses.<br>2. Email configuration (SMTP) is set up and running. |
| **Test Steps** | 1. Admin logs in and register Student/Teacher.<br>3. System creates a notification entry for Student/Teacher and sends an email through the email service.<br>4. Student/Teacher checks their email inbox for a new message about the registration.<br>5. Teacher and Student logs in.<br>6. Teacher upload files and notification send to Student Dashboard.<br>7.Student submit issue and notification send to Teacher dashboard. |
| **Input** | name: “Ali”<br>email: Ali@exp.com<br>file: “Ideas.pdf”<br>Message: “Description” |
| **Expected Result** | 1. An email is sent to the Student/ Teacher registered email address.<br>2. File is sent to Student Dashboard.<br>3. Issue message is sent to Teacher Dashboard. |
| **Actual Result** | As Expected |

| | |
|---|---|
| **Remark** | Pass |

## 5.4 Performance Testing

### Performance Test Case 1

*Table 5- 15: Performance Test Case 1*

| | |
|---|---|
| **Testcase ID** | PT1 |
| **Requirement ID** | RQ-002 |
| **Title** | Measure login and dashboard response time |
| **Description** | Assess how fast the system logs in a user (Teacher/Student) and loads their main dashboard. |
| **Objective** | Ensure that the login flow and initial dashboard page respond within acceptable time (for example, under 2 seconds) for a good user experience. |
| **Driver/Precondition** | 1. Test Teacher and Student accounts already exist.<br>2. Server and database are running in normal conditions.<br>3. A tool is available to measure response time (browser DevTools, Postman). |
| **Test Steps** | 1. Open the login page in the browser.<br>2. Start a timer to record time.<br>3. Enter valid Teacher/Student credentials and click “Login”.<br>4. Measure time from clicking “Login”. |
| **Input** | Teacher email/password (valid)<br>Student email/password (valid) |
| **Expected Result** | For both Teacher and Student, total time from form submit to dashboard load is within the defined target (e.g., ≤ 2 seconds on normal network). |
| **Actual Result** | Measured response times within target range. |
| **Remark** | Pass |

## 5.5 Summary

This chapter explained the testing strategies used to check the functionality, reliability and performance of the Project Way system. Different levels of testing including unit, functional, integration and performance testing, were applied to ensure that all important features such as login, idea submission, idea review, task management and notifications work correctly.




# Chapter 6: System Conversion

# 6. System Conversion

System conversion is the process of transforming from an existing system to a new one. It involves shifting data, processes and user operations from the legacy environment to the newly developed system. Effective system conversion ensures minimal disruption, data integrity and seamless user adaption.

## 6.1 Conversion Method

For Project Way, we use pilot conversion method. The system is first implemented for a small controlled group of final-year students and their project supervisors in the Computer Science department. This allows real testing of project idea submission, task assignment, progress tracking, diagram and teacher feedback before full deployment.

## 6.2 Deployment

Deployment means moving Project Way from the development environment to a live environment where students and teachers can access it through their browsers. This includes building the React frontend, configuring the Node.js/Express backend and database (MongoDB), setting up secure access and testing that the system works correctly for real users.

### 6.2.1 Deployment Steps

*Table 6- 1: Deployment Steps*

| Step No. | Task | Details |
|---|---|---|
| 1 | Frontend Build | Build production bundle of the React app using Vite (npm run build). |
| 2 | Backend Configuration | Deploy Node.js/Express API to a server (Render/railway/VM) and connect it to MongoDB. |
| 3 | Environment & Security | Configure .env variables (DB URI, JWT secret) and enable HTTPS where possible. |
| 4 | Hosting Frontend | Upload built frontend to a hosting service (Vercel/Netlify) and set API base URL. |
| 5 | Functional Testing | Test login, project idea submission, task assignment, progress tracking, files and diagrams on the live URL. |
| 6 | Pilot User Access | Share live link with selected final-year students and teachers for pilot use. |
| 7 | Deployment Confirmation | Collect feedback, fix issues and confirm the system is stable for full rollout. |

### 6.2.2 Data Conversion

*Table 6- 2: Data Conversion*

| Step No. | Task | Details |
|---|---|---|
| 1 | Data Preparation | Collect existing project titles, student groups and teachers details from department records. |
| 2 | Data Cleaning | Remove duplicates, fix spelling/formatting and ensure each project has valid leader and members. |
| 3 | Data Transformation | Convert the cleaned data into JSON format matching project, student and teacher schema used in MongoDB. |
| 4 | Data Loading | Use MongoDB tools to insert this data into the Project Way database collections. |

### 6.2.3 Training

This section represents a basic user manual to train users in using the application. Since the Project Way serves two primary user types: Students and Teachers, the training focuses on guiding them through the most important use cases.

The following instructions are structured step-by-step and provide clarity for first time users to interact with the system effectively:

*Table 6- 3: Training Instruction 1*

| Step No. | Instruction |
|---|---|
| 1 | Login using your teacher email credentials. |
| 2 | From the dashboard, open the “All Ideas” page. |
| 3 | Click “Accept/Reject” and give feedback to students. |
| 4 | Check the “Progress” page to see each student groups project progress. |

*Table 6- 4: Training Instruction 2*

| Step No. | Instruction |
|---|---|
| 1 | Login using your student email credentials. |
| 2 | From the dashboard, open the “Submit Idea” page. |
| 3 | Fill in project title, description, session and team members. |

| | |
|---|---|
| 4 | Click “Submit Idea” to send it to the teacher. |
| 5 | Check the “Feedback” page to see approval status and comments. |

*Table 6- 5: Training Instruction 3*

| Step No. | Instruction |
|---|---|
| 1 | Navigate to the “Task Management” section for a project. |
| 2 | Add tasks with title, assign to and deadline. |
| 3 | Team members update task status to “Completed” when done. |
| 4 | Open the “Progress Tracking” page to view overall completion. |

## 6.3 Post Deployment Testing

Post-deployment testing was conducted to confirm that the Project Way system is correctly deployed, configured and working as expected for students and teachers. The goal was to verify data integrity, core features and user experience on the live environment.

### 6.3.1 Post Deployment Testing Steps

*Table 6- 6: Post Deployment Testing Steps*

| Step No. | Test Activity | Expected Outcome |
|---|---|---|
| 1 | Launch Web Application | Live URL open in the browser without error. |
| 2 | Login Functionality | Students and teachers can log in with valid credentials; invalid login is rejected. |
| 3 | Project Idea Submission | Students submit a project idea and see it listed in their dashboard. |
| 4 | Task & Progress Tracking | Tasks can be added, updated and marked completed; overall progress updates correctly. |
| 5 | File & Diagram Handling | Project files and UML diagrams can be uploaded, viewed and downloaded without corruption. |
| 6 | Feedback Module | Teachers can send feedback; students can view status (pending/accepted/rejected). |
| 7 | Data Integrity Verification | Project, task and feedback data remain correct after refresh and logout/login. |
| 8 | Role-Based Access | Student and teacher permissions behave according |

| | | |
|---|---|---|
| | | to their roles. |
| 9 | Performance Check | Pages load within acceptable time on common browsers and network conditions. |

## 6.4 Challenges and Solutions

During deployment and post-deployment of Project Way, some technical and user-related challenges were faced. The main issues and their solutions are listed below.

*Table 6- 7: Challenges and Solution*

| Challenges | Solution Implemented |
|---|---|
| API and CORS configuration issues | Adjusted backend CORS settings and environment variables so frontend could call all APIs correctly. |
| Database schema changes during testing | Updated MongoDB schemas and added migration scripts to keep existing data compatible. |
| Progress and task status not updating live | Fixed API logic and refreshed state handling on the frontend so status changes appear immediately. |
| File and diagram uploads failing for some users | Added file size/type validation and improved error handling on upload endpoints. |
| Performance slowdown with larger project data | Optimized queries and reduced unnecessary API calls on the dashboard. |
| Role-based access confusion (student vs teacher) | Reviewed access rules and adjusted UI options so each role only sees allowed actions. |

## 6.5 Summary

This chapter explains how Project Way is deployed, how basic training is provided to students and teachers and how existing project data is converted into a useful format for the system. It also highlights the main challenges faced during deployment and post-deployment and briefly describes the solutions applied to keep the system stable, usable and reliable for real users.




# Chapter 7: Conclusion

# 7. Introduction

This chapter presents the overall completion status of the main objectives of Project Way System. It summarizes which features are fully implemented, tested and available and confirms that all core objectives have been achieved.

## 7.1 Evaluation

*Table 7- 1: Progress Table*

| Objectives | Status |
|---|---|
| Allow admin to log in. | Completed |
| Allow admin to register teachers. | Completed |
| Allow admin to register students. | Completed |
| Implement role-based access control (Student, Teacher). | Completed |
| Manage users (Student, Teacher). | Completed |
| Allow teachers to log in. | Completed |
| Allow teachers to upload project ideas files and template. | Completed |
| Enable teachers to review submitted projects ideas and give feedback. | Completed |
| Enable teachers to view whole group progress. | Completed |
| Allow students to log in. | Completed |
| Allow students to submit project ideas for approval. | Completed |
| Enable task creation, assignment and status update for team members. | Completed |
| Implement project progress tracking with visual indicators. | Completed |
| Provide diagram editor support for UML diagrams (use case, class etc.). | Completed |
| Allow students to submit project task issues. | Completed |
| Allow students to view feedback. | Completed |

## 7.2 Requirement Traceability Matrix

*Table 7- 2: Requirement Traceability Matrix*

| Requirement ID | Requirement Description | Design Specification | Code | Test ID |
|---|---|---|---|---|
| **RQ-001** | Admin registers Teachers and Students and assign roles | Admin Dashboard (User registration and role assignment) | AdminDashboard.jsx, API routes | FT1 |
| **RQ-002** | Secure authentication | Login Page (Authentication | App.jsx, Login.jsx, | UT1, |

| | | | | |
|---|---|---|---|---|
| | system for Teachers and Students | and Authorization) | auth API files | FT2, PT1 |
| **RQ-003** | Teachers upload project ideas and templates | Teachers Dashboard (Upload System for ideas and templates) | TeacherDashboard.jsx, upload endpoints | FT3 |
| **RQ-004** | Teachers review project ideas and manage Students in groups | Teachers Dashboard (Student group Management) | TeacherDashboard.jsx, group APIs | FT4 |
| **RQ-005** | Teachers track Students project progress | Teachers Dashboard (Progress Tracking System) | TeacherDashboard.jsx, progress APIs | UT2, FT5 |
| **RQ-006** | Students view shared project ideas and templates | Student Dashboard (View ideas/templates) | Dashboard.jsx | FT6 |
| **RQ-007** | Students select, submit project ideas and see status | Students Dashboard (Submission System) | SubmitIdea.jsx, Dashboard.jsx | FT7 |
| **RQ-008** | Students manage tasks and track progress | Task Management (Assign task and track progress) | AssignTask.jsx, ProgressTracking.jsx | UT2, FT8 |
| **RQ-009** | Students submit issues and receive feedback | Help Box Page (Help Box for issue reporting) | Help.jsx, Feedback.jsx | IT1 |
| **RQ-010** | Students create, save and download diagrams | Diagram Editor Page (Diagram Editor tool) | DiagramEditor.jsx | UT3, FT9 |

| | | | | |
|---|---|---|---|---|
| **RQ-011** | Notification system | Notification System (notify teachers and students) | Backend service files | IT2 |

## 7.3 Conclusion

This project was the need of the hour for academic project management. Users accepted the app, used it regularly and responded positively to its features. Overall, it successfully met its main objectives of improving project coordination between students and teachers.

## 7.4 Future Work

- **AI-Driven Recommendation System**

  Implementing an AI module to suggest project ideas to students based on the skill set, interests and technology preferences.

- **Mobile Application Development**
  Creating a cross-platform mobile app to provide real-time push notifications for task deadlines and project approvals.
- **Automated Progress Reports**
  Developing a feature that generates weekly or monthly progress reports automatically based on the tasks completed in the student dashboard.
- **Peer Review System**
  Introducing a module where students can provide constructive feedbacks on each other's project ideas to foster a collaborative learning environment.

# References

CodeWithHarry. (n.d.). *"Node.js tutorials for beginners"*. Retrieved from YouTube: https://www.youtube.com/@codewithharry

Evans, A. Y. (2017). *"ClickUp"*. Retrieved from Internet: https://clickup.com

Joel, S. (2011, September 13). *"Trello"*. Retrieved from Internet: https://www.trello.com

Microsoft. (2017, March 14). *"Microsoft Teams"*. Retrieved from Internet: https://teams.microsoft.com

ReactContributers. (n.d.). *"A JavaScript Library for Building User Interfaces"*. Retrieved from Internet: https://reactjs.org/

W3Schools. (n.d.). *"Web Development Tutorials"*. Retrieved from Internet: https://www.w3schools.com/

# Appendix-A
# Use Case Description Template

# A. Appendix-A: Use Case Description(Fully Dressed Format)

### Use Case 1

*Table A- 1: Use Case 1*

| | |
|---|---|
| **Use Case ID** | UC-1 |
| **Use Case Name** | Login with Email and Password |
| **Actors** | Admin, Teacher, Student |
| **Description** | This use case allows an existing user to log in to Project Way using email and password. |
| **Trigger** | User opens the web app and clicks the “Login” button. |
| **Preconditions** | 1. User account already exists in the system.<br>2. Login page is loaded. |
| **Postconditions** | User is authenticated.<br>User is redirected to their dashboard (Admin, Teacher or Student). |
| **Normal Flow** | 1. User opens the web app.<br>2. Login page appears.<br>3. User enters email and password.<br>4. User clicks “Login”.<br>5. System checks email and password in the database.<br>6. If valid, system creates a session.<br>7. System redirects user to the correct dashboard based on role. |
| **Alternative Flows** | 1. System does not find a matching account.<br>2. System shows an error message like "Invalid email or password".<br>3. User can try again. |
| **Business Rules** | 1. Only valid registered users can log in.<br>2. Role (Admin, Teacher, Student) is checked after login. |
| **Assumptions** | 1. Authentication backend is working.<br>2. User has internet access. |

**Use Case 2**

*Table A- 2: Use Case 2*

| | |
|---|---|
| **Use Case ID** | UC-2 |
| **Use Case Name** | Upload Project Idea / Template |
| **Actors** | Teacher |
| **Description** | Teacher uploads project ideas and templates for students to see. |
| **Trigger** | Teacher clicks “Upload” in Teacher dashboard. |
| **Preconditions** | 1. Teacher is logged in.<br>2. Teacher dashboard is open. |
| **Postconditions** | New idea/template is saved and visible to students. |
| **Normal Flow** | 1. Teacher opens Teacher dashboard.<br>2. Teacher clicks on “Upload”.<br>3. Teacher uploads a ideas/template file and write message.<br>4. Teacher clicks “Upload”.<br>5. System saves the idea/template in the database.<br>6. System shows a success message. |
| **Alternative Flows** | 1. Teacher leaves input field empty.<br>2. System shows a validation message.<br>3. Teacher fills the missing fields and submit again. |
| **Business Rules** | Only Teachers can upload ideas/templates. |
| **Assumptions** | File upload is working. |

**Use Case 3**

*Table A- 3: Use Case 3*

| | |
|---|---|
| **Use Case ID** | UC-3 |
| **Use Case Name** | Submit Project Idea |
| **Actors** | Student |
| **Description** | Student submits project idea to the teacher for approval. |

| | |
|---|---|
| **Trigger** | Student clicks “Submit” in Student dashboard. |
| **Preconditions** | 1. Student is logged in.<br>2. Student dashboard is open. |
| **Postconditions** | Student idea is saved with status “Pending”. |
| **Normal Flow** | 1. Student opens Student dashboard.<br>2. Student clicks “Submit Idea”.<br>3. Student fills form (title, description, group information).<br>4. Student clicks “Submit”.<br>5. System saves the idea with status “Pending”.<br>6. System shows a confirmation message. |
| **Alternative Flows** | 1. Some required fields are empty.<br>2. System shows an error message.<br>3. Student fills them and submit again. |
| **Business Rules** | One group can have only one active idea request at a time. |
| **Assumptions** | Teacher will later review the idea. |

**Use Case 4**

*Table A- 4: Use Case 4*

| | |
|---|---|
| **Use Case ID** | UC-4 |
| **Use Case Name** | Review Ideas and Manage Groups |
| **Actors** | Teacher |
| **Description** | Teacher reviews student ideas and places students lists (accepted, rejected). |
| **Trigger** | Teacher clicks “All ideas” in Teacher dashboard. |
| **Preconditions** | 1. Teacher is logged in.<br>2. There are submitted student ideas. |
| **Postconditions** | 1. Each idea gets a status (Accepted/Rejected).<br>2. Students are assigned to lists. |

| | |
|---|---|
| **Normal Flow** | 1. Teacher opens Teacher dashboard.<br>2. Teacher see ideas request.<br>4. Teacher opens one idea and reads details.<br>5. Teacher sets status (Accepted or Rejected).<br>7. System saves status and group information. |
| **Alternative Flows** | System shows message “No idea requests”. |
| **Business Rules** | Only the Teacher can accept or reject ideas of Students. |
| **Assumptions** | Groups and sections (morning/evening) are already defined in the system. |

**Use Case 5**

*Table A- 5: Use Case 5*

| | |
|---|---|
| **Use Case ID** | UC-5 |
| **Use Case Name** | Create Tasks and Track Progress |
| **Actors** | Student (Leader and members) |
| **Description** | Team leader creates tasks and group members track progress. |
| **Trigger** | Leader clicks “Tasks Management” in Student dashboard. |
| **Preconditions** | 1. Student is logged in and type leader name and system assign them as group leader.<br>2. Group exists. |
| **Postconditions** | Tasks are saved and group progress is updated. |
| **Normal Flow** | 1. Leader opens Student dashboard.<br>2. Leader goes to “Tasks Management” page.<br>3. Leader clicks “Add Task”.<br>4. Leader enters task title, description, due date and assigns member.<br>5. Leader clicks “Save”.<br>6. System saves task and updates progress bar.<br>7. Group members can view their tasks and mark them as done. |

| | |
|---|---|
| **Alternative Flows** | If the student is not in a group then system shows message “No group assigned”. |
| **Business Rules** | Only group leader can create or delete tasks. |
| **Assumptions** | Progress bar is calculated from completed vs total tasks. |

**Use Case 6**

*Table A- 6: Use Case 6*

| | |
|---|---|
| **Use Case ID** | UC-6 |
| **Use Case Name** | View Group Progress |
| **Actors** | Teacher |
| **Description** | Teacher views project progress of each student group. |
| **Trigger** | Teacher clicks “Progress Tracking” in Teacher dashboard. |
| **Preconditions** | 1. Teacher is logged in.<br>2. Groups and tasks exist. |
| **Postconditions** | Teacher sees whole student groups progress. |
| **Normal Flow** | 1. Teacher opens Teacher dashboard.<br>2. Teacher clicks “Progress Tracking”.<br>3. System shows list of groups.<br>4. Teacher selects list.<br>5. System shows progress bar/percentage. |
| **Alternative Flows** | System shows message “No tasks created for this group”. |
| **Business Rules** | Teacher view whole Student groups progress. |
| **Assumptions** | Task status is up to date from student side. |

### Use Case 7

*Table A- 7: Use Case 7*

| | |
|---|---|
| **Use Case ID** | UC-7 |
| **Use Case Name** | Create, Save and Download Diagram |
| **Actors** | Student |
| **Description** | Student creates diagrams using the Diagram Editor and can save or download them. |
| **Trigger** | Student clicks “Diagram Editor” in Student dashboard. |
| **Preconditions** | Student is logged in. |
| **Postconditions** | Diagram is saved in the system or downloaded as an image. |
| **Normal Flow** | 1. Student opens Student dashboard.<br>2. Student clicks “Diagram Editor”.<br>3. Diagram Editor page opens.<br>4. Student drags shapes and connectors to build a diagram.<br>5. Student clicks “Save” to store it.<br>6. Student can click “Download” to download as an image. |
| **Alternative Flows** | 1. Student tries to leave page.<br>2. System can warn about unsaved changes. |
| **Business Rules** | Diagrams are linked to the project. |
| **Assumptions** | Browser supports canvas/SVG and download. |

### Use Case 8

*Table A- 8: Use Case 8*

| | |
|---|---|
| **Use Case ID** | UC-8 |
| **Use Case Name** | Submit Issue and View Teacher Feedback |
| **Actors** | Student, Teacher |
| **Description** | Student sends an issue to Teacher and Teacher give feedback. |
| **Trigger** | Student clicks “Help Box” in Student dashboard. |

| | |
|---|---|
| **Preconditions** | Student is logged in. |
| **Postconditions** | Issue and teacher reply are saved and viewable. |
| **Normal Flow** | 1. Student opens Student dashboard.<br>2. Student clicks “Help Box”.<br>3. Student writes issue title and message.<br>4. Student clicks “Send”.<br>5. System saves issue and notifies the Teacher.<br>6. Teacher opens Teacher dashboard and goes to “Help Box”.<br>7. Teacher reads the issue and types a reply.<br>8. System saves the reply and notifies the Student. |
| **Alternative Flows** | 1. Student sends empty issue.<br>2. System shows error and asks to write a message. |
| **Business Rules** | Issues and replies are kept for record. |
| **Assumptions** | Notification system is working for teacher and student alerts. |

**Use Case 9**

*Table A- 9: Use Case 9*

| | |
|---|---|
| **Use Case ID** | UC-9 |
| **Use Case Name** | Notifications |
| **Actors** | Teacher, Student |
| **Description** | User views system notifications about important events. |
| **Trigger** | User clicks on “Notifications” icon. |
| **Preconditions** | 1. User is logged in.<br>2. Some events already happened like idea submit, issue submit, feedback etc. |
| **Postconditions** | Notifications are shown and can be marked as read. |
| **Normal Flow** | 1. User logs in.<br>2. User clicks on “Notifications”. |

| | |
|---|---|
| | 3. System shows a list of notifications.<br>4. User clicks on notification to open related page (idea, issue). |
| **Alternative Flows** | System shows message “No notifications”. |
| **Business Rules** | Notifications should show who did what and when. |
| **Assumptions** | Notification data is stored in database. |

# Appendix-B
# Coding Standards

# B. Appendix-B: General Coding Standards & Guidelines

To ensure the maintainability, readability and scalability of the codebase for Project Way, the following general coding standards and guidelines were adopted throughout the development lifecycle.

## 1. Variable Naming Conventions

- Consistent naming id followed throughout the application.
- The following conventions are used base on the context:
  - Snake_case: Used for configuration files and constants.
  - PascalCase: Used for React components and main files(e.g., StudentDashboard, TeacherDashboard).
  - camelCase: Used for variables and functions names(e.g., userName, projectIdeas, updateTaskStatus()).
  - Hungarian Notation: Used occasionally in local variables to clarify type or usage(e.g., arrTasksList (array), btnSubmitIdea (button)).

## 2. Visual Scope Indicators

- Names show what the variable is and how it is used.
- Simple patterns used in Project Way:
  - tempValue for small helper variables used only inside one function(e.g., _tempTasks, _tempFilteredList).
  - is / has for boolean values (true/false).
  - Plural names for arrays/lists, singular for one item.

## 3. Read-Only and Immutable Fields

- For values that never change, we use const to keep them immutable.
- Examples:
  - const API_BASE_URL = “https://api.projectway.com”;
  - const MAX_TASK_LIMIT = 10;
- Props in React components are always treated as read-only; we never change props directly.
- When updating arrays/objects in state, we use spread instead of changing them directly.
- Example:
  - setTasks({tasks, newTask});
  - setUser({user, name: "Ali"});

**4. Function Naming and Behavior**

- All function names clearly describe what they do, using camelCase and starting with a verb.
- Examples in Project Way (JSX):
  - loadStudentProjects()
  - validateLoginForm()
  - submitProjectIdea()
  - updateTaskStatus()
  - fetchTeacherDashboardData()

**5. Commenting Style**

- Comments are added only where logic is not obvious from the code.
- We prefer short, clear comments above complex blocks or conditions.

**6. Indentation and Code Structure**

- A consistent 2-space indentation is used in all JavaScript and JSX files to keep code neat.
- Control Structures such as if, for, while and try-catch always use {} braces even for single-line blocks to avoid mistakes.
- Very deep nesting is avoided; big functions are broken into small helper functions to make code easy to understand.

**7. File and Folder Naming**

- React components are saved in files with the same PascalCase name as the component.
- Examples:
  - StudentDashboard.jsx
  - TeacherDashboard.jsx
  - DiagramEditorPage.jsx
- Folders are used to separate features and layers:
  - components/: Small reusable UI parts (e.g., TaskCard, ProjectStatusBadge).
  - pages/: Full pages (e.g., StudentDashboard, AdminDashboard).
  - services/: API and logic files (e.g., projectService.js, authService.js).
  - contexts/: React context files (e.g., AuthContext.jsx).
  - utils/: Helper functions (e.g., dateUtils.js, validation.js).

**8. Component Structure and Separation of Concerns**

- Each React component has one main job like showing the student dashboard or a task list.
- UI work (showing data, handling clicks, forms) stays inside the component.
- Business work (API calls, calculations) goes in separate service or helper files.

- For bigger pages, data fetching and state can be moved into custom hooks or services (like useStudentProjects, projectService) so components stay simple.

**9. Access and Scoping**

- Variables and functions are kept in the smallest scope needed (inside a function or component) so they are not visible everywhere.
- Only the main component or function needed in other files is exported (usually with export default); helper functions stay inside the same file.
- Sensitive data (like API, URLs) is stored in environment variables, not hard-coded in frontend code.

**10. Cohesion and Coupling**

- High cohesion: Related code stays together (project logic in projectService.js, task UI in TaskCard.jsx).
- Low coupling:
  - Components share data through props instead of directly depending on each other.
  - Services/hooks hide API details from components.
  - Each feature (students, teachers, admin, diagrams) has its own folder to reduce side effects.

**11. DRY Principle (Don't Repeat Yourself)**

- Same logic is not written again and again; we reuse code.
- Shared logic goes into helper/service files like validation.js, projectService.js, authService.js.
- Common UI patterns (buttons, cards, forms) are made as reusable React components.

**12. Single Responsibility Principle (SRP)**

- Each function does one clear task only.
- Each component/file has one main responsibility (for example, StudentDashboard shows student dashboard, TaskList shows tasks).
- Features are split so that changes in one responsibility (like project upload or issue reporting) do not affect others unnecessarily.

**13. Extensibility and Maintainability**

- Components are designed so new features can be added without changing a lot of existing code (Open/Closed Principle).
- Common behavior is placed in reusable helpers/services, so we can extend the app by adding new components or functions instead of editing old ones everywhere.

## 14. Exception Handling and Logging

- Risky code (like API calls) is wrapped in try-catch to avoid app crashes.
- Errors are logged with clear messages using console.error() on frontend and logging in backend (e.g., inside Express error handlers).
- Log messages include where the error happened and a short reason, for example: console.error("ProjectModule: Failed to load projects", error).

# Appendix-C
# Prototype

# C. Appendix-C: Application Prototype

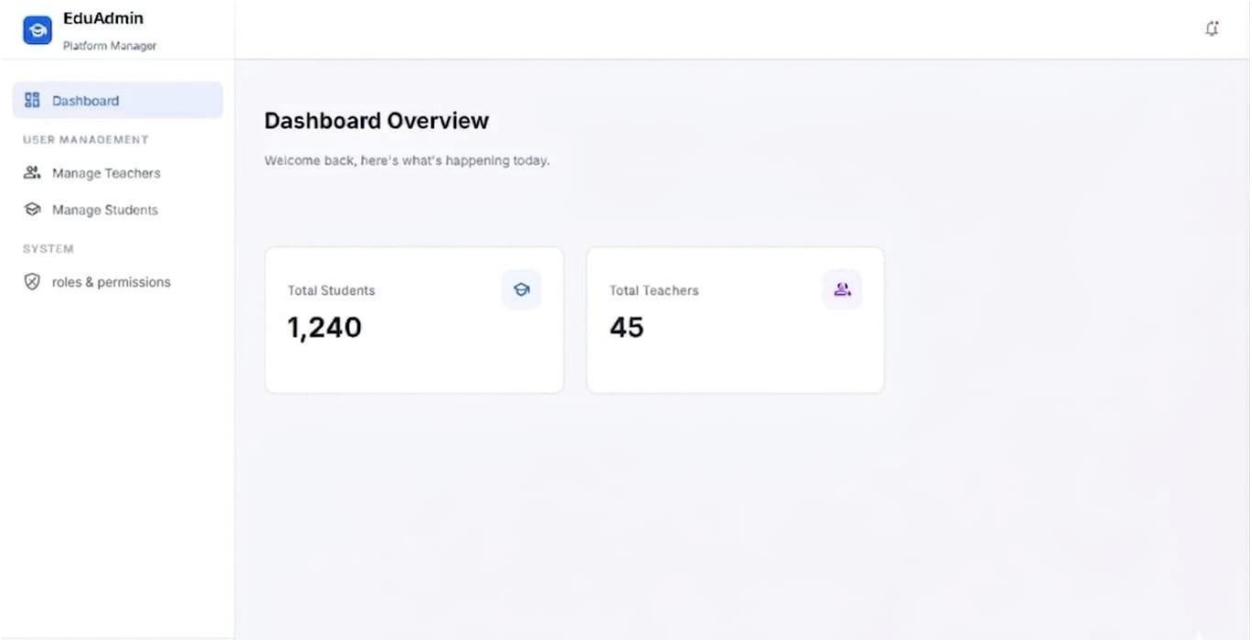



*Figure 40: Screen Image 1*

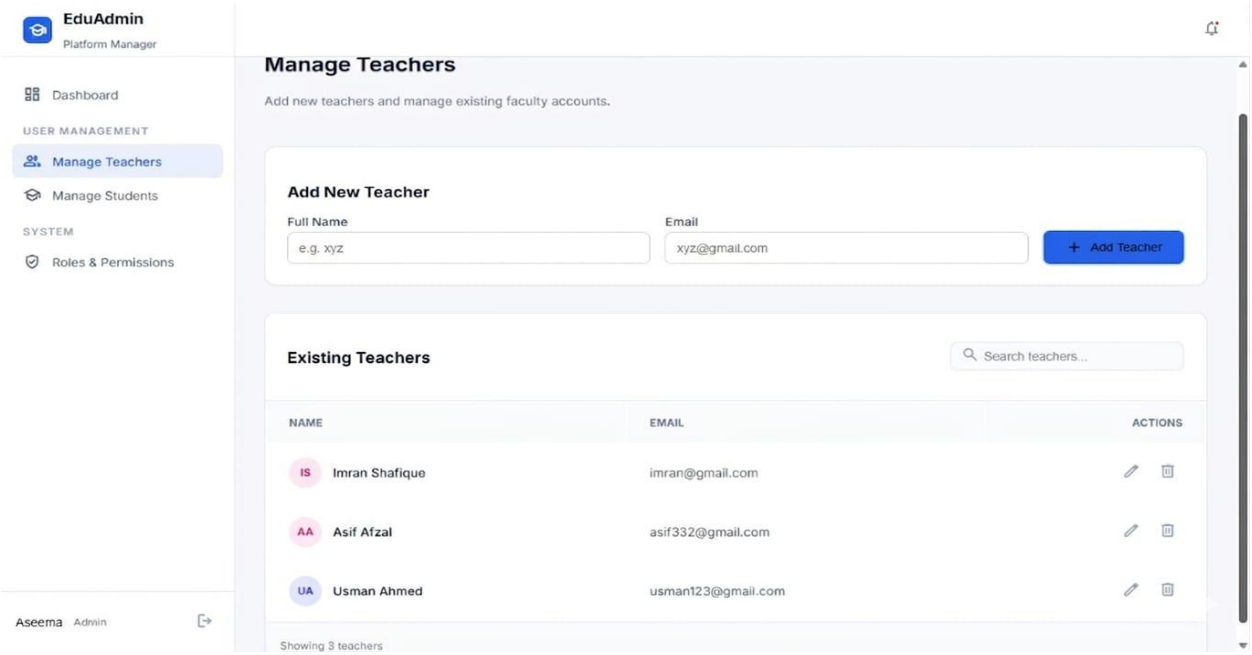



*Figure 41: Screen Image 2*

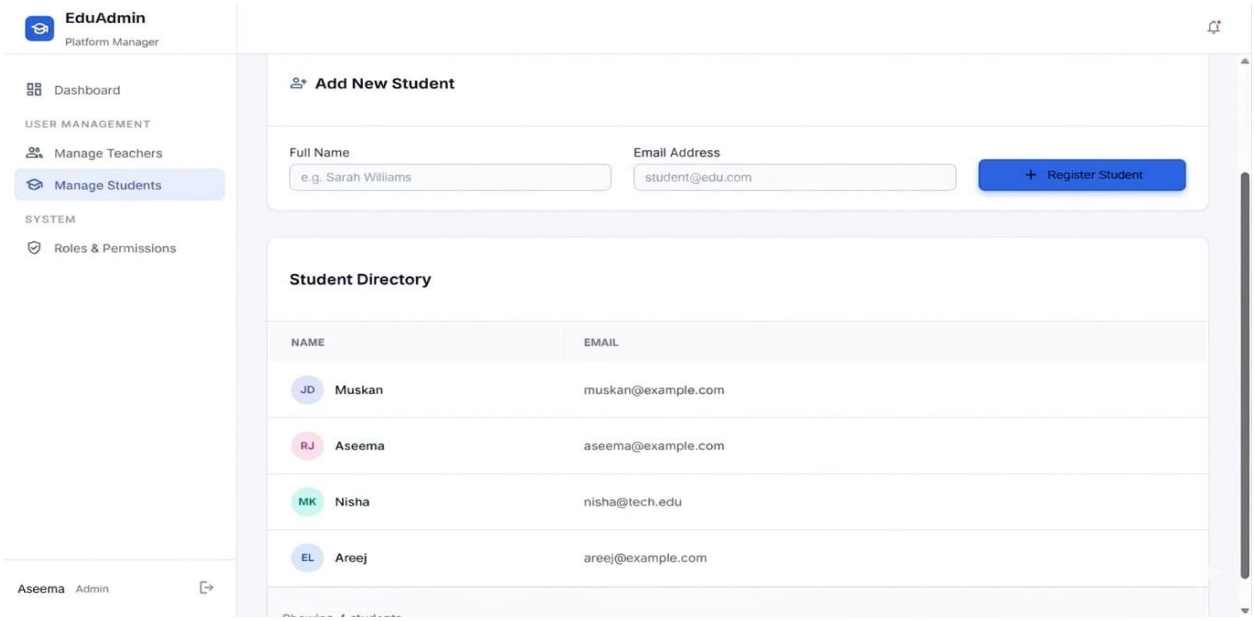



*Figure 42: Screen Image 3*

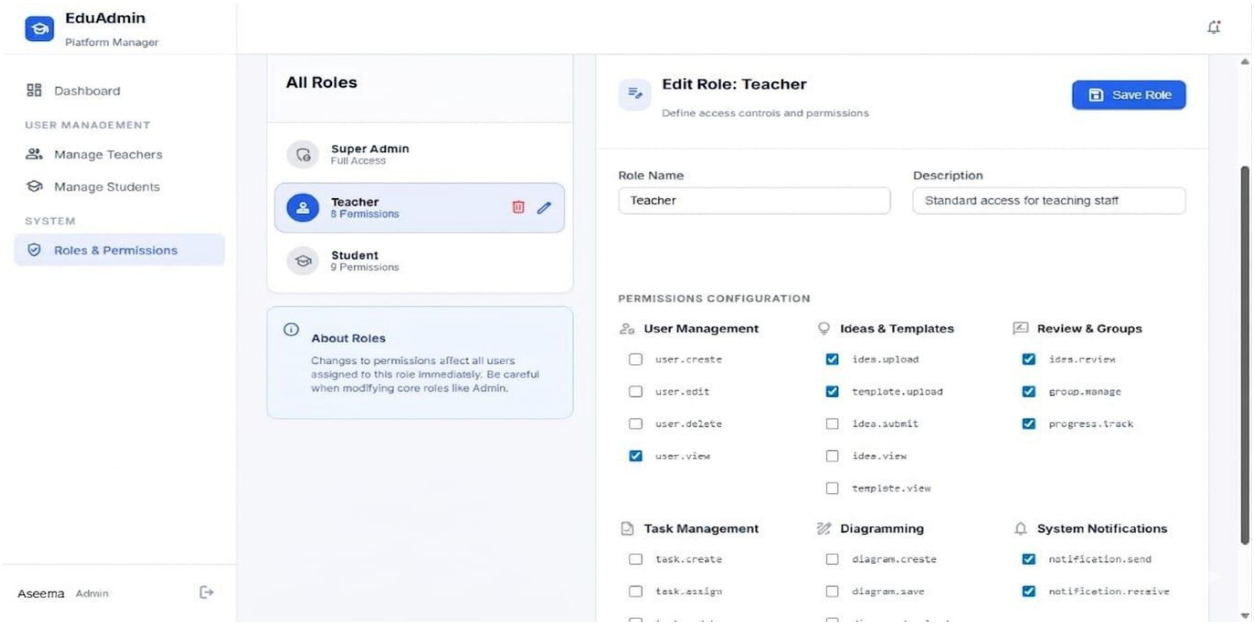



*Figure 43: Screen Image 4*

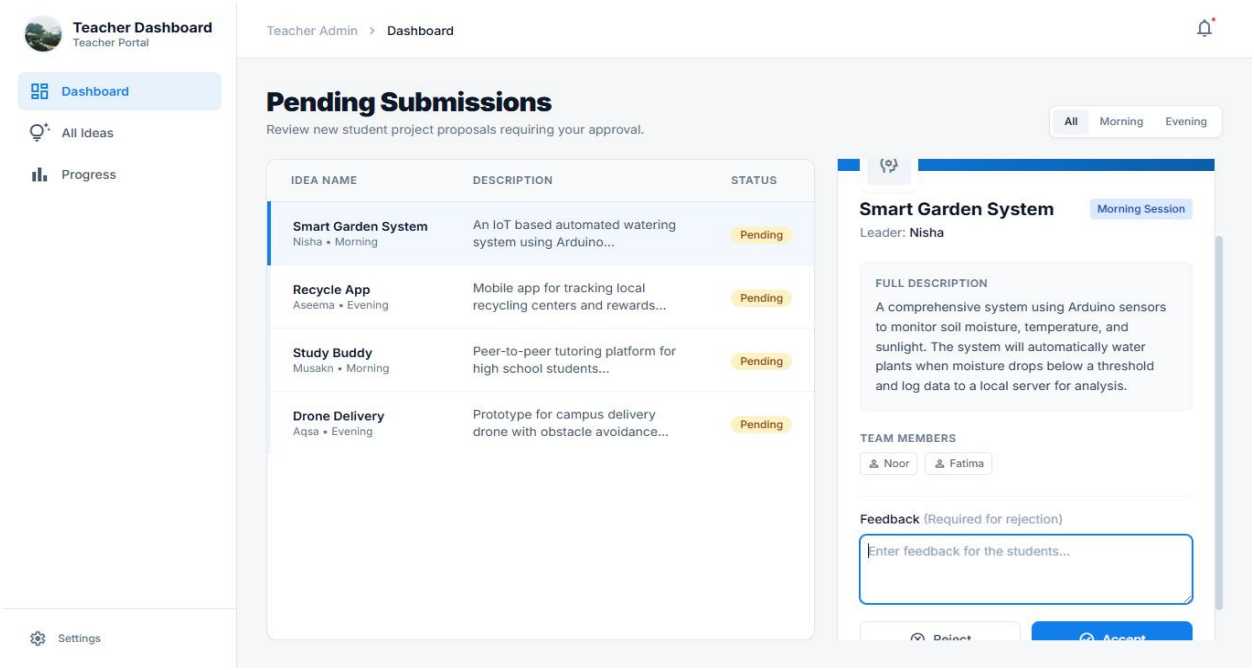



*Figure 44: Screen Image 5*

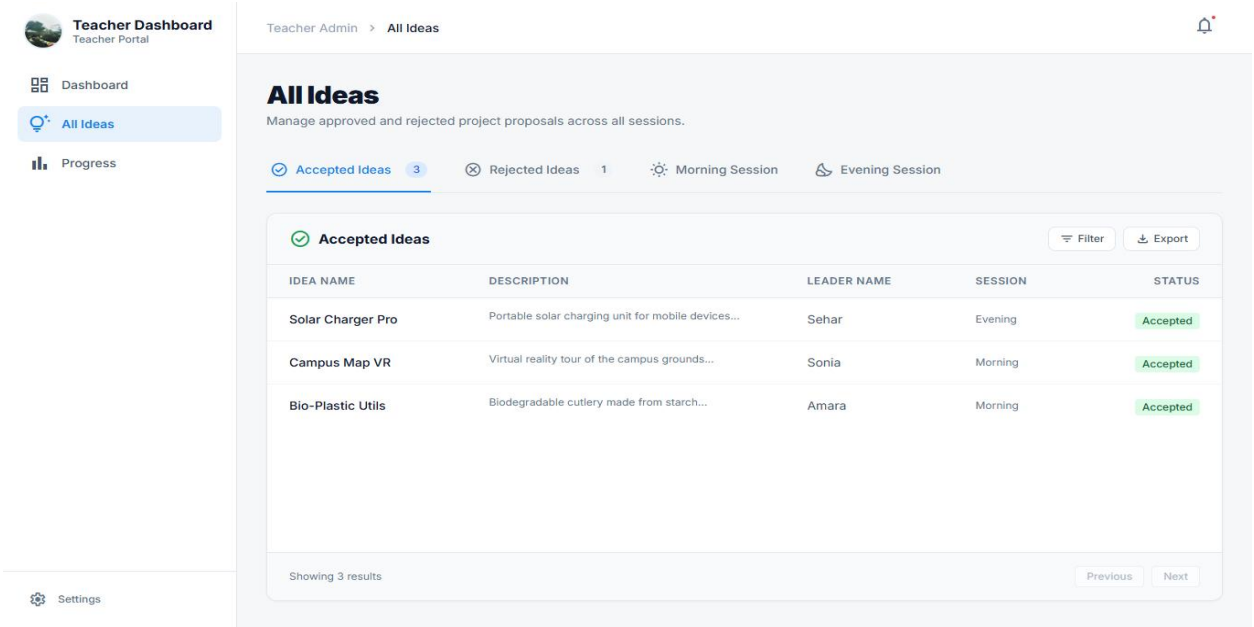



*Figure 45: Screen Image 6*

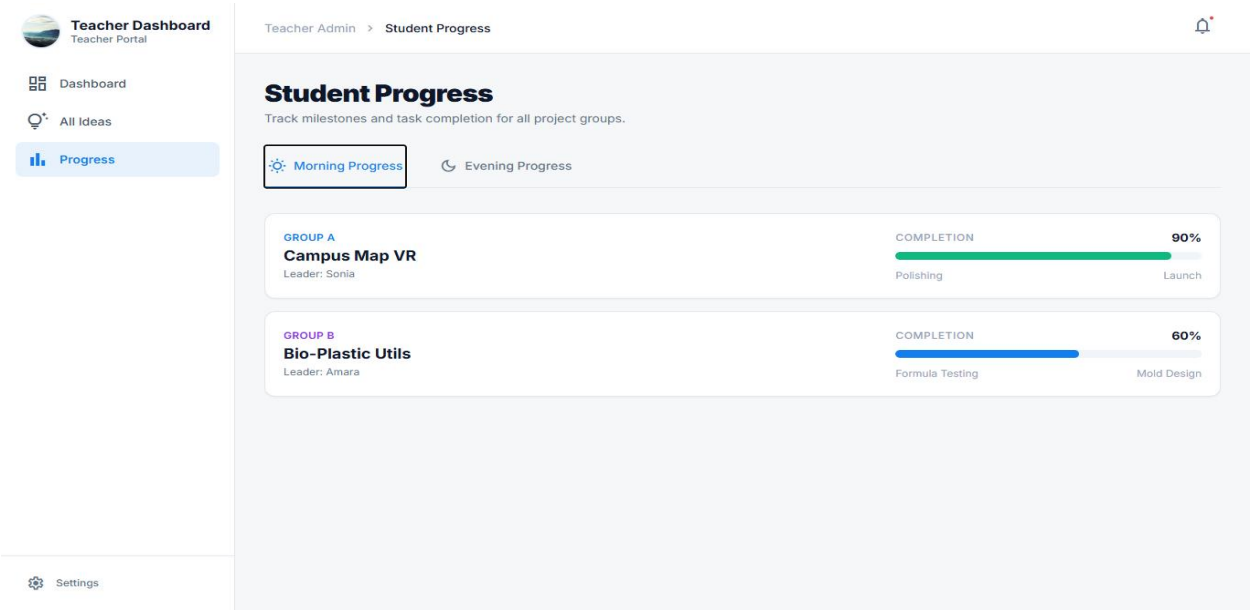



*Figure 46: Screen Image 7*

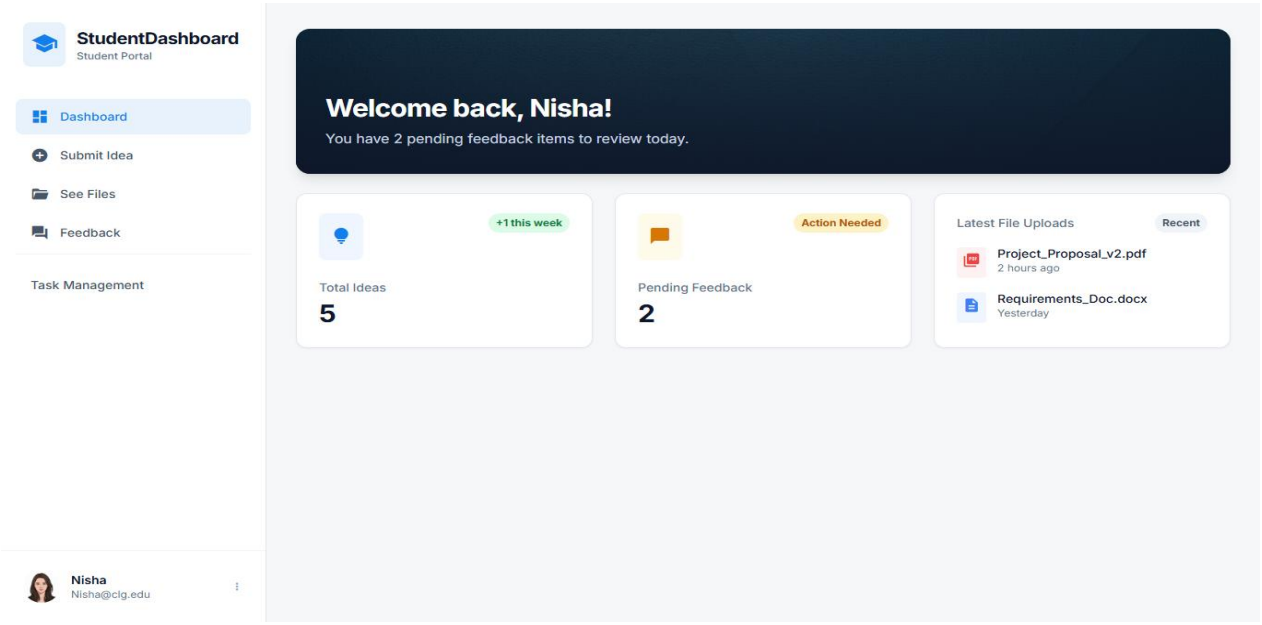



*Figure 47: Screen Image 8*

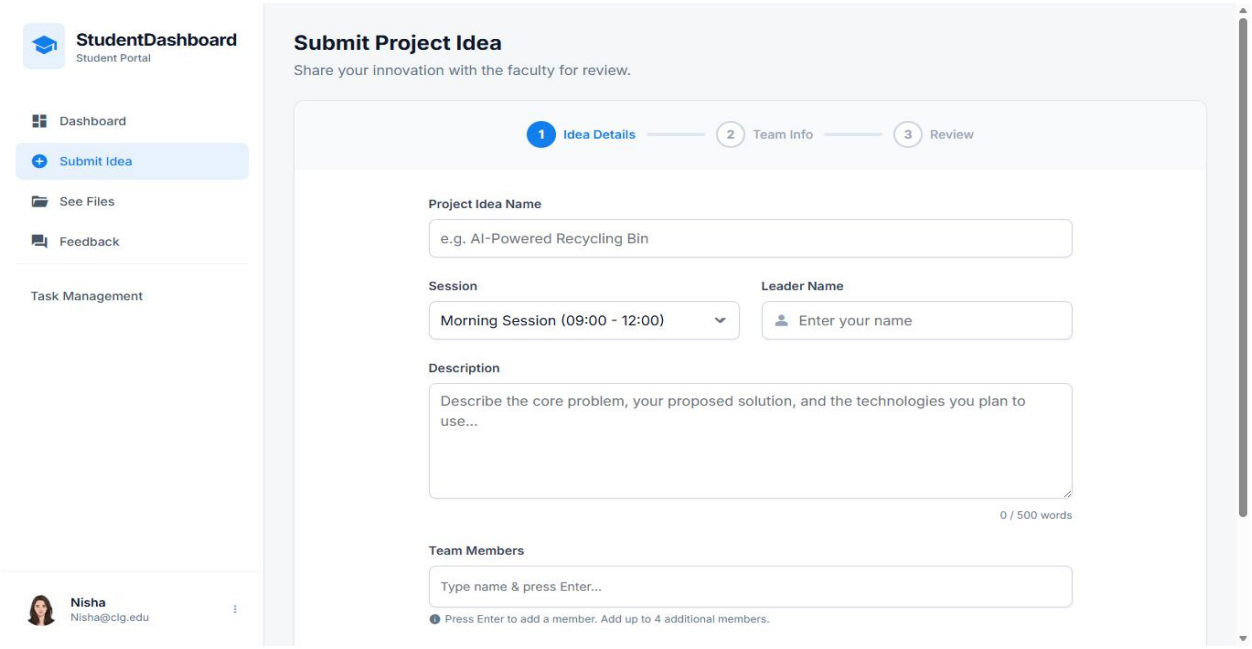



*Figure 48: Screen Image 9*

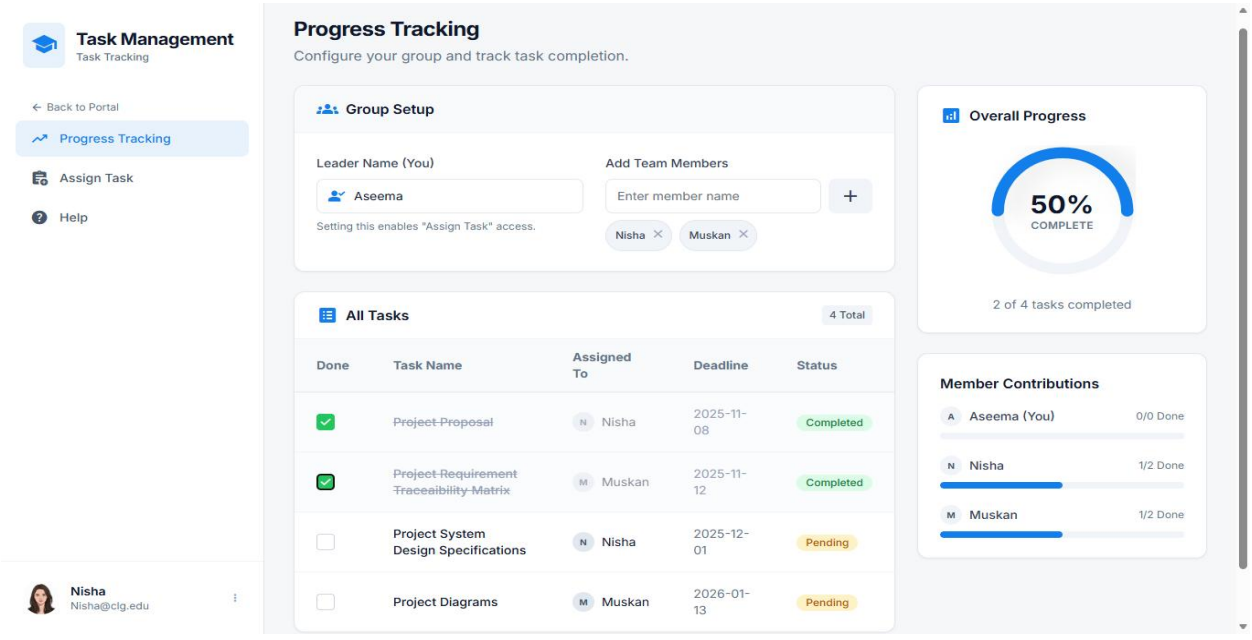



*Figure 49: Screen Image 10*